Engine Installation & Hello World Verification Guide

Instructor reference - get every engine installed and prove each one actually runs

VIDEO GAME CREATION

How to Use This Guide

This is your personal setup reference so your demo laptop always has a working copy of every supported engine - the goal from the original course plan was to be able to demonstrate any of them on request, and to never be caught by a build that silently stopped working. For each engine below: follow the Detailed Install steps once, then run the Hello, World! Verification test to prove the full pipeline works end-to-end (editor opens, code/logic runs, output appears). Re-run the Hello World test after any engine update, OS update, or before the start of each semester.

These steps are also the backbone of Lesson 1's lab time and the per-lesson Teacher's Checklists - Level 1 for Lesson 1 is exactly this: a working install and a confirmed Hello World.

Per-Engine Setup

1. Scratch

Detailed Install

1. Go to scratch.mit.edu/download - this opens the "Download the Scratch App" page.
2. Under Choose your OS, confirm Windows (or macOS) is selected, then click Get it from Microsoft - Windows now installs Scratch through the Microsoft Store; macOS still offers a direct .dmg download.
3. On the Microsoft Store listing page, click Download. The browser downloads Scratch 3 Installer.exe into your Downloads - click Open file in the download flyout to run it.
4. A small Microsoft Store window confirms the install and shows a download/progress bar. Let it finish - it's a fairly large app (~265 MB).
5. Scratch launches automatically once installed. On first launch, a one-time dialog asks whether to share usage data with the Scratch Team - choose either option and click Close.
6. Confirm the editor opens to a blank project with the cat sprite on stage.

Hello, World! Verification

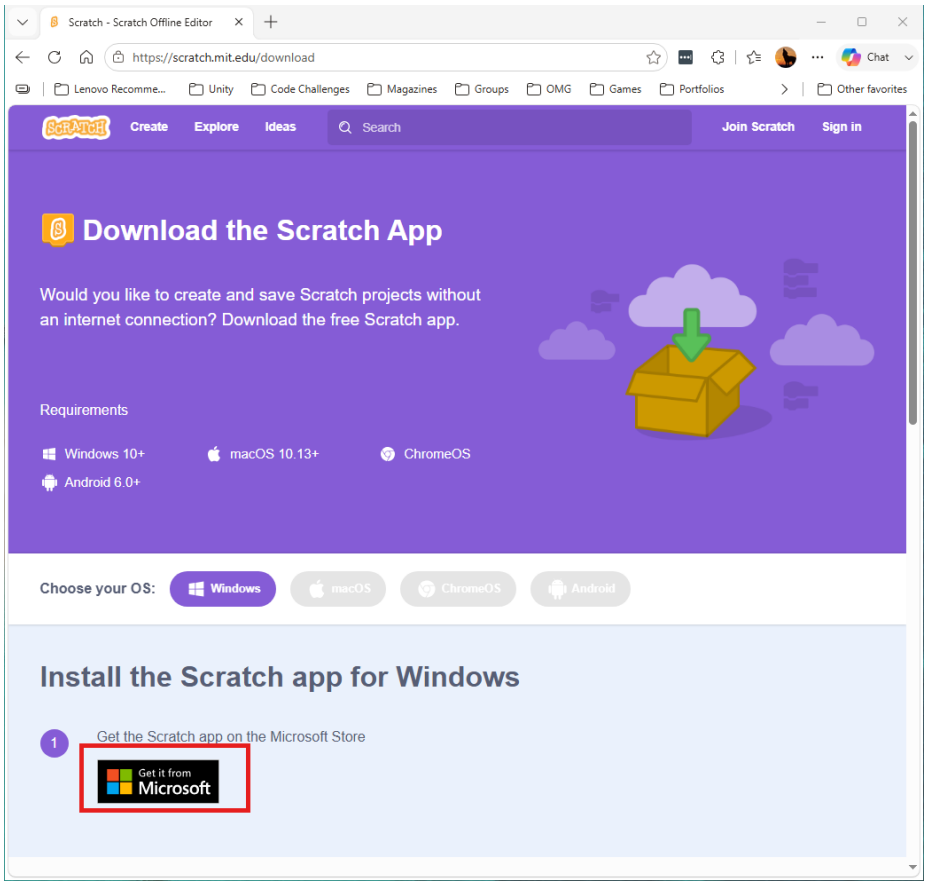
1. With Scratch open to a blank project, click the Events category in the block palette (left side).
2. Drag "when green flag clicked" into the empty code area to the right of the palette.
3. Click the Looks category, then drag "say Hello! for 2 seconds" so it snaps directly underneath the when-clicked block.
4. Click into the say block's text field (it highlights blue) and change the text to "Hello, World!"
5. Click the green flag above the stage (top-right of the stage area).
6. Optional: File > Save to your computer, name it HelloWorld, and click Save to keep a copy of the project (.sb3 file).

Expected result: The cat sprite shows a speech bubble reading "Hello, World!" for 2 seconds, and the script glows yellow while it runs.

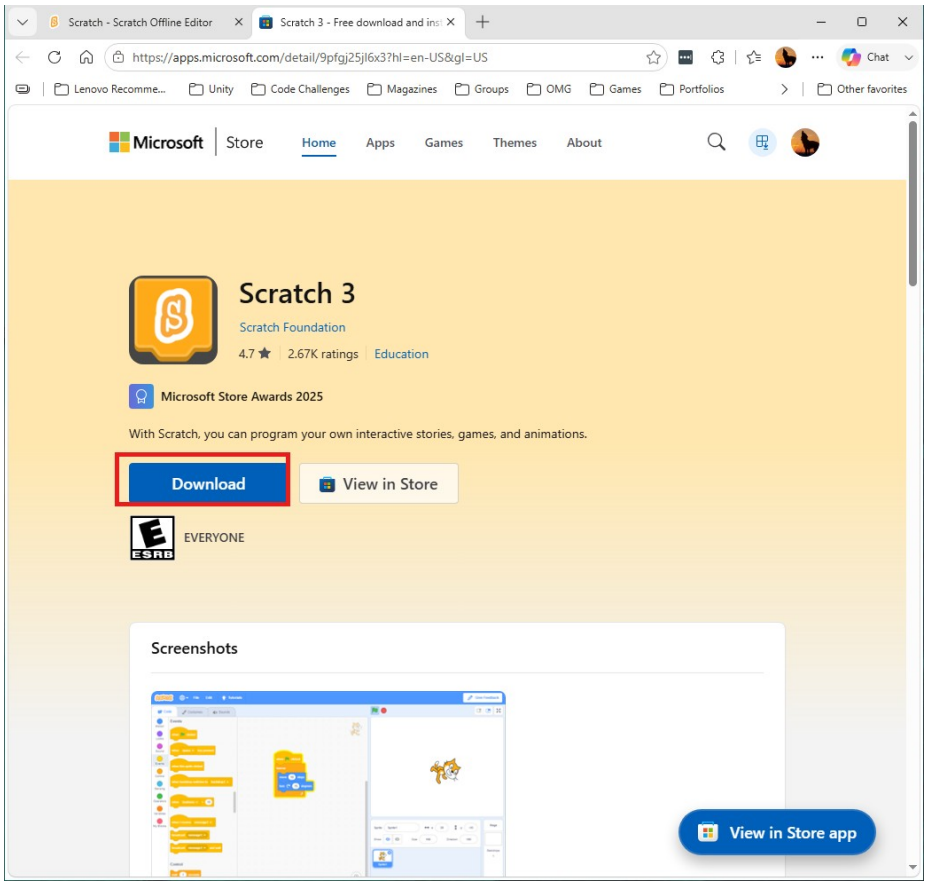
If something's wrong: *If the Microsoft Store install seems stuck, check your internet connection - it's a large download for a first-run app. If Scratch won't launch, confirm you're on Windows 10+ or macOS 10.13+; the browser version at scratch.mit.edu (click Create) works as a backup with no install at all.*

Visual Walkthrough (captured on Scratch 3.32.0)

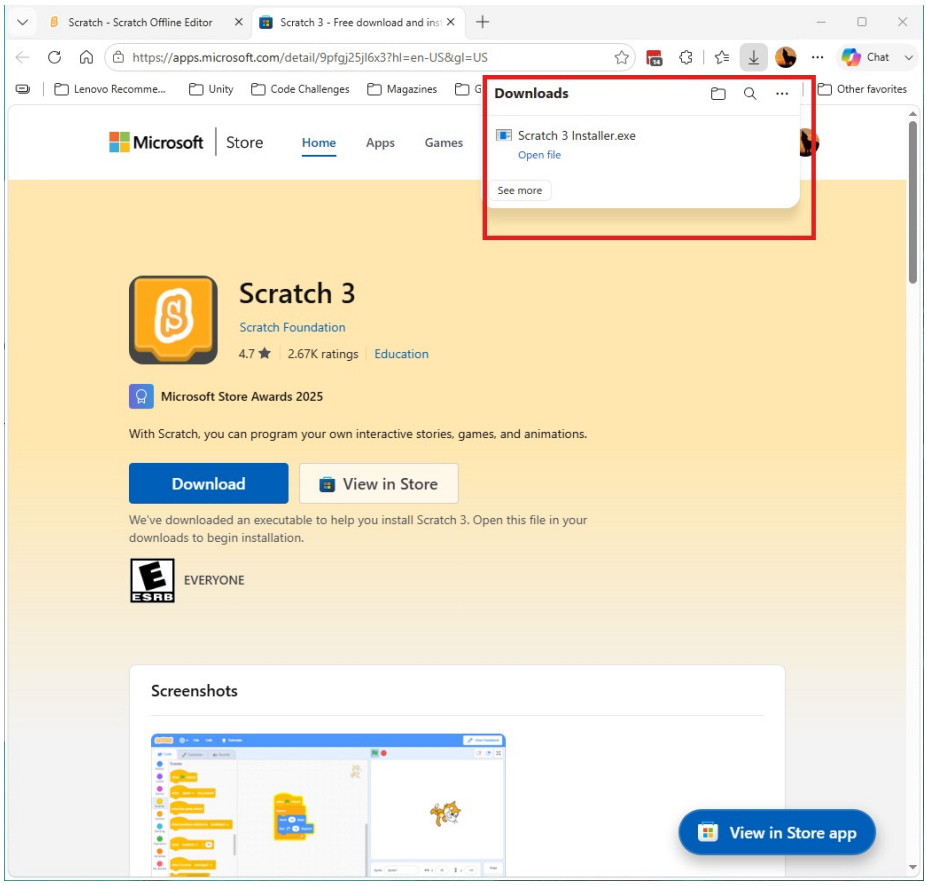
Screenshots from an actual run-through, in order. Circled/boxed elements mark exactly what to click or check at that step.



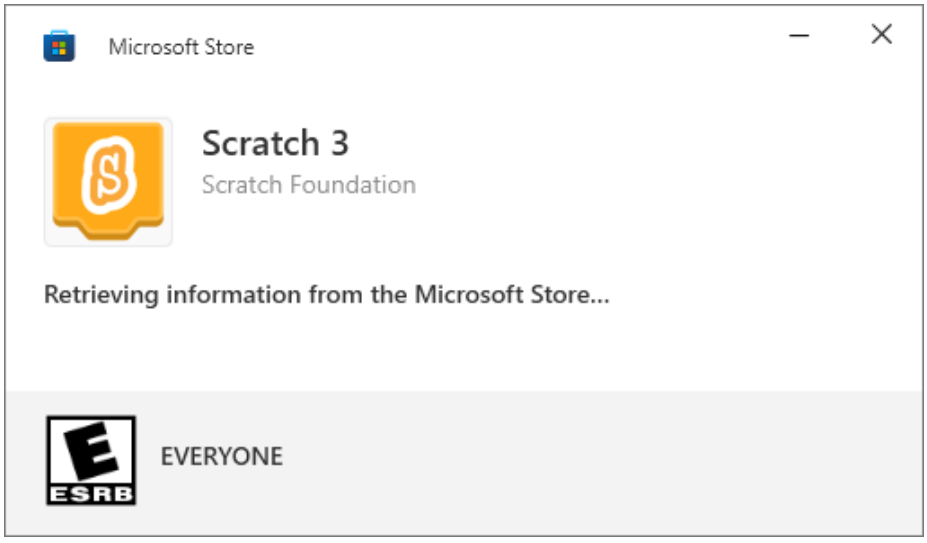
The Download page - Windows is selected; click Get it from Microsoft.



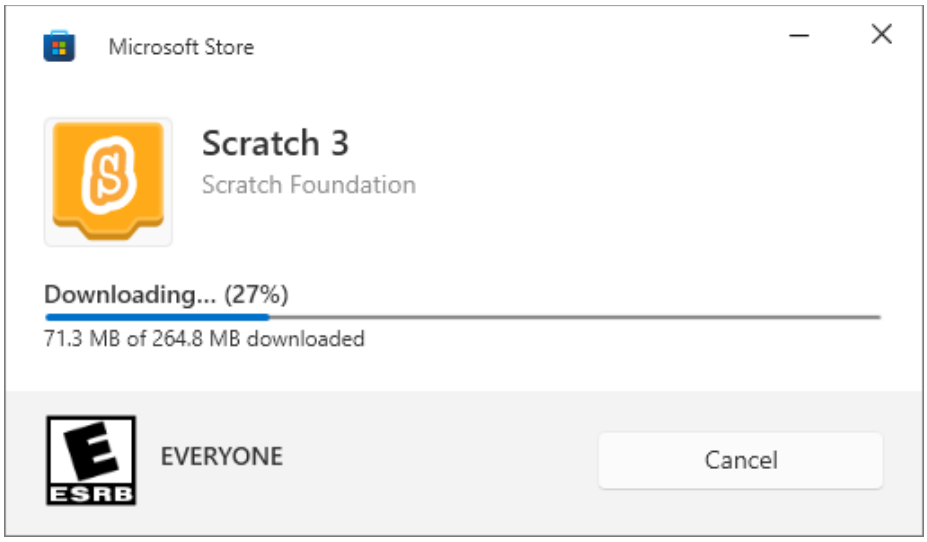
The Microsoft Store listing for Scratch 3 - click Download.



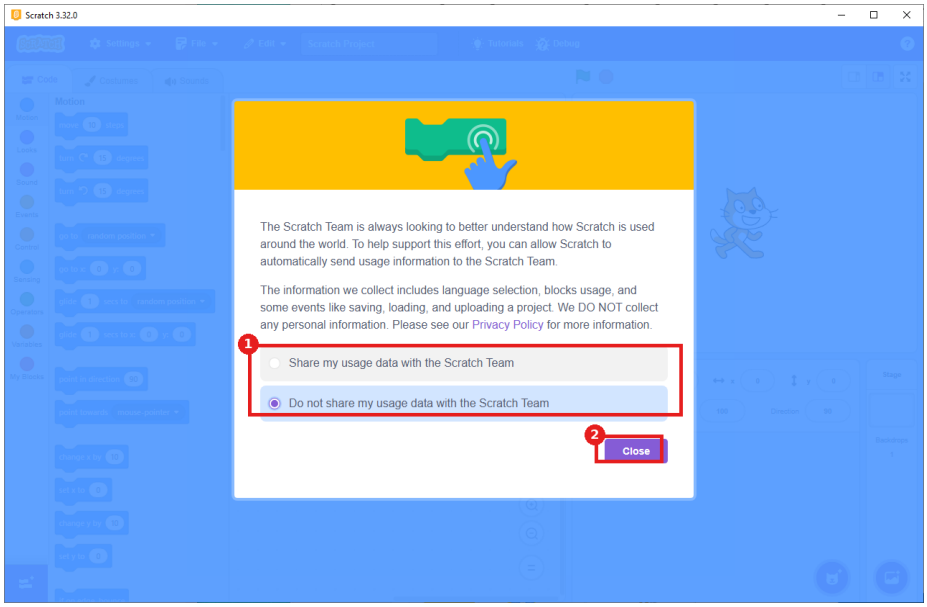
The installer downloads as Scratch 3 Installer.exe - click Open file.



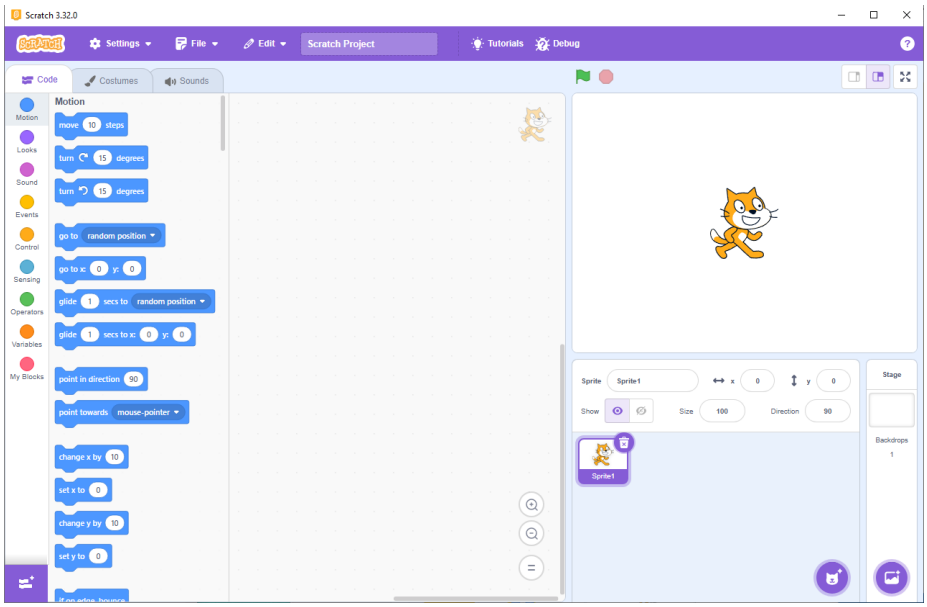
The Microsoft Store install dialog appears.



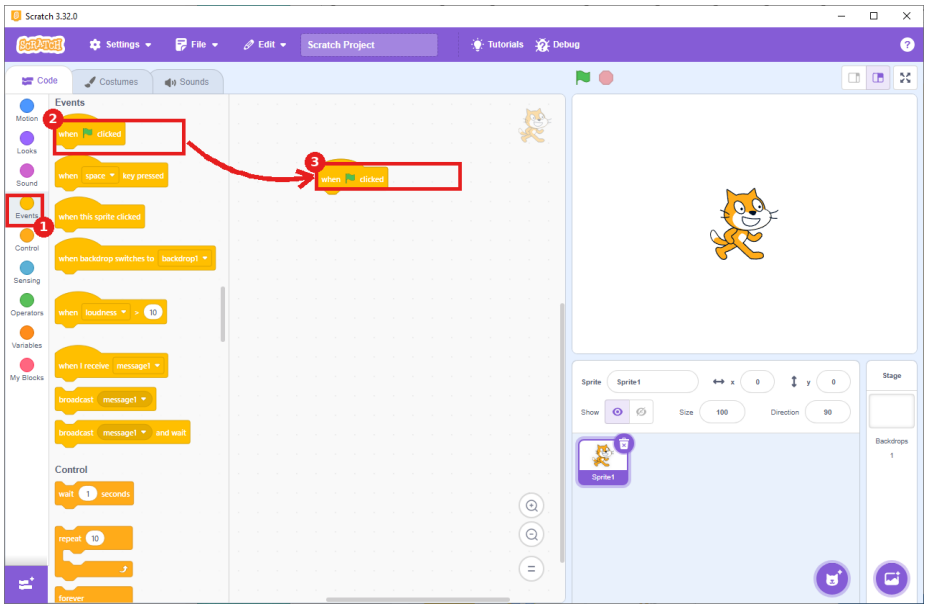
Download and install in progress (~265 MB).



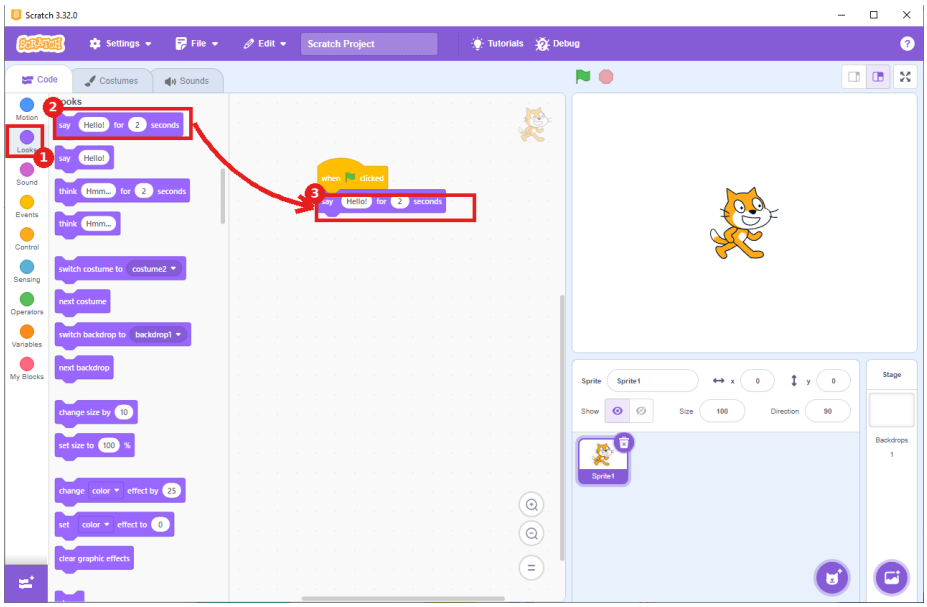
First launch: 1) choose whether to share usage data, 2) click Close.



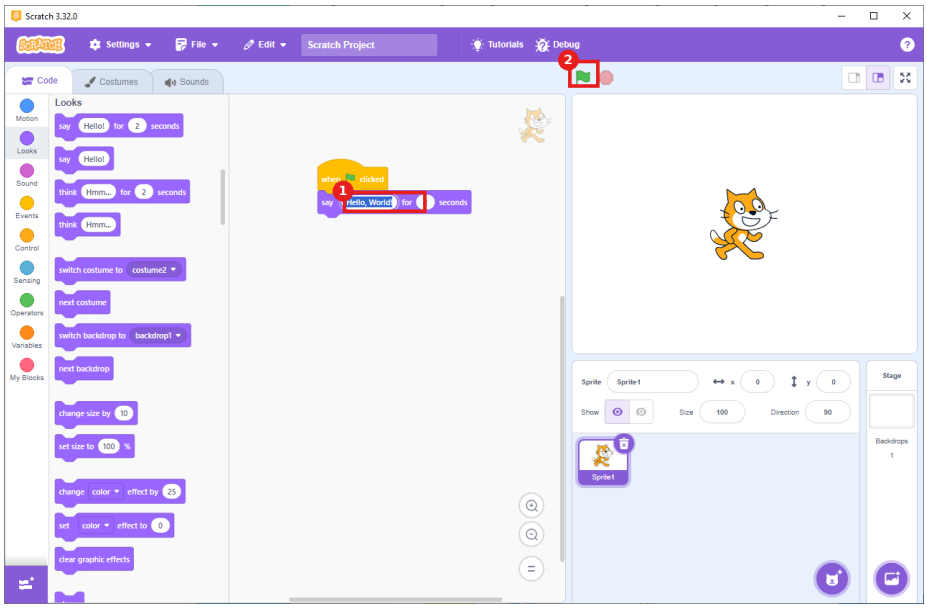
A blank project opens with the cat sprite on stage, ready to go.



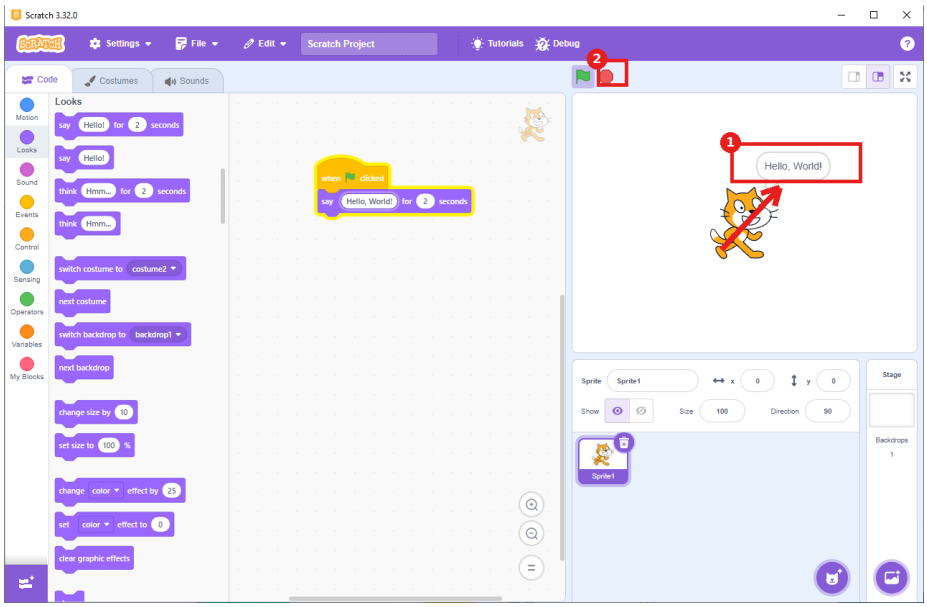
1) Click Events in the left navigation, 2) drag "when green flag clicked" from the palette, 3) into the code area (arrow shows the drag).



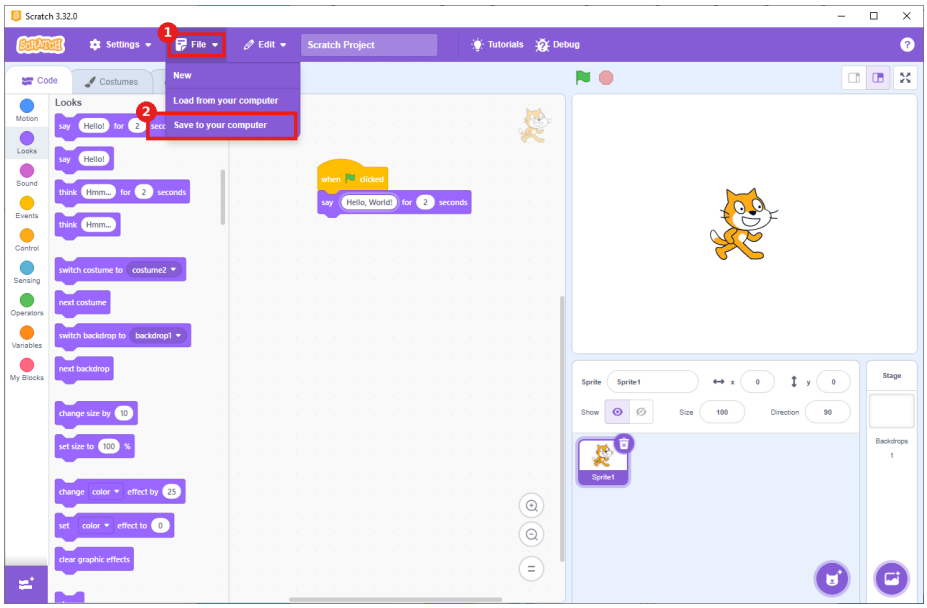
1) Click Looks in the left navigation, 2) drag "say Hello! for 2 seconds", 3) snap it under the when-clicked block (arrow shows the drag).



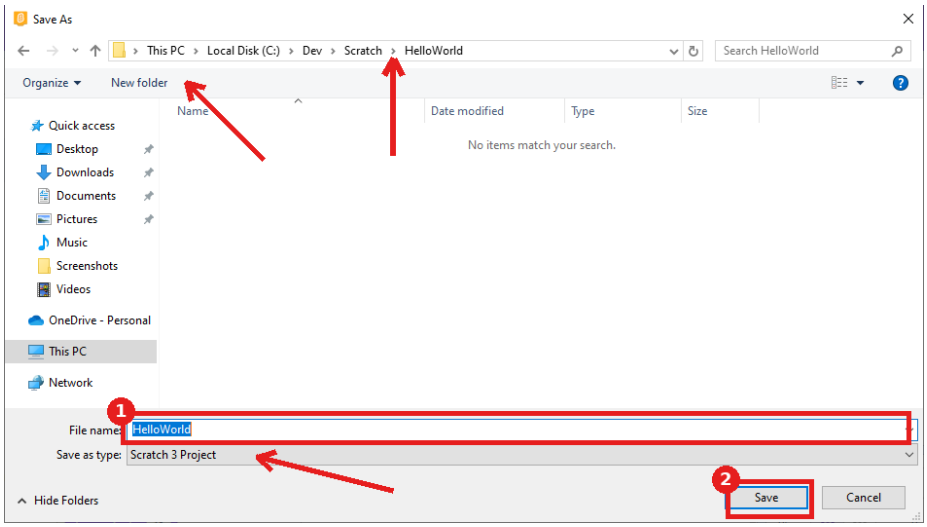
1) Click into the say block's text field and change it to "Hello, World!", 2) then use this green flag to run it.



Success - 1) the speech bubble reads "Hello, World!", 2) the stop button confirms the script ran.



Optional: 1) click File, 2) choose Save to your computer.



1) Name the file HelloWorld, 2) click Save. Arrows mark the folder path, New folder button, and Save as type field.

2. Godot Engine

Detailed Install

1. Go to godotengine.org/download - this opens the Godot 4 download page for your OS.
2. Click the Godot Engine button (the version badge, e.g. 4.7.1, confirms the release) - skip the "Godot Engine - .NET" button unless you plan to demo C# specifically.
3. The browser downloads a .zip file (e.g. Godot_v4.7.1-stable_win64.exe.zip).
4. In File Explorer, find the zip in your Downloads folder, right-click it, and choose Extract All...
5. Confirm the destination folder (the default, right next to the zip, is fine) and click Extract.
6. Extraction produces two files: a standard executable and a matching _console version (which also opens a background terminal window) - you only need the standard one.
7. Optional: drag the standard executable (not the _console one) to your Desktop for a shortcut you can double-click anytime.
8. Double-click the standard Godot executable to launch - there is no installer.
9. If Windows SmartScreen warns "Windows protected your PC," click More info, then Run anyway (expected for unsigned indie software).

Hello, World! Verification

1. In the Project Manager window, click Create New Project.
2. Set Project Name to Hello World and Project Path to a folder of your choice - the Renderer and Version Control Metadata defaults are both fine to leave (or set Version Control Metadata to None if you don't plan to use Git).
3. Click Create - since Edit Now is checked by default, the editor opens automatically afterward.
4. The first time a project opens, Godot spends a few seconds (Re)Importing Assets - this is normal, just wait for it to finish.
5. In the Scene panel, the empty project automatically prompts you to choose a root node type - click Other Node.

6. In the Create New Node dialog, type Node2D into the Search bar, select it from the list, and click Create.
7. With the Node2D root selected, scroll to the bottom of the Inspector panel (right side) to the Script field, click its dropdown arrow, and choose New Script...
8. In the Attach Node Script dialog, keep all the defaults and click Create.
9. The script opens with a default _ready() function containing a placeholder pass line - replace it with the code below.
10. Notice the unsaved indicators: the editor tab, the main title bar, the script editor's own title bar, and the script's entry in the Filter Scripts list all show an asterisk (*) - press Ctrl+S to save the script.
11. Press F6 (Run Current Scene). Since the scene has never been saved, Godot prompts you to save it first - keep the default filename (e.g. node_2d.tscn) and click Save.

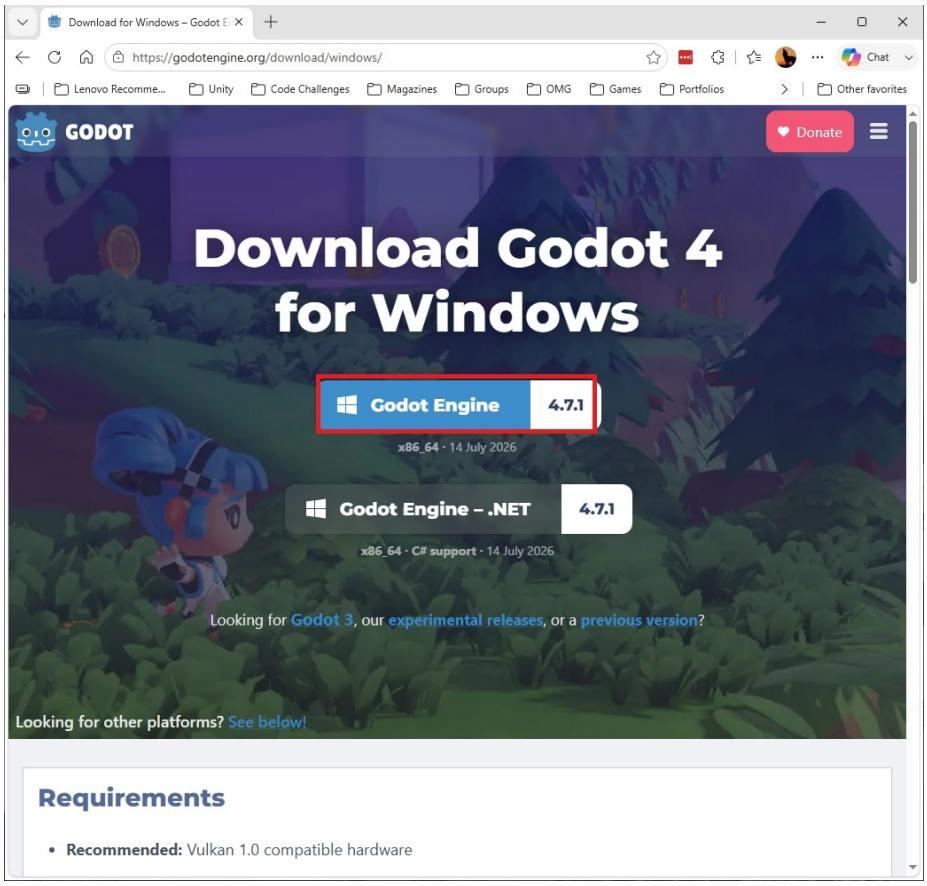
```
func _ready():
    print("Hello, World!")
```

Expected result: "Hello, World!" appears in the Output panel at the bottom of the editor. A small blank "Hello World (DEBUG)" game window also pops up - that's expected, since this exercise only prints text and doesn't draw anything to the screen. You may need to move or close that window to see the Output panel underneath it.

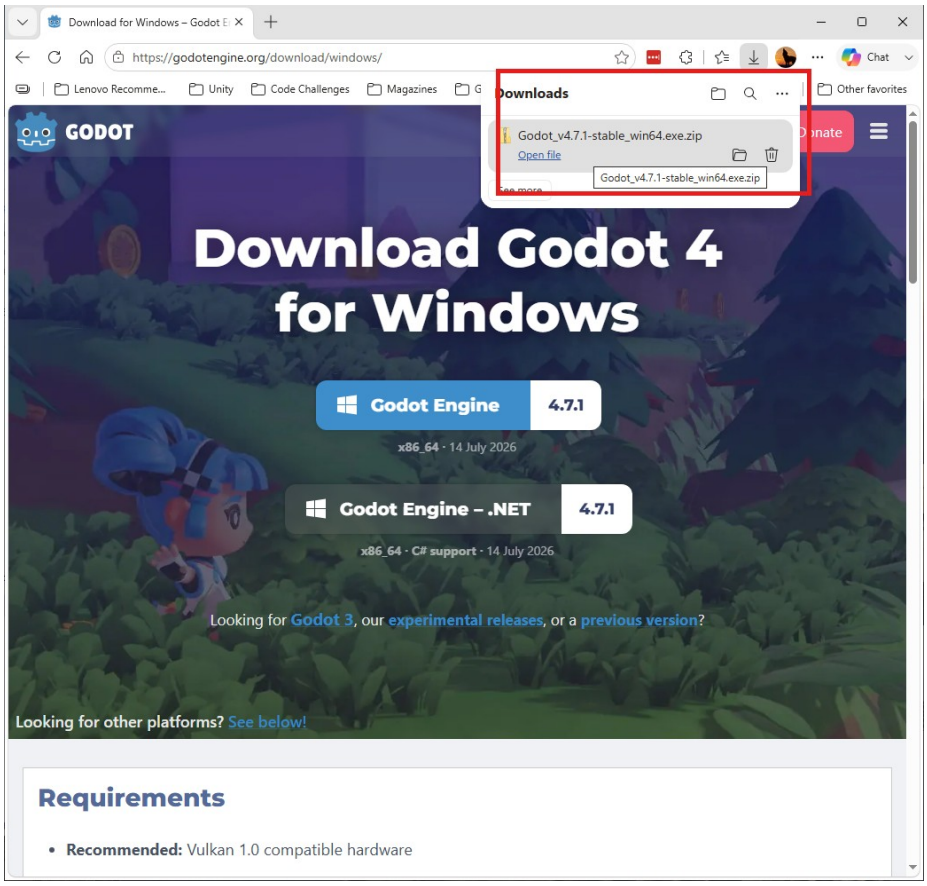
If something's wrong: *If nothing shows in Output, confirm the scene was actually saved before running (the asterisks from the step above should be gone), and that the script is attached to the Node2D that's the scene's root.*

Visual Walkthrough (captured on Godot Engine 4.7.1)

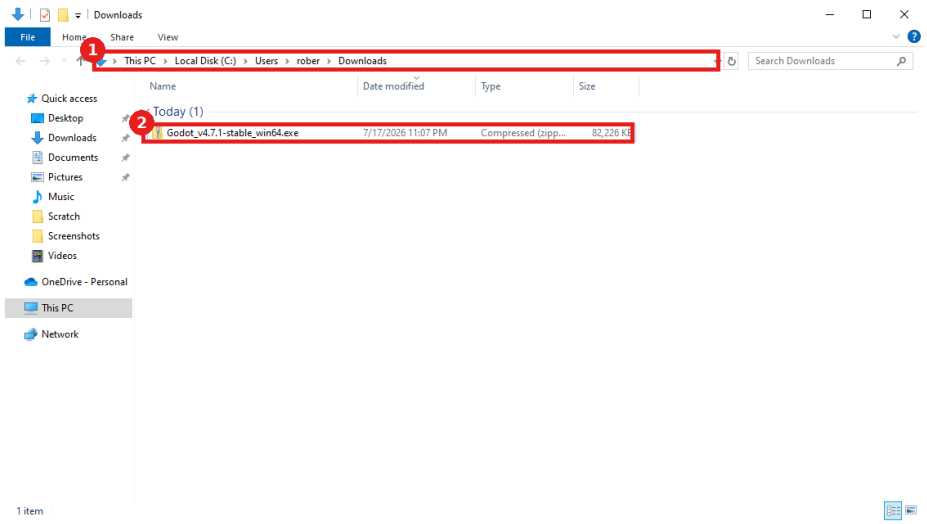
Screenshots from an actual run-through, in order. Circled/boxed elements mark exactly what to click or check at that step.



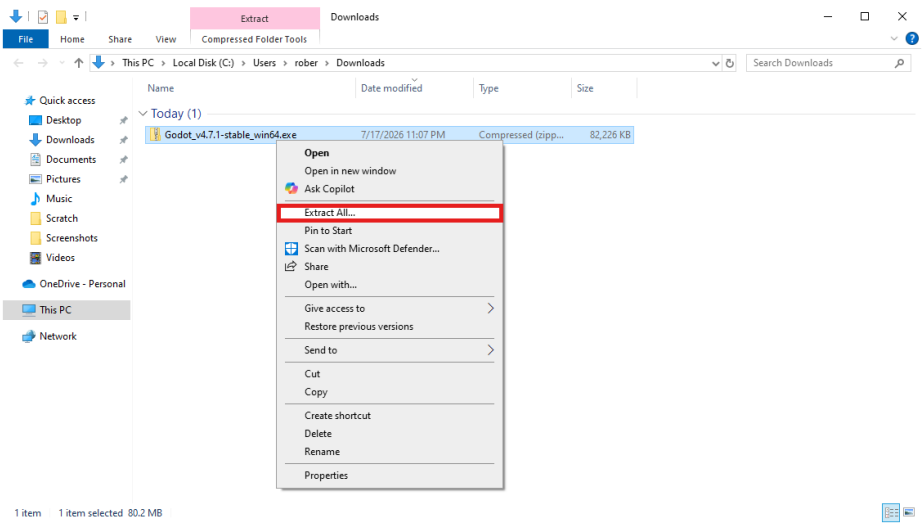
The Godot 4 download page - click the Godot Engine button (not the .NET version).



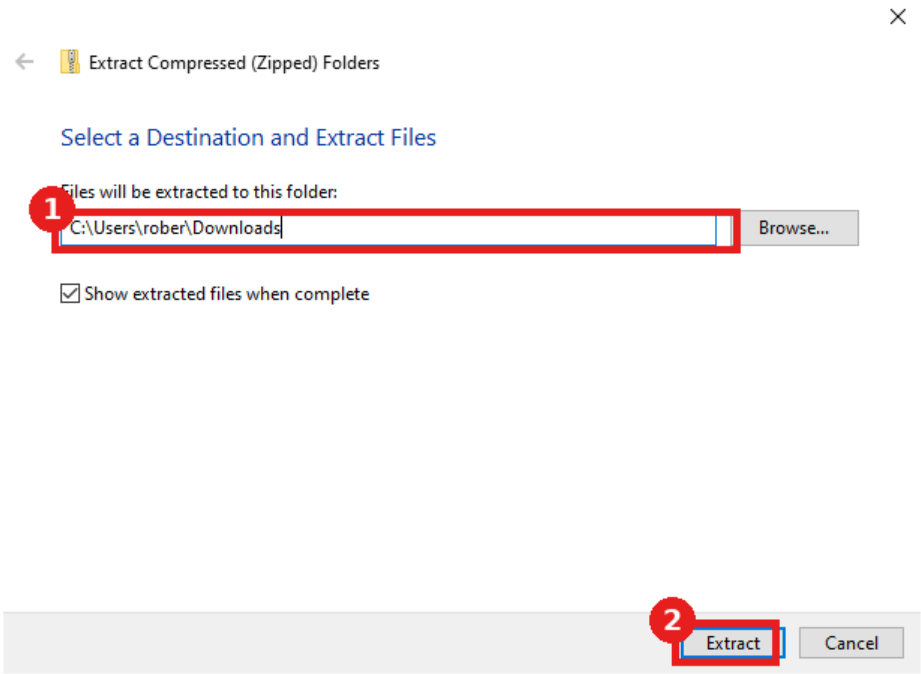
The browser downloads a .zip file.



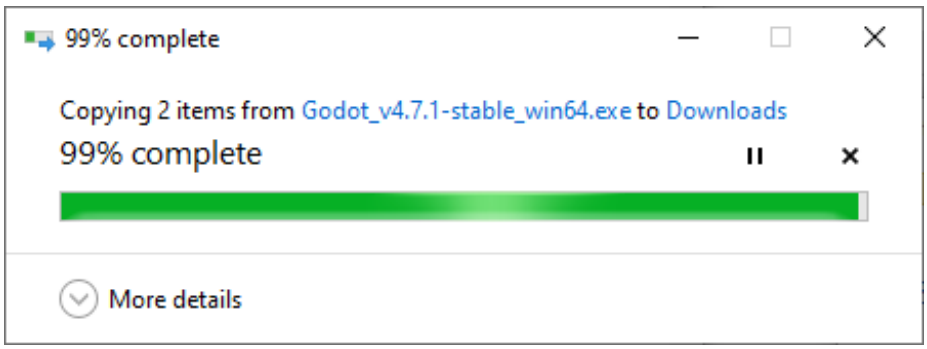
1) The Downloads folder, 2) the downloaded zip file.



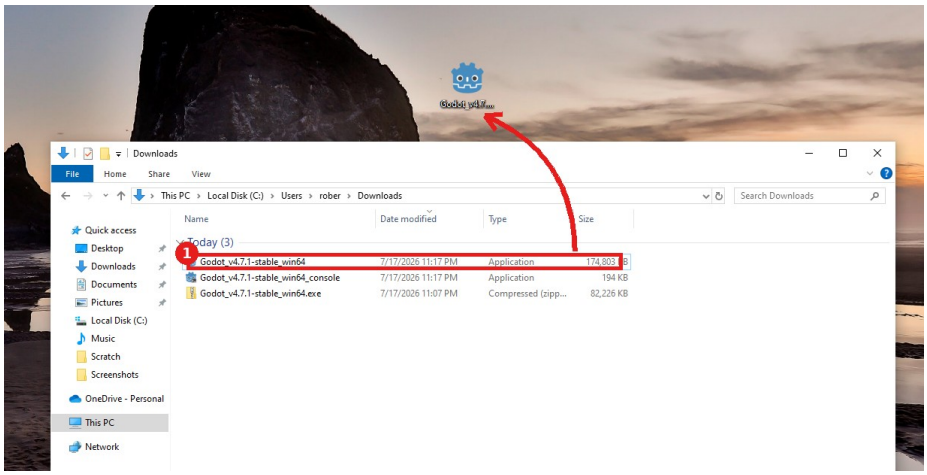
Right-click the zip file and choose Extract All...



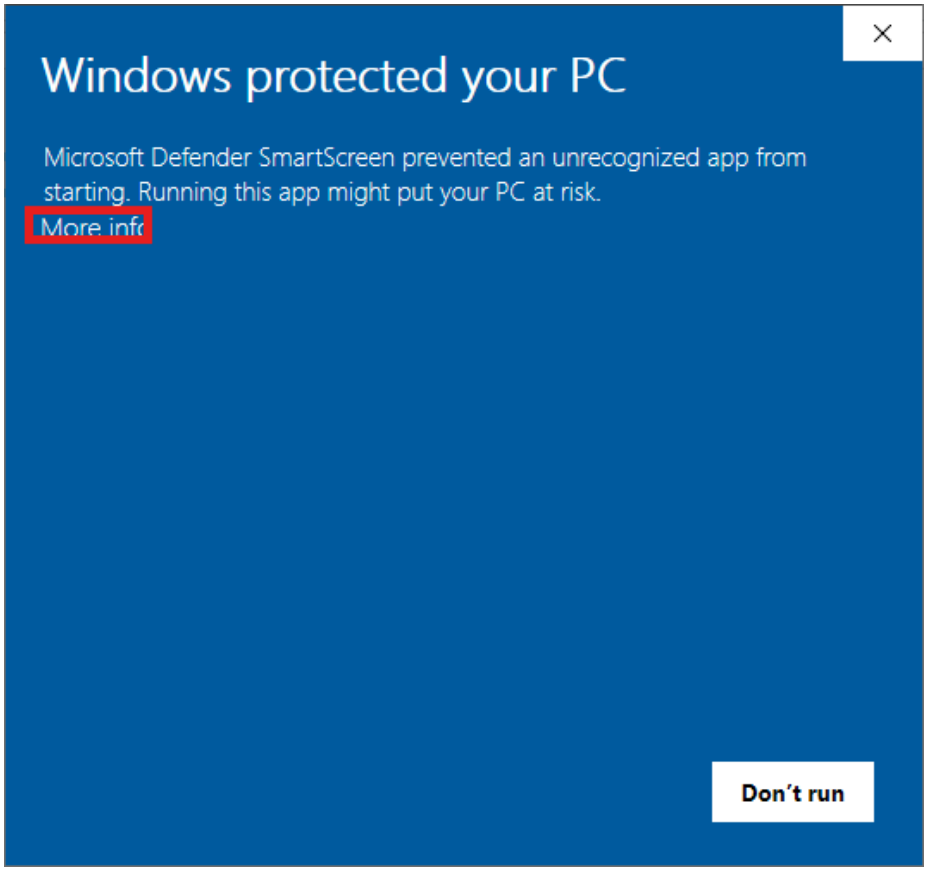
1) Confirm the destination folder, 2) click Extract.



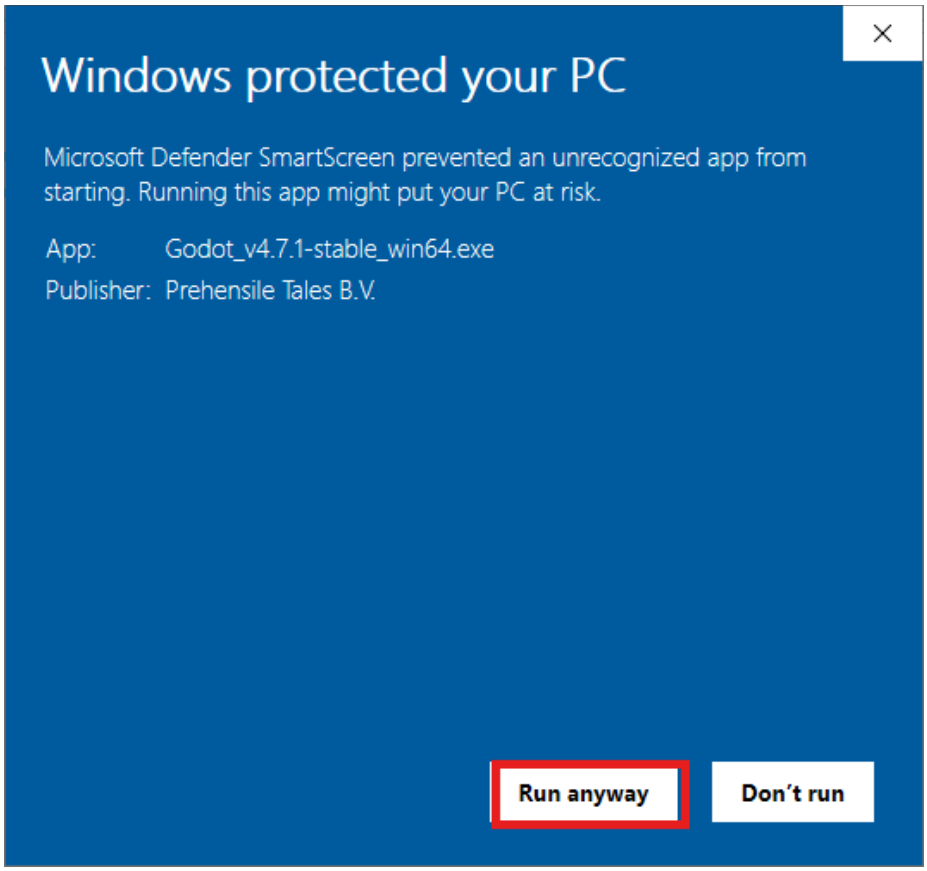
Extraction in progress.



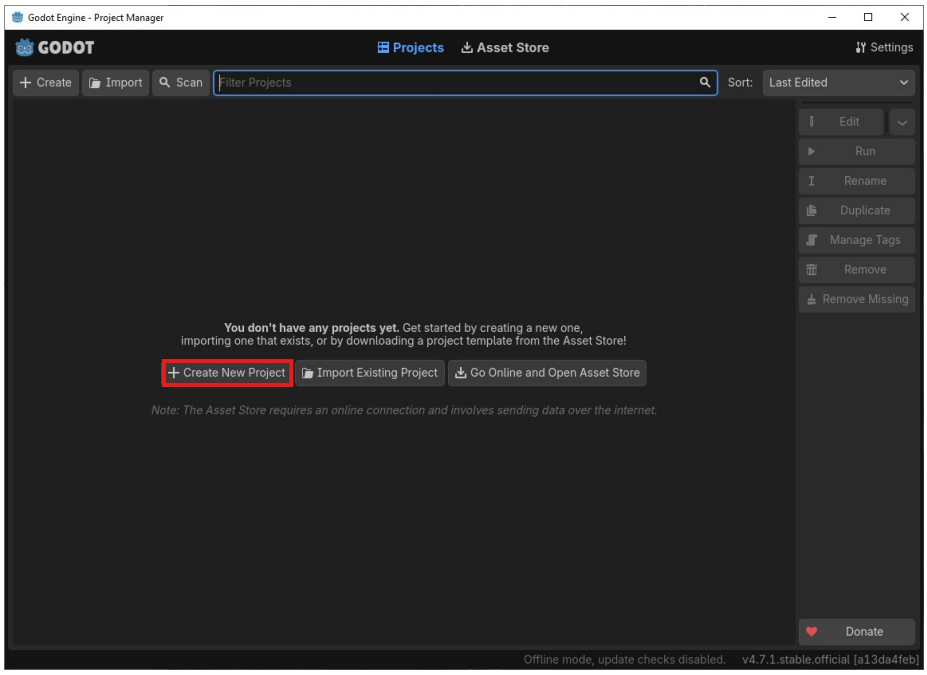
1) The standard executable (not the _console version) - optionally drag it to the Desktop for a shortcut.



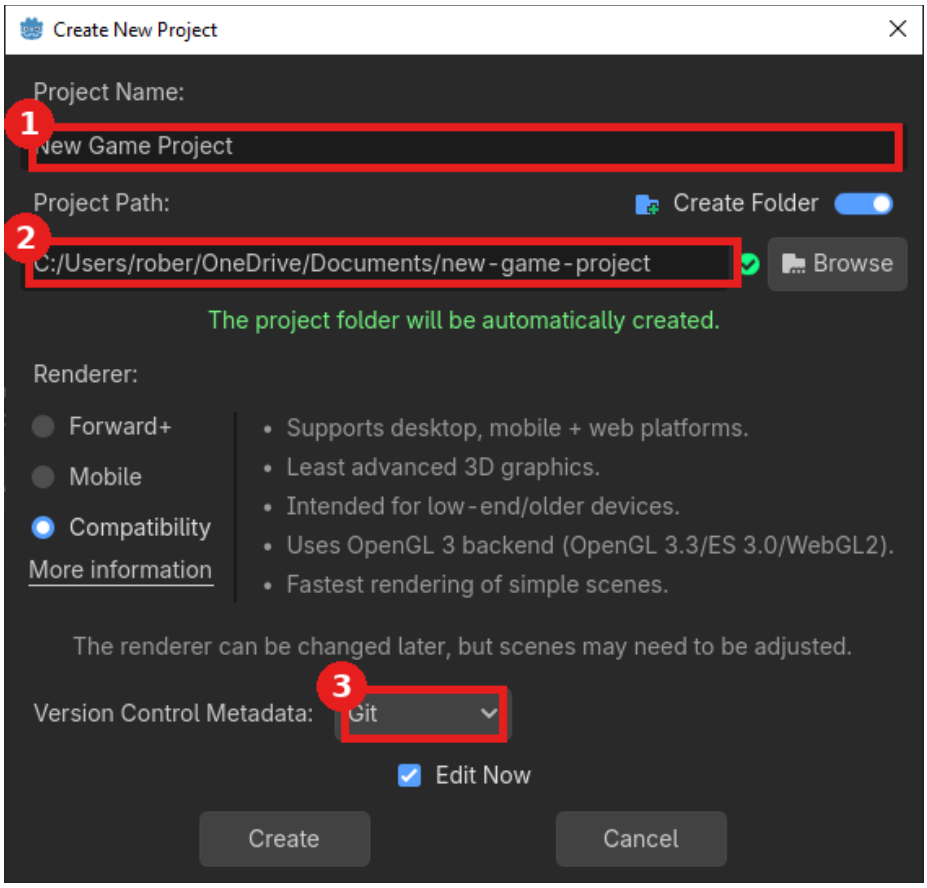
Windows SmartScreen may appear on first launch - click More info.



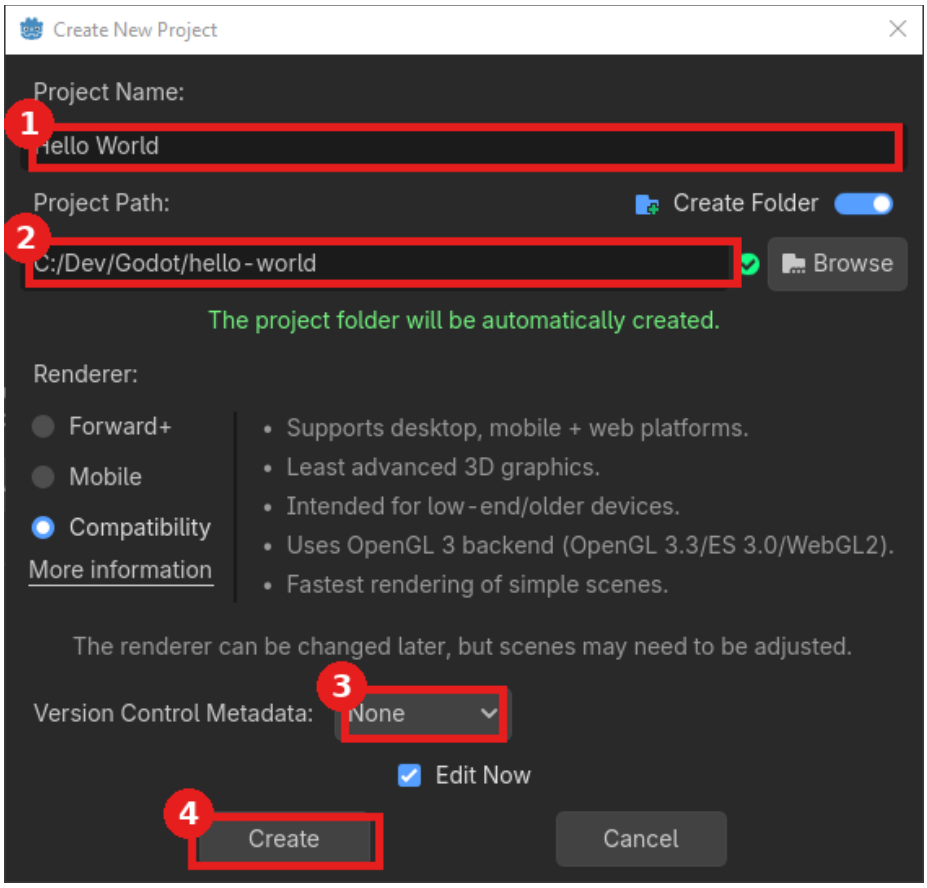
Click Run anyway to continue (expected for unsigned indie software).



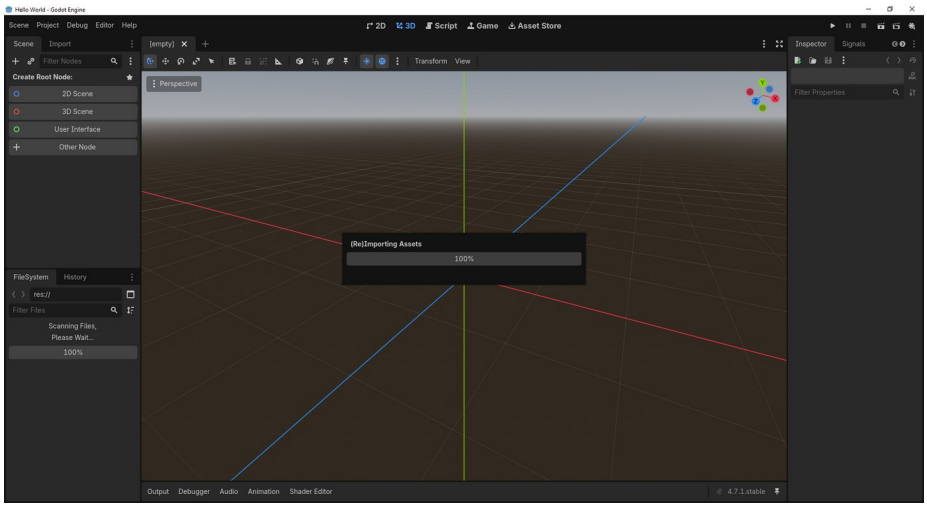
The Project Manager - click Create New Project.



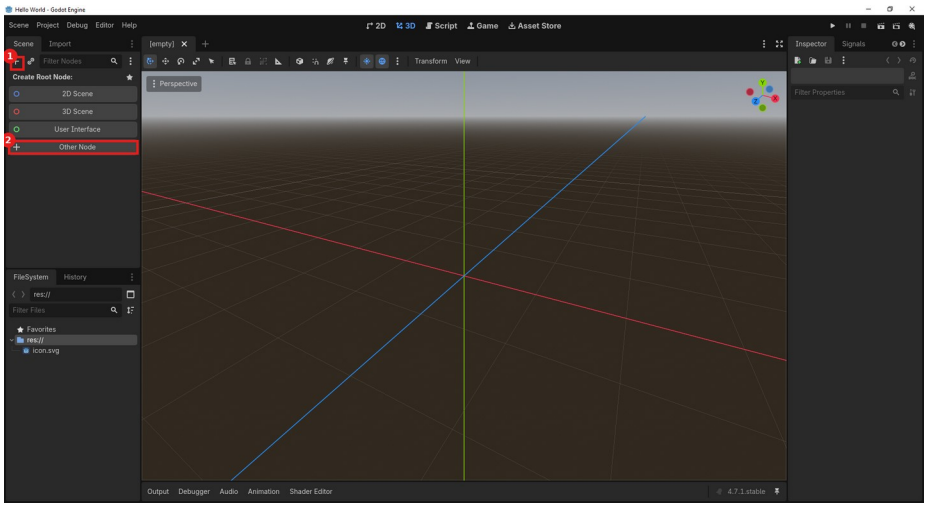
The dialog's defaults: 1) Project Name, 2) Project Path, 3) Version Control Metadata.



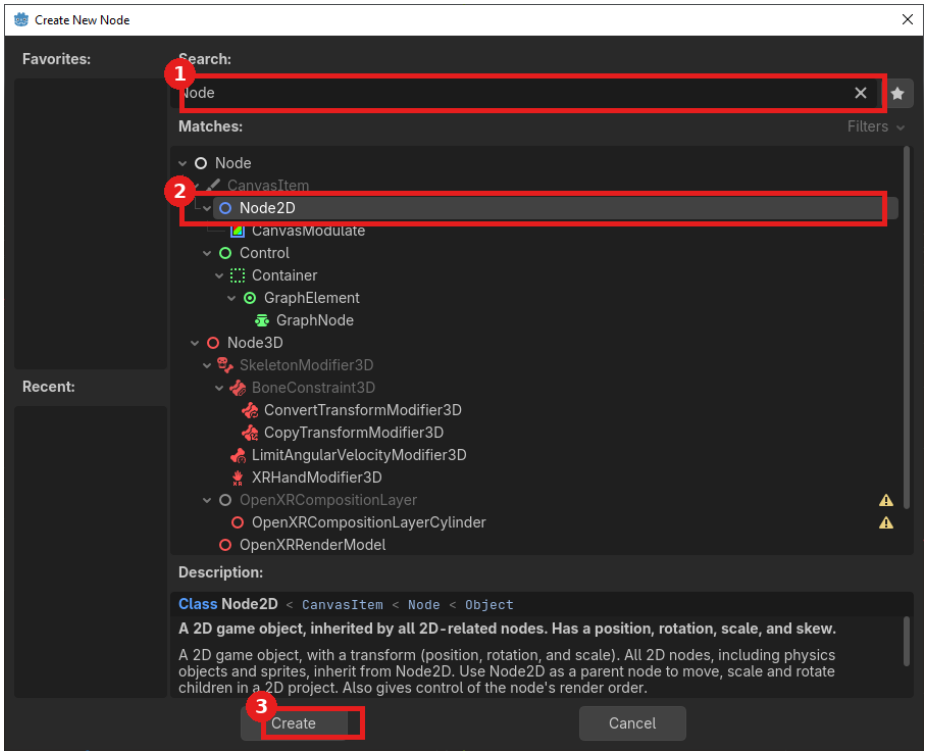
Set for this exercise: 1) Project Name = Hello World, 2) Project Path, 3) Version Control Metadata = None, 4) click Create.



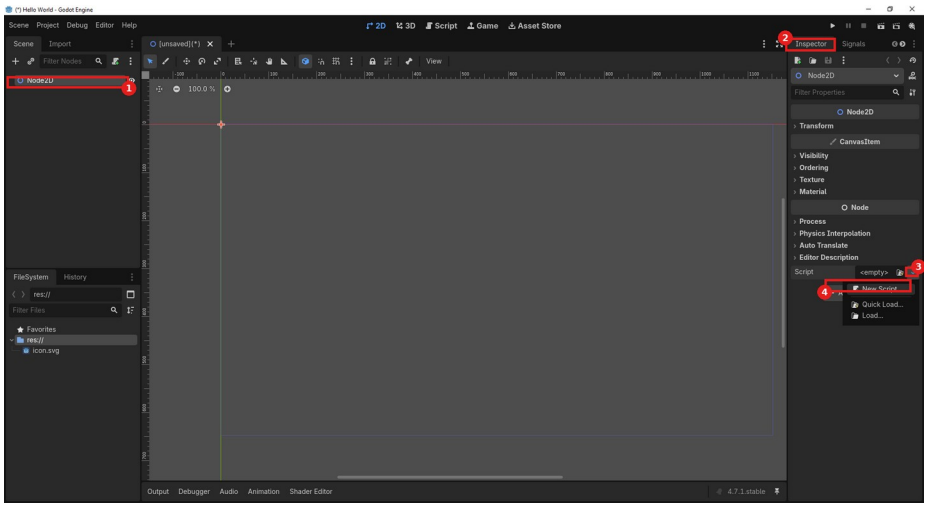
First load: Godot (Re)Imports Assets - just wait for it to finish.



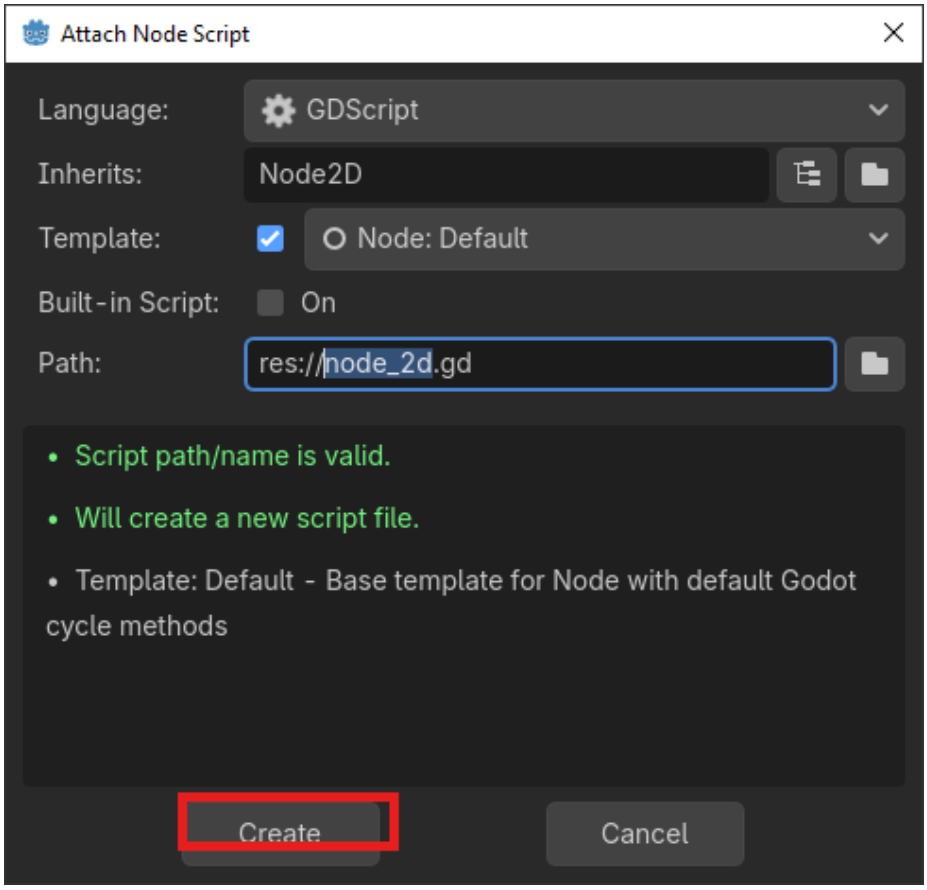
The empty scene prompts for a root node: 1) the + icon, 2) click Other Node.



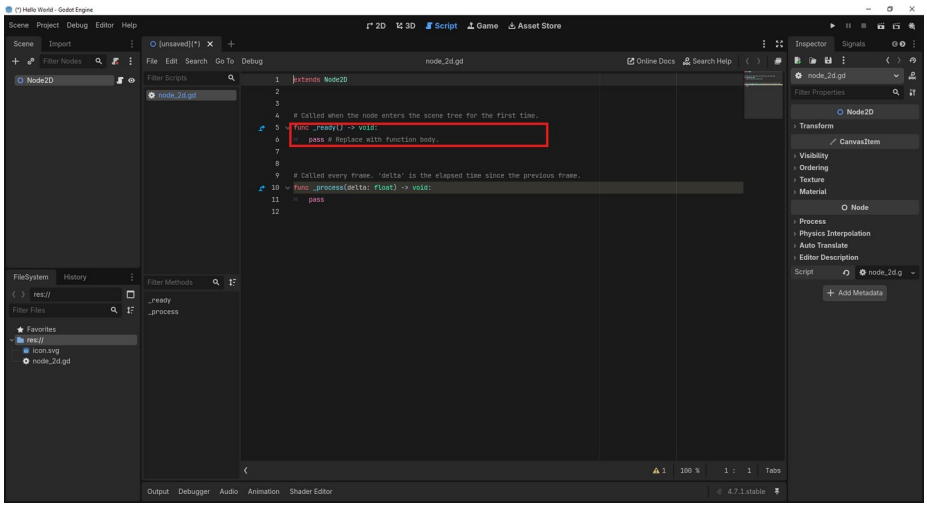
1) Search for Node2D, 2) select it, 3) click Create.



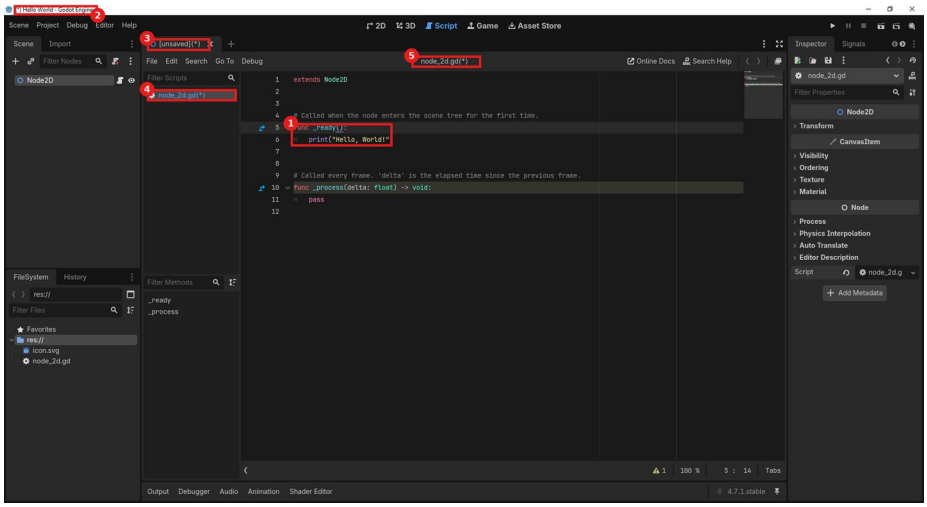
1) The Node2D root, 2) the Inspector panel, 3) the Script field's dropdown, 4) choose New Script...



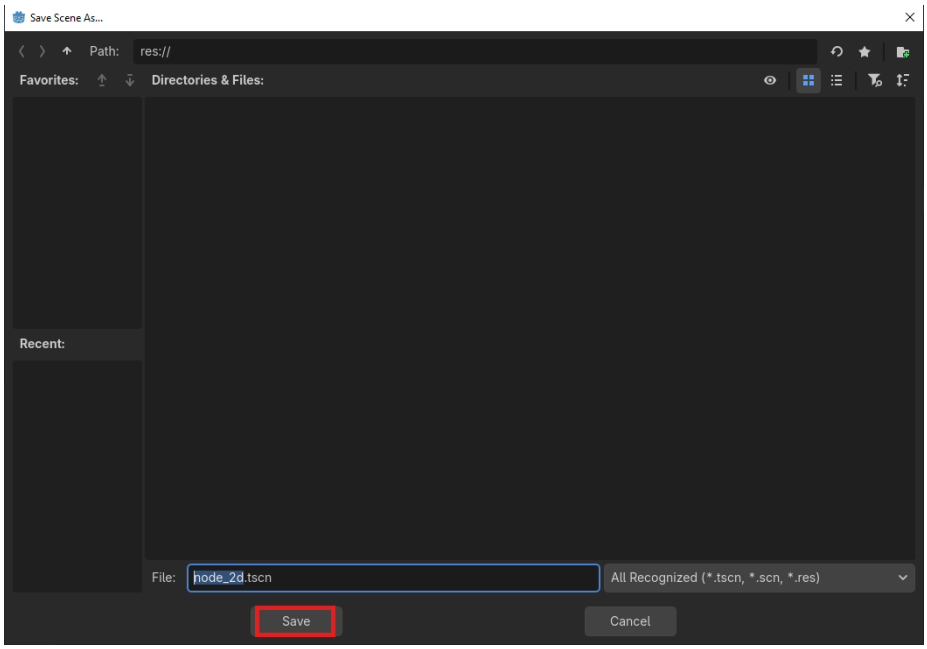
Keep all the defaults and click Create.



The default _ready() function - replace the pass placeholder with the print line.



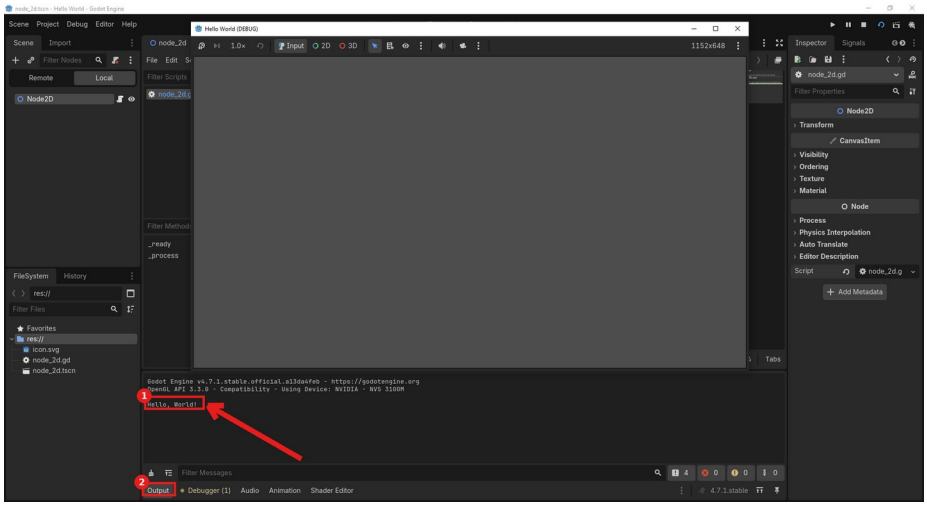
1) The edited _ready() function, 2-5) all four unsaved indicators: main title bar, editor tab, script list, and the script editor's own title bar - press Ctrl+S to save.



Running for the first time prompts a save - keep the default name and click Save.



Saved: the asterisks are gone from all three places.



Success - 1) "Hello, World!" appears in the Output panel, 2) the Output tab. You may need to move or close the small blank game window to see it.

3. Unity

Detailed Install

1. Go to unity.com and create a Unity account - use an existing Google or Apple account, or sign up with any email address (click Sign in instead if you already have one).
2. Sign in to your new (or existing) Unity account.
3. On the download page, for Windows the x64 download is correct unless you know you need Arm64.
4. The browser downloads the Unity Hub installer.
5. Click Open file in the downloads flyout to launch the Unity Hub Setup installer.
6. Choose an install location (the default is fine) and click Install.
7. Wait for Unity Hub itself to install.
8. When complete, leave Run Unity Hub checked and click Finish.
9. Unity Hub opens showing its Software License Terms - use the scroll bar to reach the bottom of the text; the Agree button lights up once you have.
10. Click the blue Agree button.
11. Sign in to Unity Hub - this is the same account you just created or signed into on the website.
12. This reopens your browser to confirm the sign-in; click Open (checking "Always allow" is optional - it just skips this confirmation next time).
13. Agree to the Unity Personal license terms.
14. Unity Hub may begin downloading the latest Editor version automatically - click the Downloads icon in the upper right to watch its progress.
15. If the download does not start automatically, skip ahead to the next step to start it manually from the Installs page.
16. Once downloading, all four items (Editor application, Web Build Support, Documentation, Get Started With Unity) should finish with green checkmarks - this can take a while.
17. To start the install manually (or add components later): click Installs in the left navigation, click the Manage dropdown next to your Editor version, choose Add Modules, then click Install Editor.
18. In the Add Modules dialog, make sure Microsoft Visual Studio Community is checked, and scroll down to also check Documentation near the bottom of the list - a blue checkmark means an item is not yet installed and has been preselected for you.
19. The Visual Studio Installer launches on its own to install the code editor.
20. Let the Visual Studio Community install finish - this is a large download and can take a while the first time.

Hello, World! Verification

1. In Unity Hub, click Projects in the left navigation, then click either New Project button.
2. Open the template filter dropdown and choose Core.
3. Select the Universal 2D template (shown in the panel on the right), set a Project Name (e.g. HelloWorld) and Location, leave Source Code Provider set to none, then click Create Project.
4. Accept the Unity Editor Software Terms: check the box, then click Accept and Continue.

5. Unity's own splash screen appears while the editor loads - this can take a while on an older machine; watch the green progress bar at the bottom.
6. If a Windows Defender Firewall prompt appears asking for network access, click Allow access - it can appear behind other windows, so watch for a glowing taskbar icon if you don't see it right away.
7. Unity finishes importing its starter packages (shader graphs, default assets, etc.) - just wait for it to finish.
8. In the Hierarchy panel, right-click empty space and choose Create Empty.
9. With the new GameObject selected, in the Inspector panel click Add Component.
10. Type "script" into the filter field, click New Script in the results, then click the arrow to continue.
11. Name the script HelloWorld and click Create and Add.
12. In the Save As dialog, confirm the file will save into your project's Assets/Scripts folder - creating a Scripts folder is optional but recommended - and click Save.
13. Unity compiles the new script in the background; once done, confirm the Scripts folder and the HelloWorld script both appear in the Assets panel, and that the Script field in the Inspector shows it attached.
14. Single-click the script in the Assets panel to preview its contents in the Inspector, or double-click it to open it for editing in Visual Studio.
15. If Visual Studio prompts you to sign in, it's safe to click "Skip and add accounts later" for this class - a Microsoft or Xbox account works too if a student already has one.
16. If prompted about GitHub Copilot, click "Maybe later" - we won't use it for this class, though students are welcome to experiment with it on their own.
17. The very first time you open a script from Unity, Visual Studio may not load it automatically - if that happens, close Visual Studio and double-click the script in Unity's Project panel again.
18. In Visual Studio, replace the body of the Start() function with the line below.
19. Save the script (Ctrl+S) and switch back to Unity - it recompiles automatically.
20. Press the Play button at the top of the Unity editor, and confirm the updated Start() method is visible in the Inspector's script preview.
21. Click the Console tab and confirm "Hello, World!" appears in the output, then click the Stop button.
22. Stop, press Ctrl+S once more to save the script, then exit.

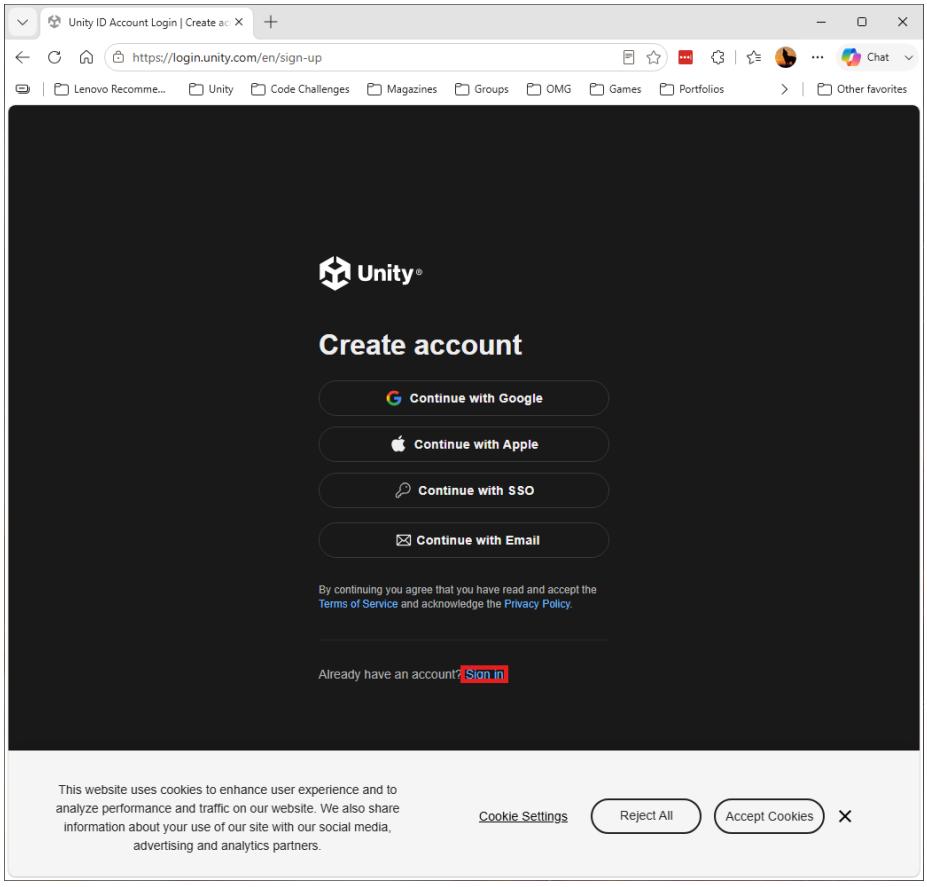
```
void Start() {
    Debug.Log("Hello, World!");
}
```

Expected result: "Hello, World!" appears as a log entry in the Console panel (Window > General > Console if it isn't visible).

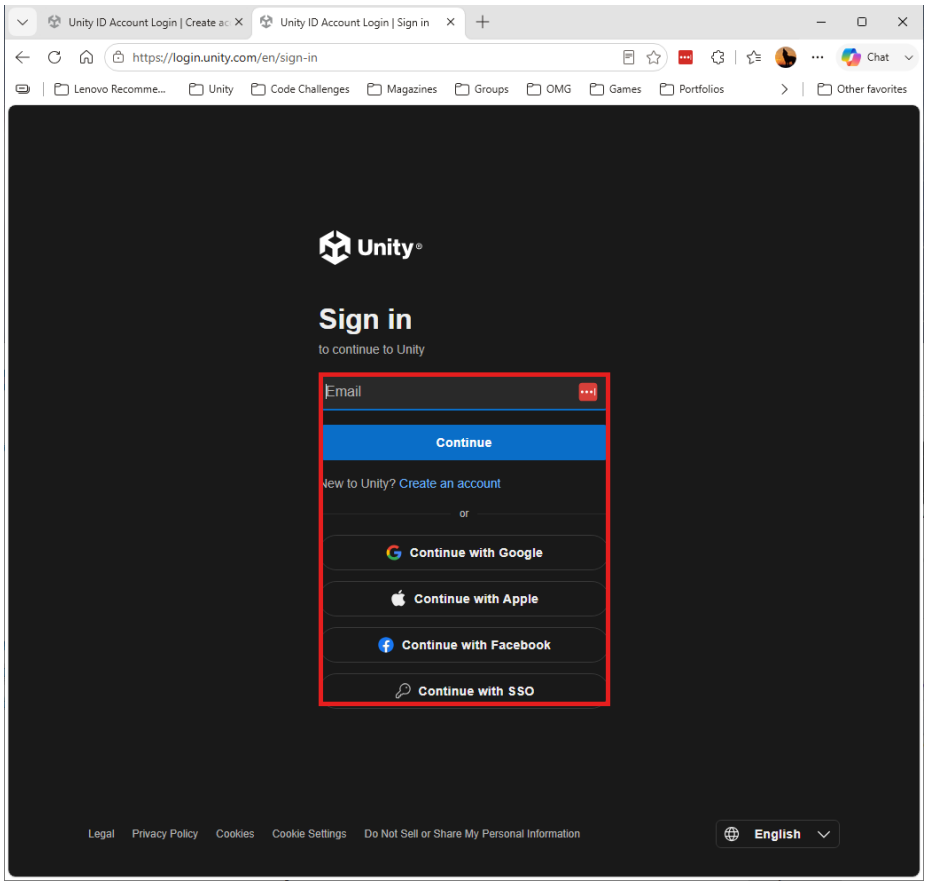
If something's wrong: *If the script won't compile, check the Console for a red error first - a missing semicolon or brace is the most common cause. If Play mode does nothing, confirm the GameObject with the script is still in the Hierarchy. On some older laptops with only partial DirectX 11 support, you may also see a GpuFence::Create() error or a shader warning about "Edge Adaptive Spatial Upsampling" in the Console - these do not stop Hello World from working, but if you plan to do any 3D work, see the companion document Older_GPU_DirectX_Troubleshooting.docx (in this same folder) for the fix.*

Visual Walkthrough (captured on Unity 6.5 (6000.5.4f1))

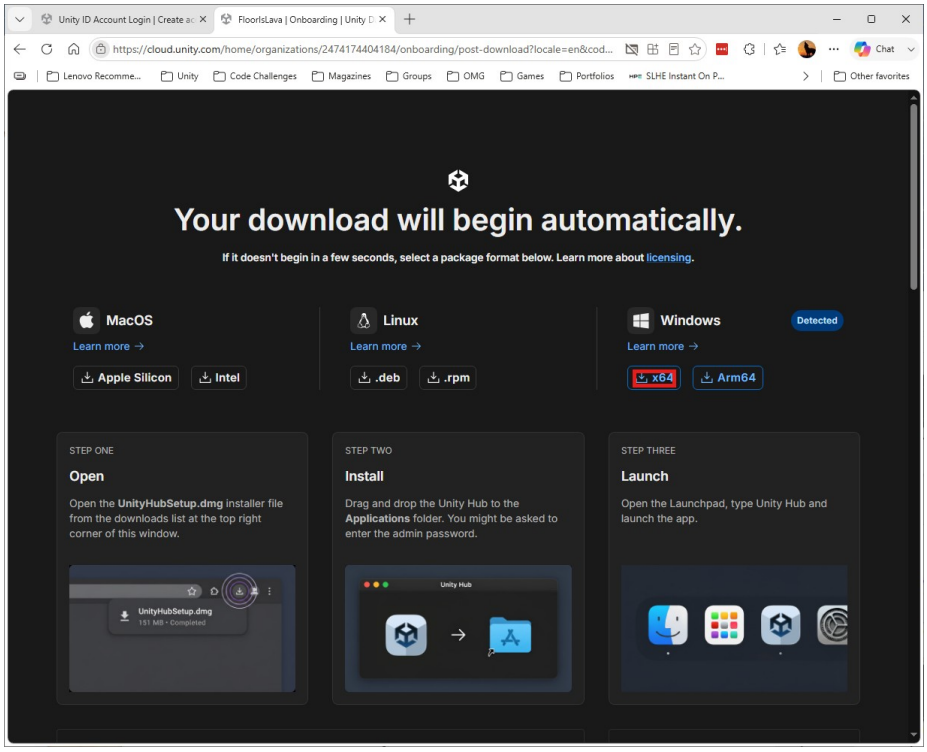
Screenshots from an actual run-through, in order. Circled/boxed elements mark exactly what to click or check at that step.



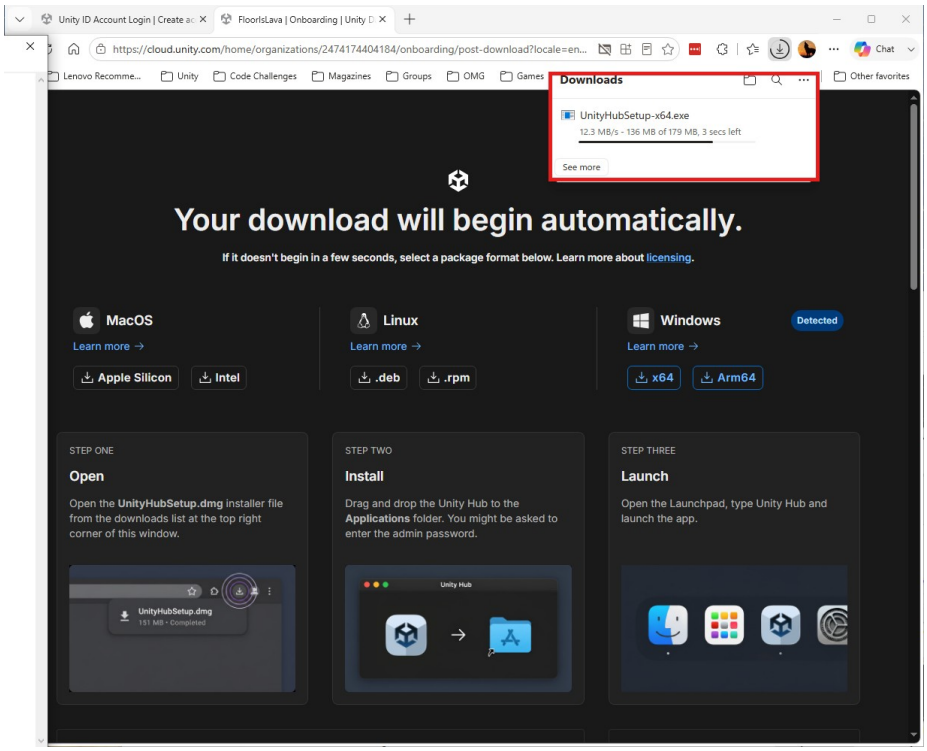
The Create account page - if you already have a Unity account, click Sign in instead.



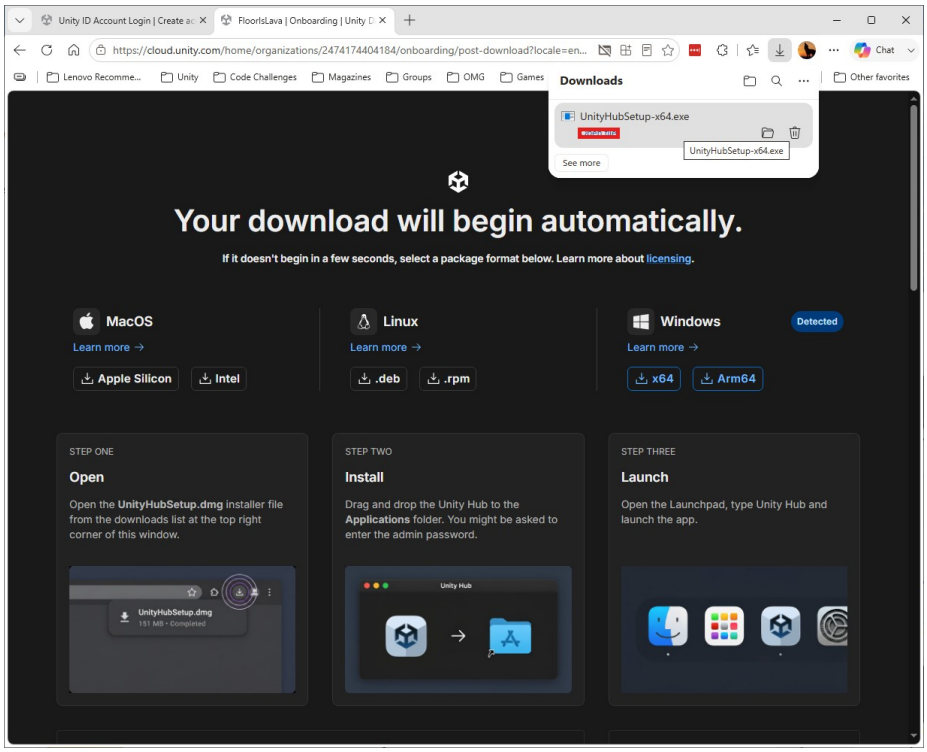
Sign in with your chosen method (Google, Apple, or email).



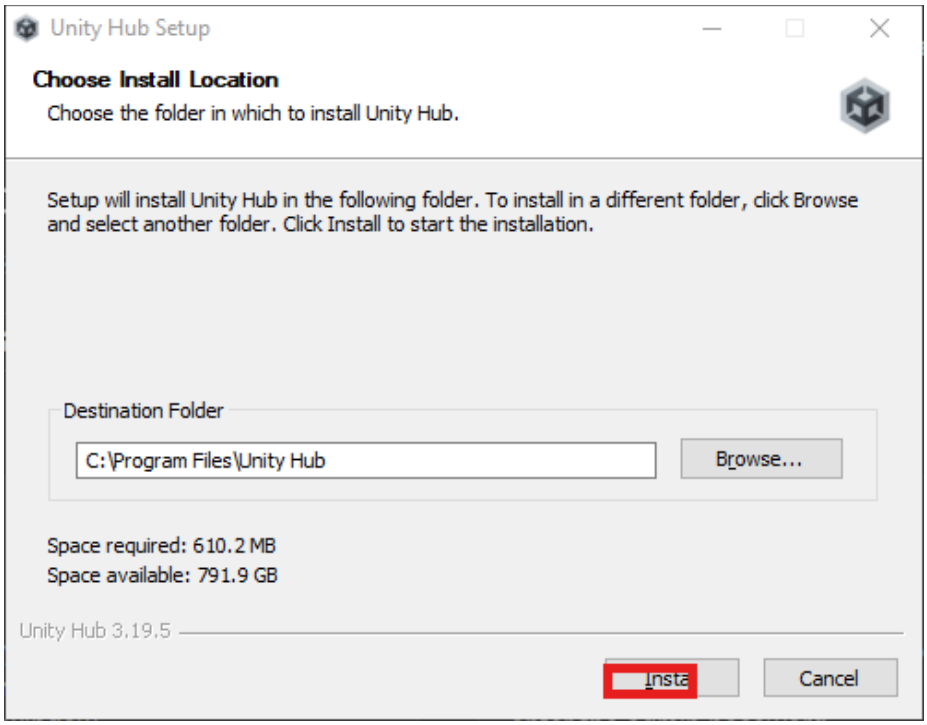
For Windows, the x64 download is correct unless you know you need Arm64.



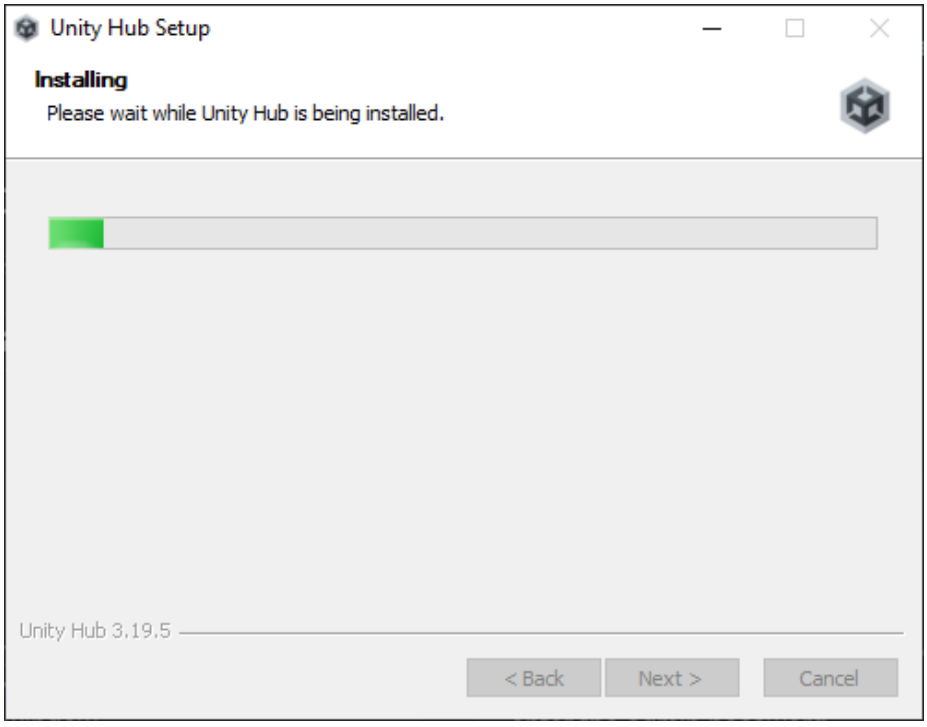
The Unity Hub installer downloading.



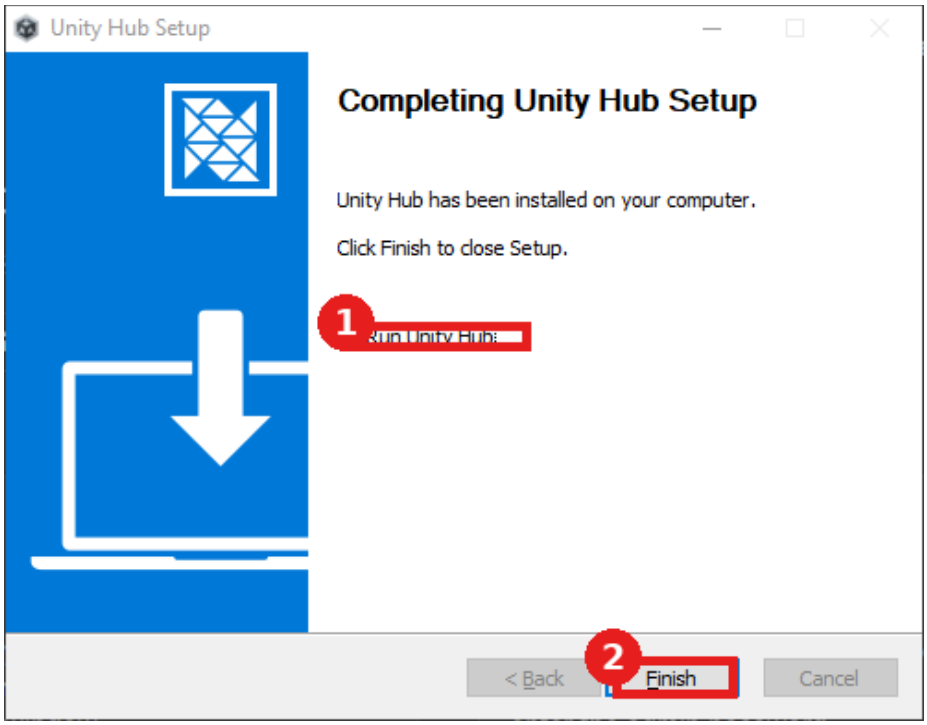
Click Open file to launch the installer.



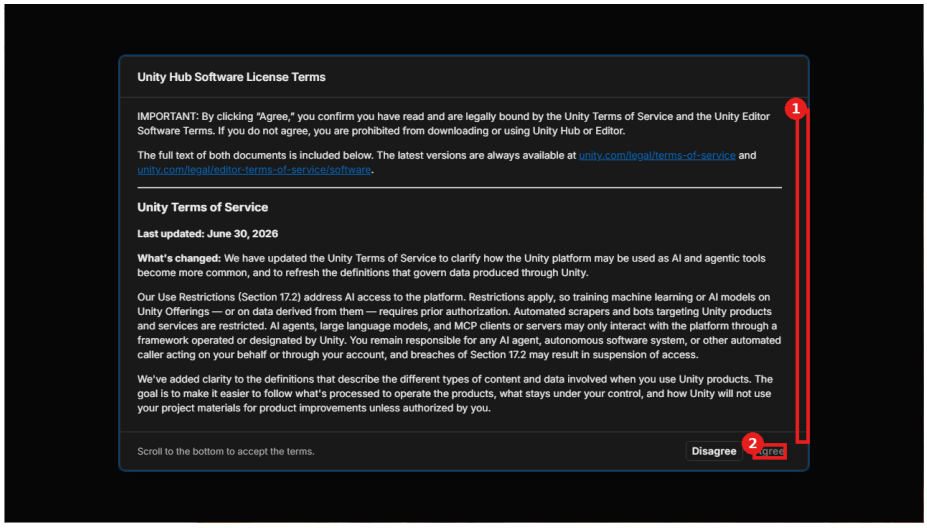
Choose an install location (the default is fine) and click Install.



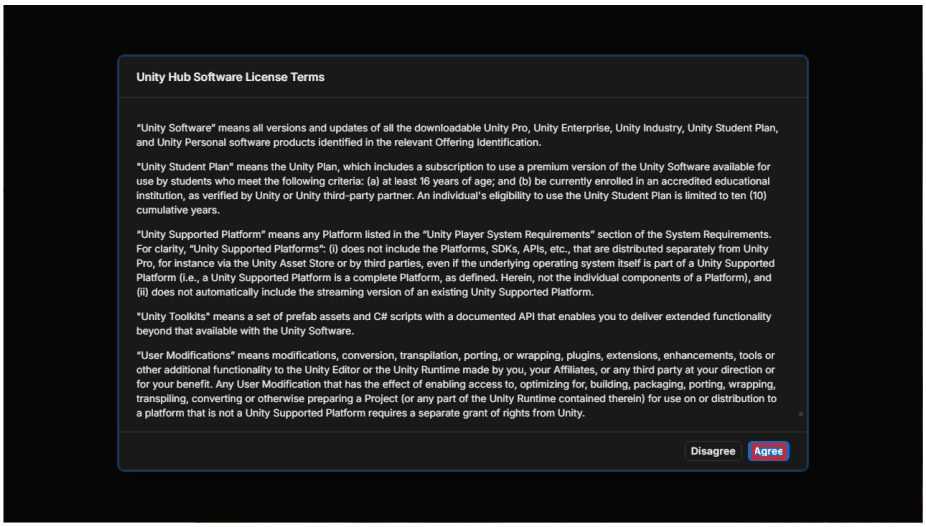
Unity Hub installing.



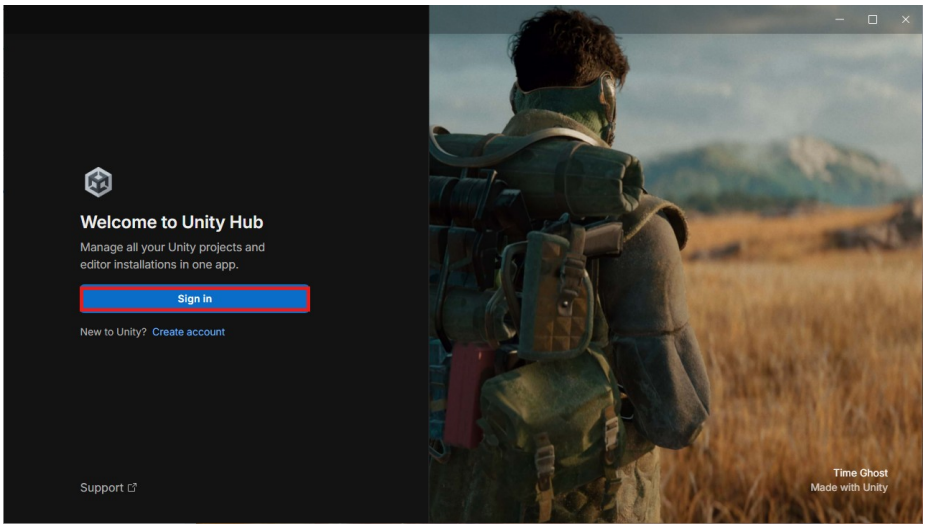
1) Leave Run Unity Hub checked, 2) click Finish.



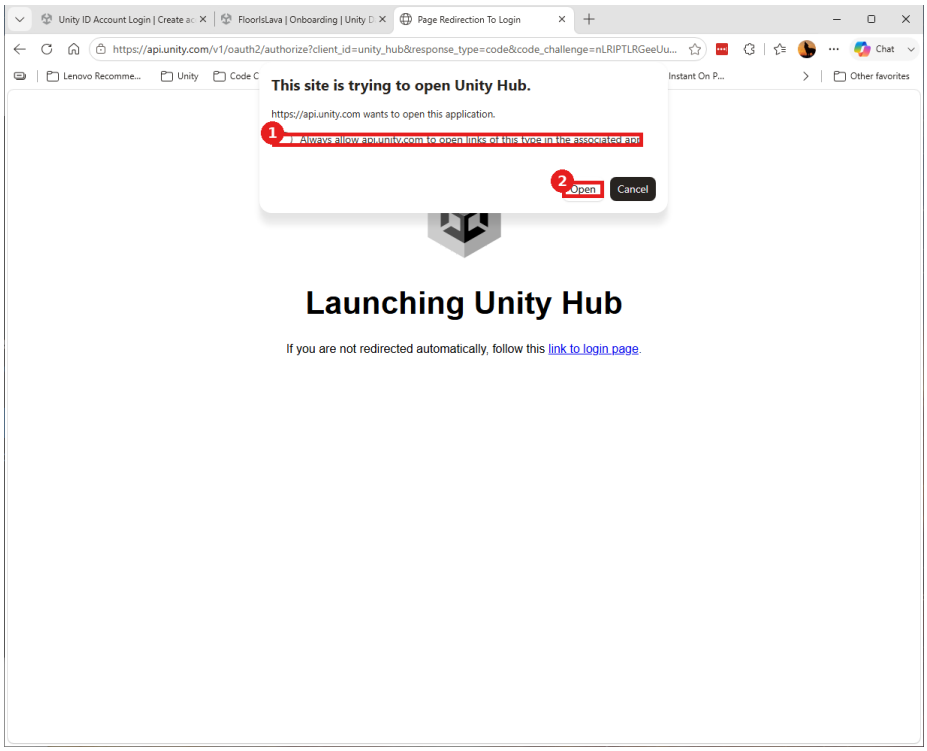
1) Scroll to the bottom of the license text, 2) until the Agree button lights up.



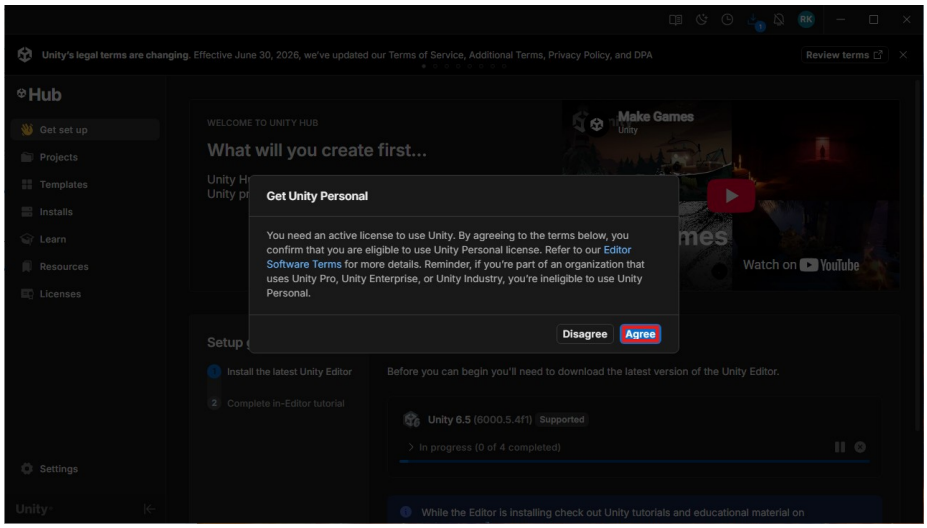
Click the blue Agree button.



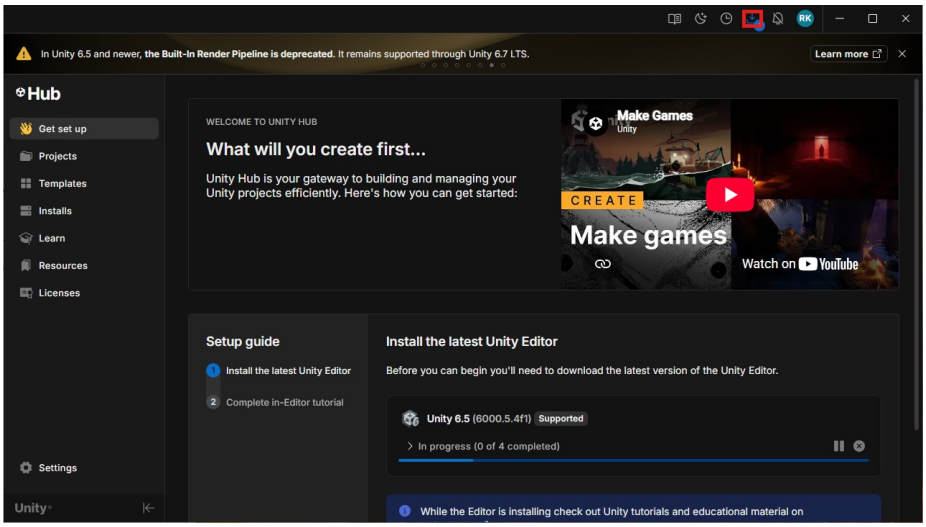
Click Sign in to Unity Hub with the same account.



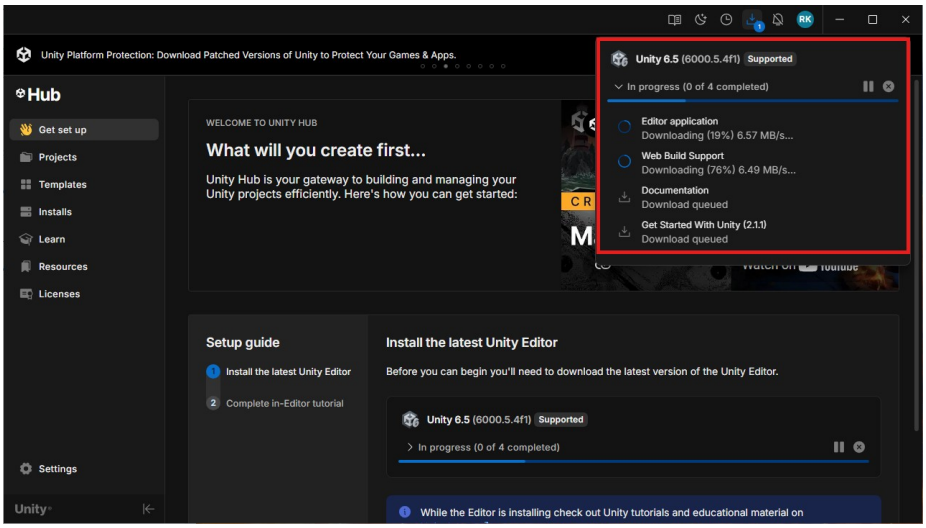
Confirming in the browser: 1) "Always allow" is optional, 2) click Open.



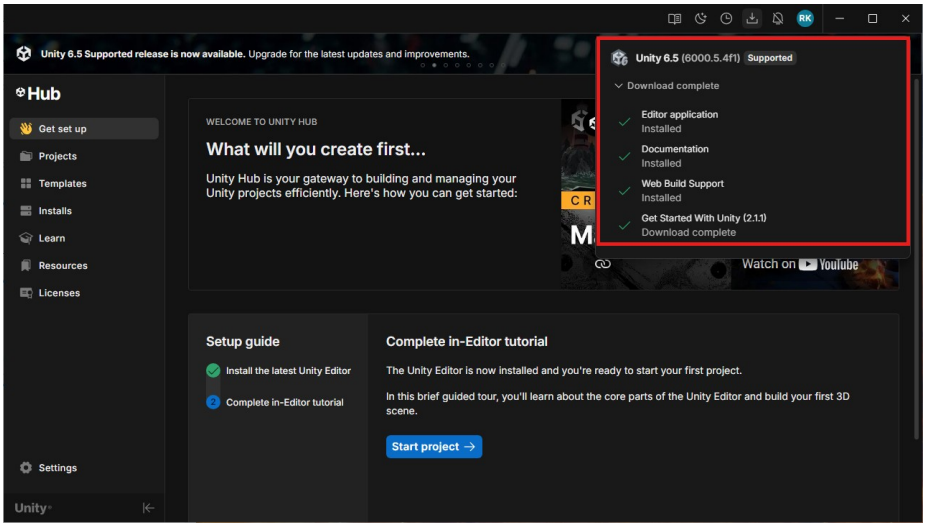
Click Agree to accept the Unity Personal license.



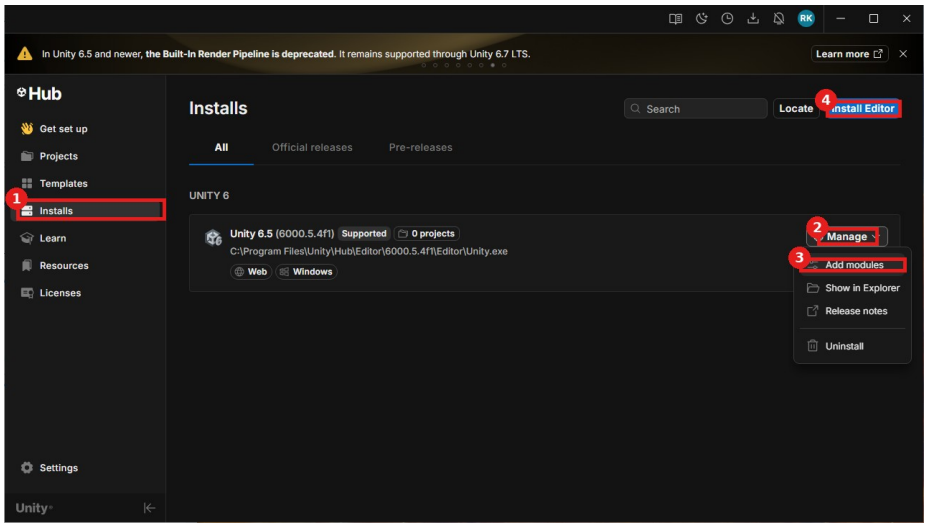
The Downloads icon, upper right - click to watch install progress.



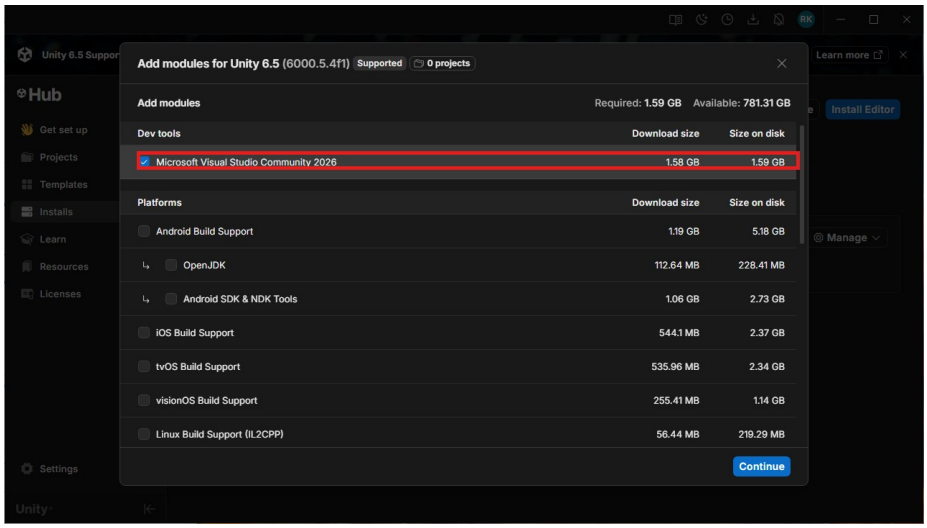
The Editor and its components downloading automatically.



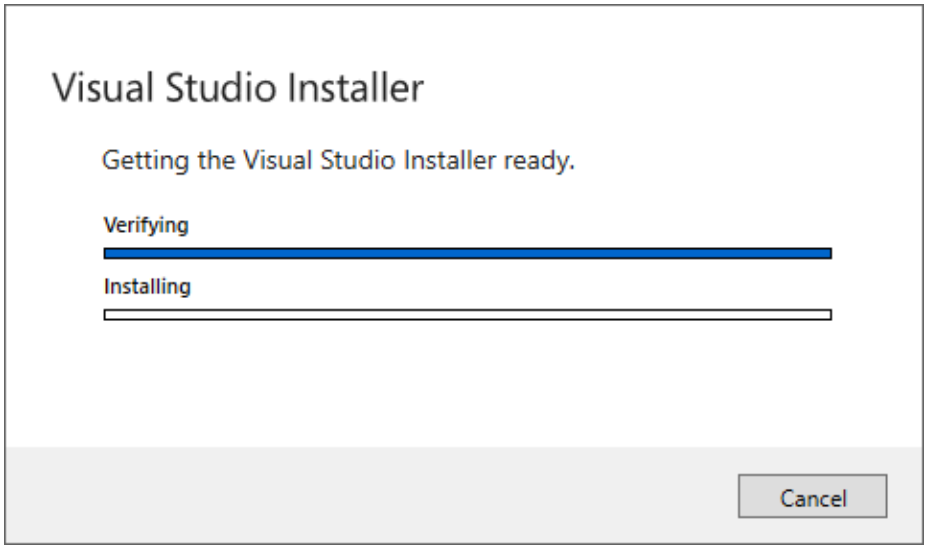
All items complete with green checkmarks.



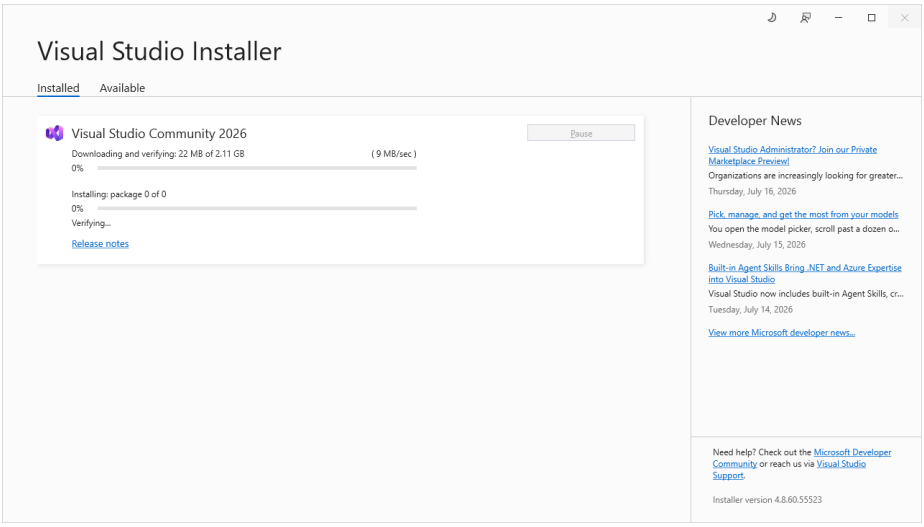
To install manually instead: 1) Installs in the left nav, 2) Manage dropdown, 3) Add Modules, 4) Install Editor.



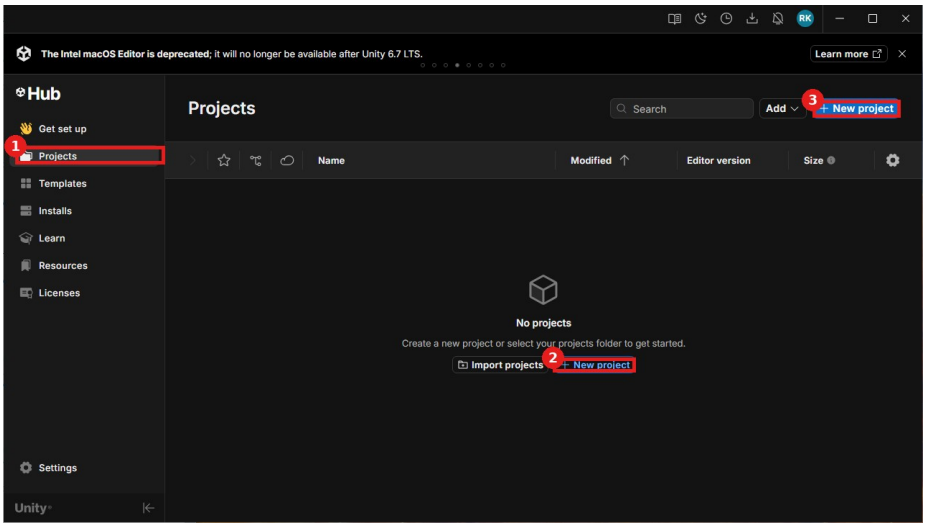
Confirm Microsoft Visual Studio Community is checked (scroll down for Documentation too) and click Continue.



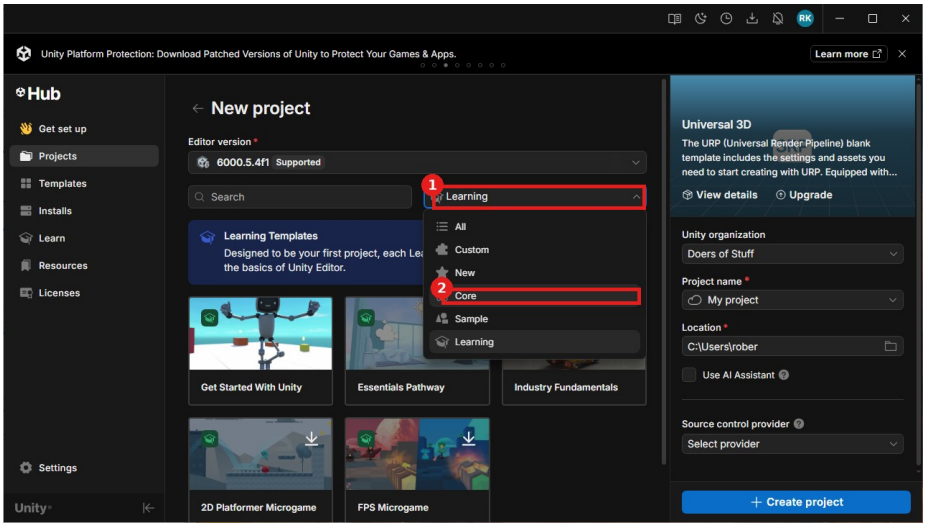
The Visual Studio Installer launches automatically.



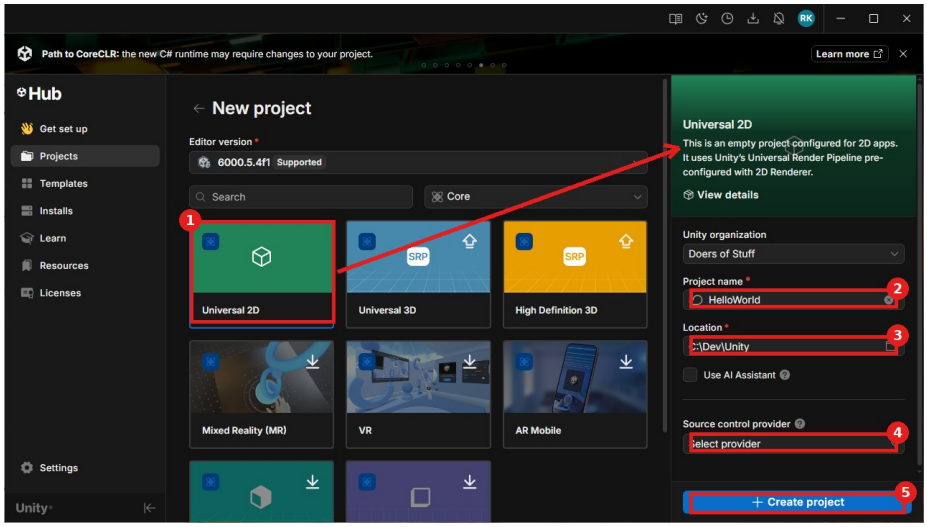
Visual Studio Community installing - this can take a while the first time.



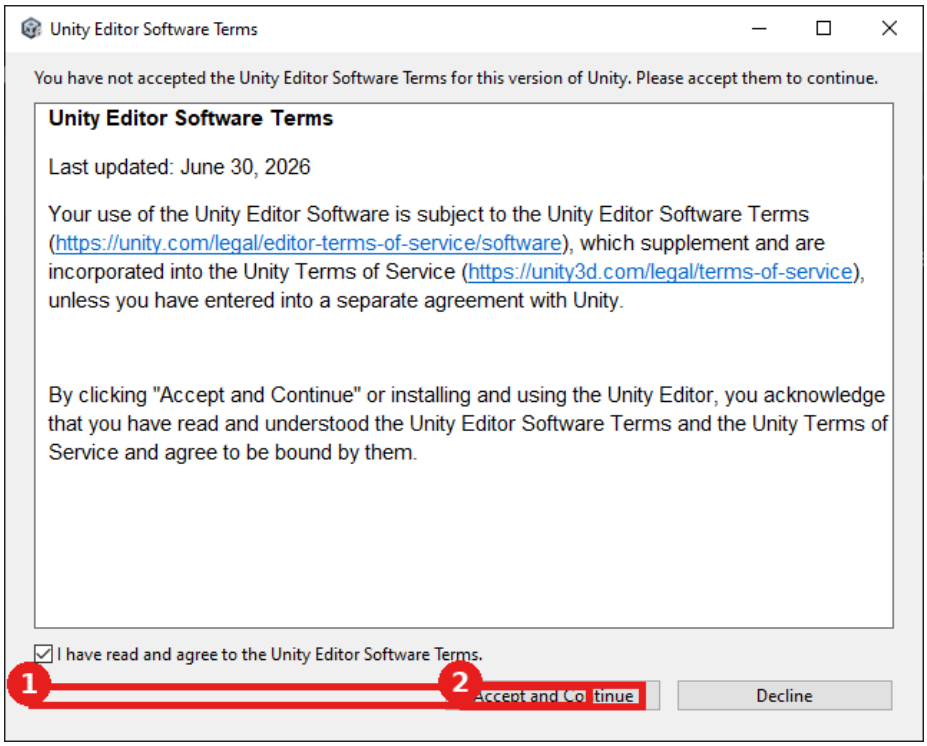
1) Projects in the left nav, 2) New Project (middle), 3) New Project (top right).



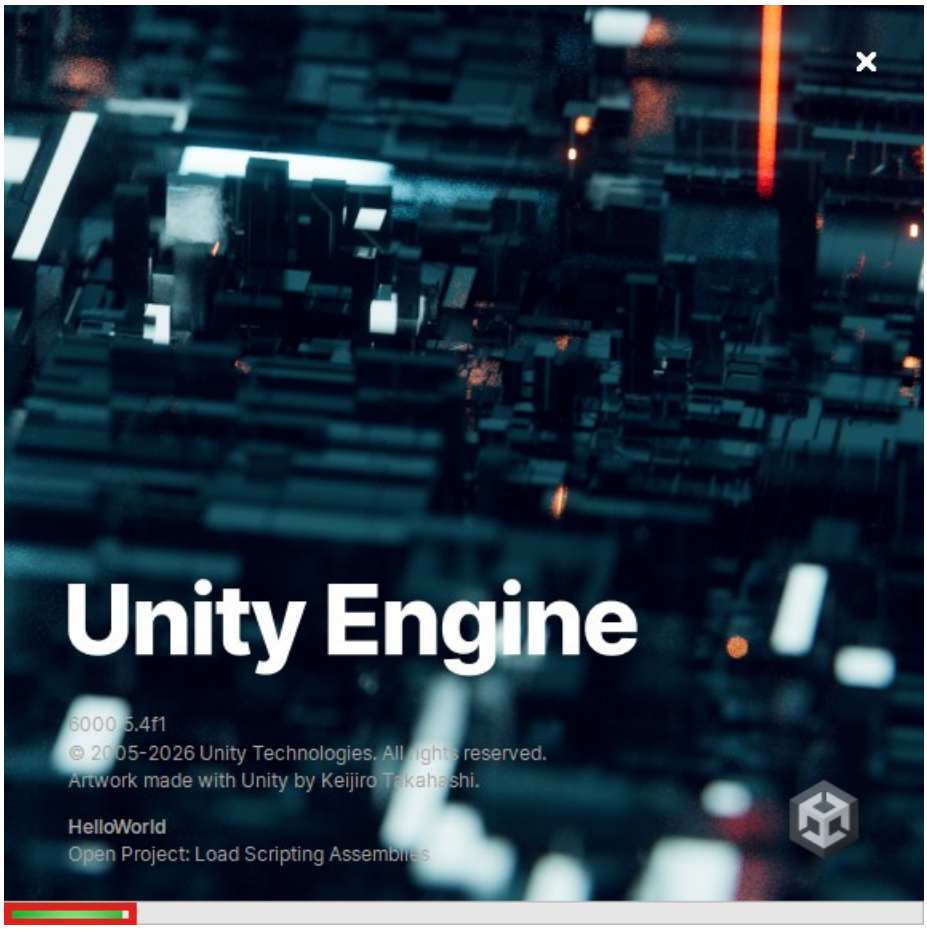
1) Open the template filter dropdown, 2) choose Core.



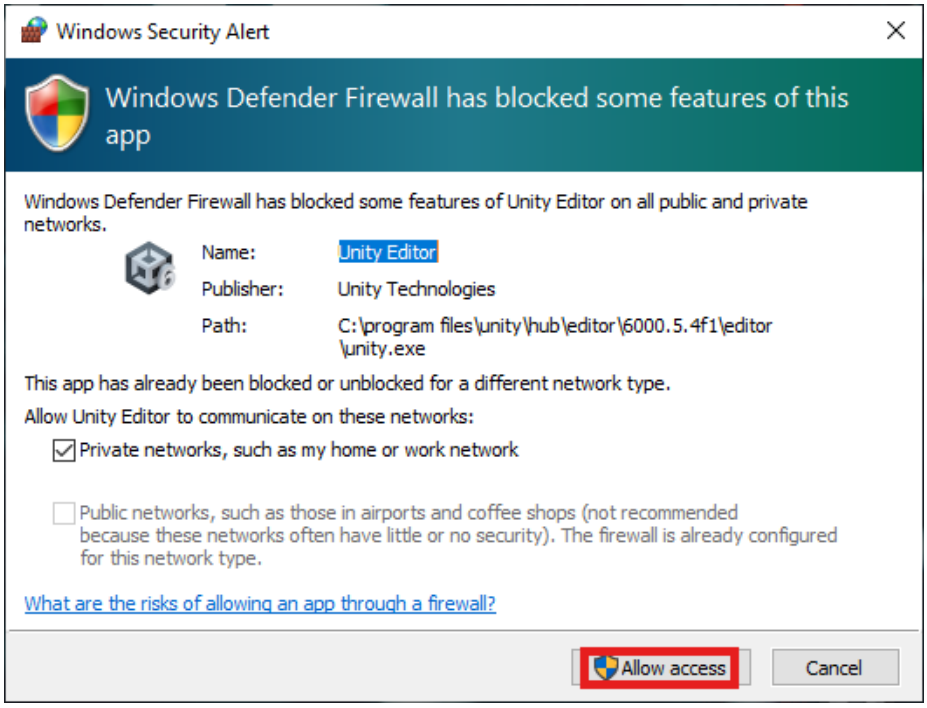
1) Select Universal 2D, 2) confirm the right panel, 3) Project Name, 4) Location, 5) Create Project (leave Source Code Provider on none).



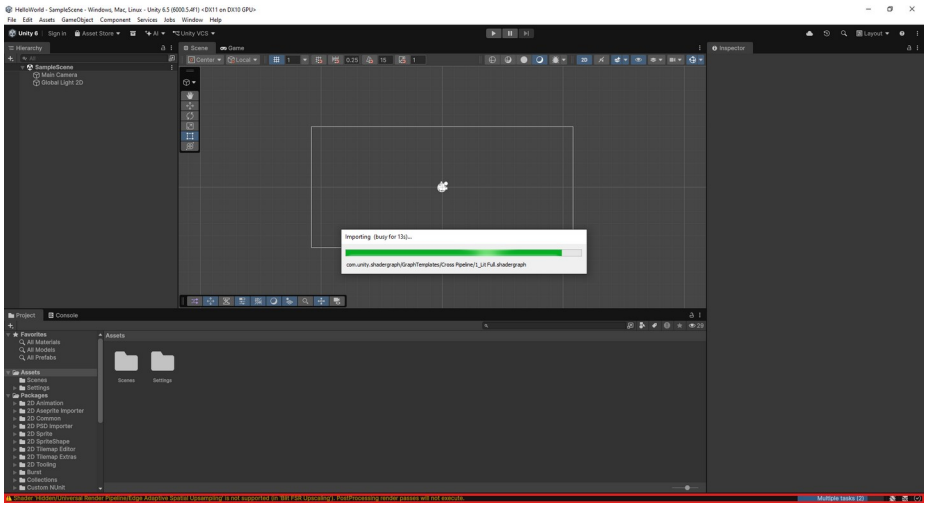
1) Check the box, 2) click Accept and Continue.



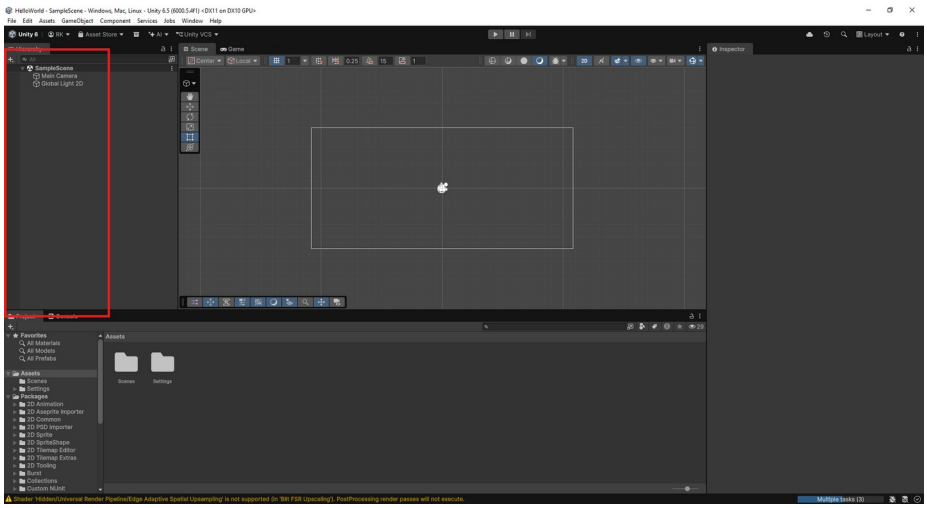
Unity's own splash screen - watch the green progress bar; this can take a while.



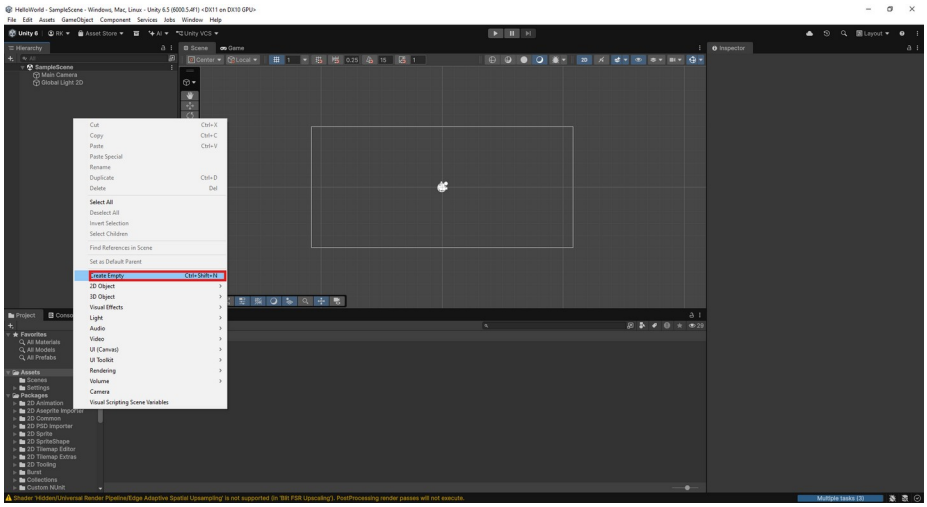
If Windows Defender Firewall prompts, click Allow access.



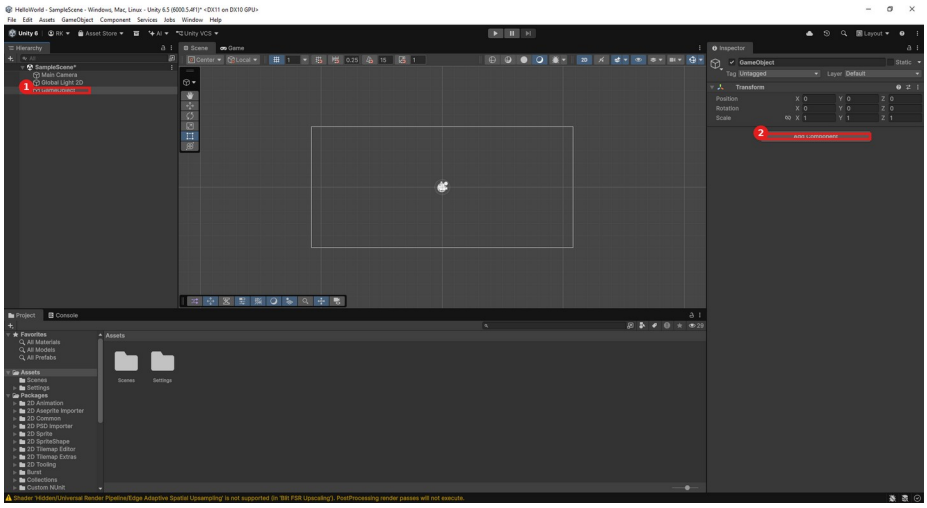
Unity imports its starter packages - note the shader warning at the bottom; this ties to the GPU troubleshooting doc, not to an error in these steps.



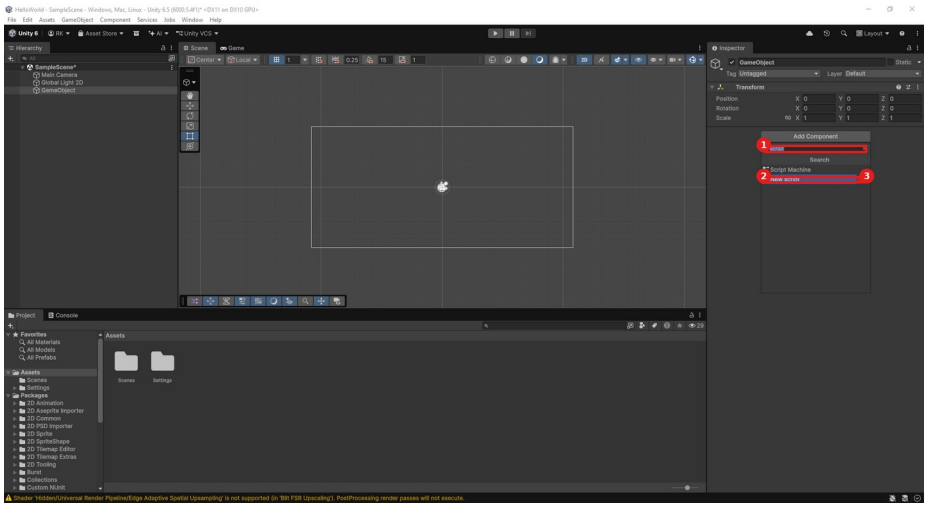
The Editor is ready - the Hierarchy panel is boxed.



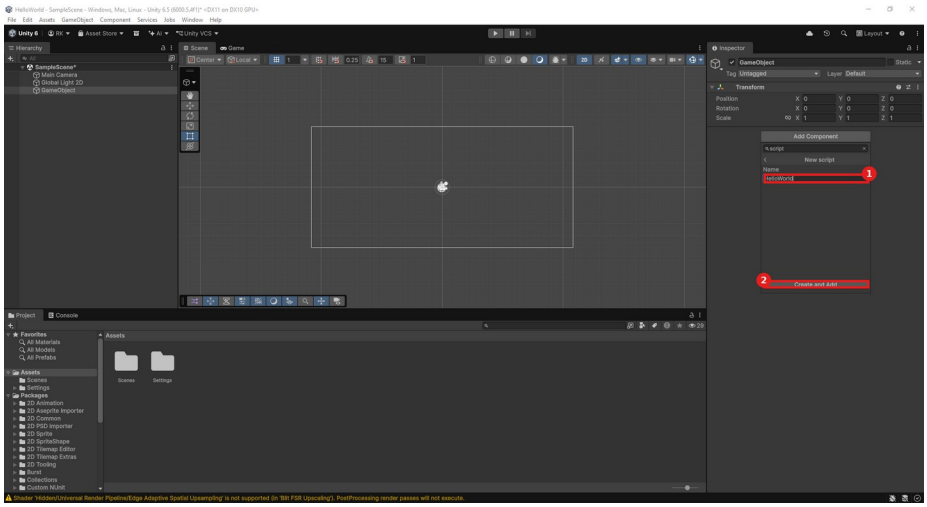
Right-click empty space in the Hierarchy and choose Create Empty.



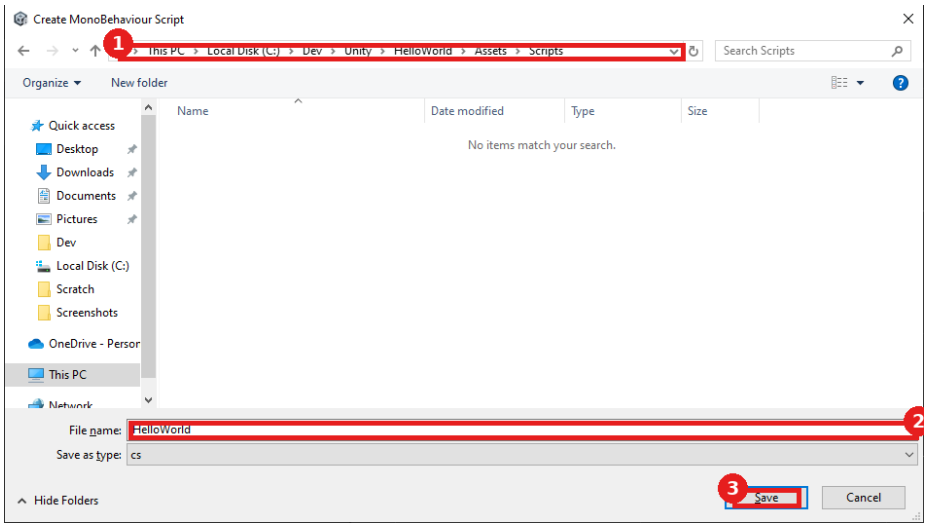
1) The new GameObject in the Hierarchy, 2) click Add Component.



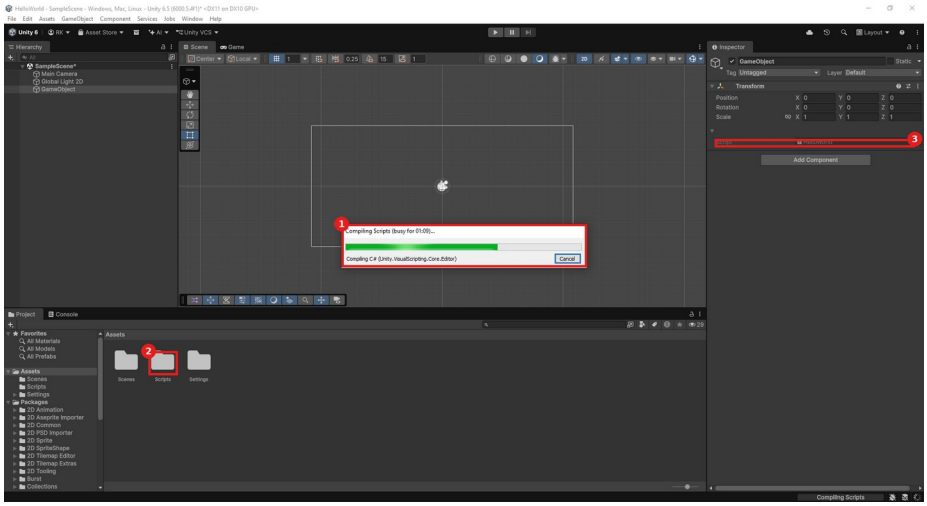
1) Type "script" into the filter, 2) choose New Script, 3) click the arrow to continue.



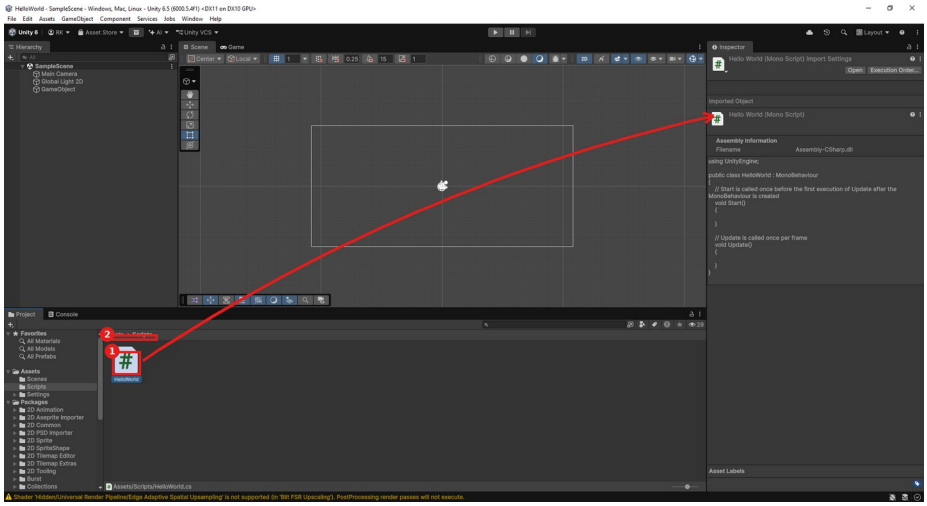
1) Name field set to HelloWorld, 2) click Create and Add.



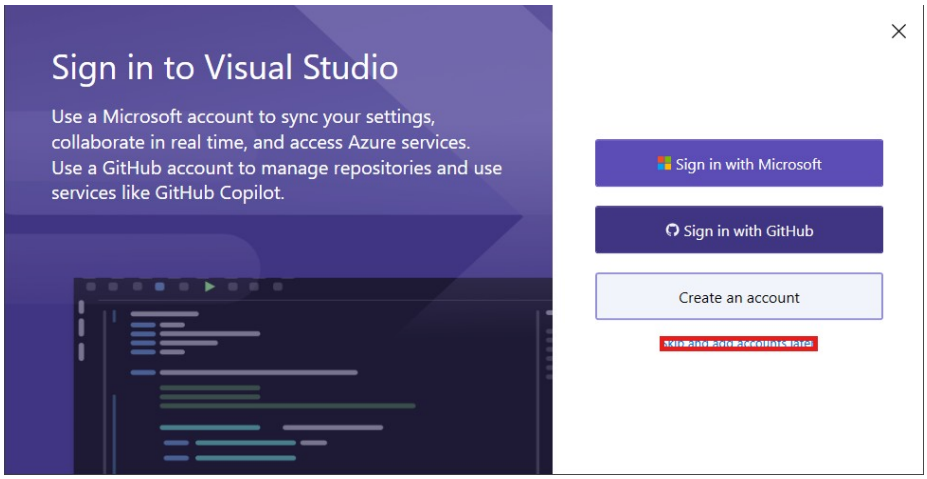
1) The breadcrumb path, 2) the file name, 3) click Save (a Scripts folder is optional but recommended).



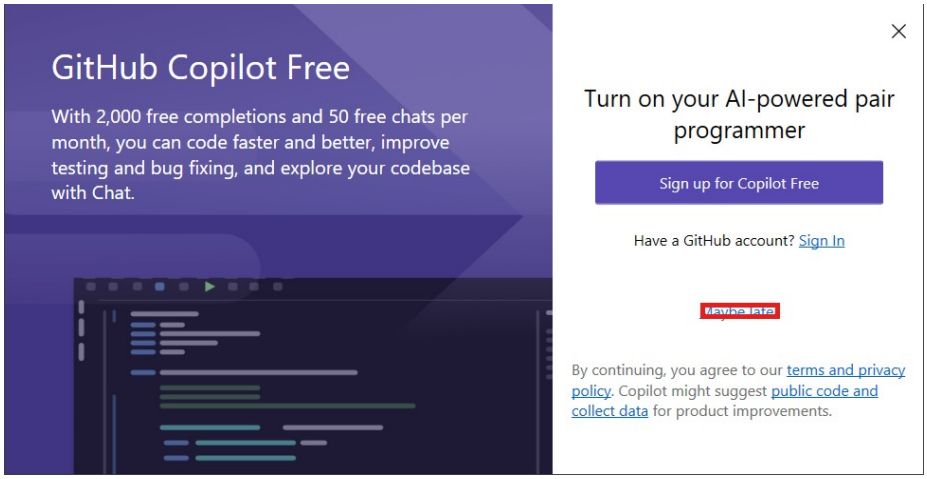
1) The compiling status, 2) the Scripts folder in Assets, 3) the Script field in the Inspector.



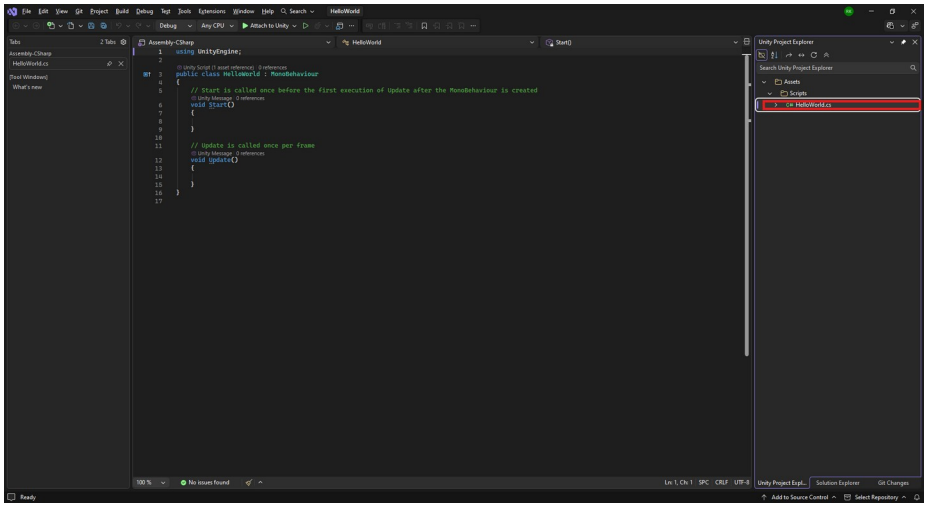
1) The HelloWorld script, 2) the breadcrumb path - single-click to preview it in the Inspector (arrow), or double-click to edit.



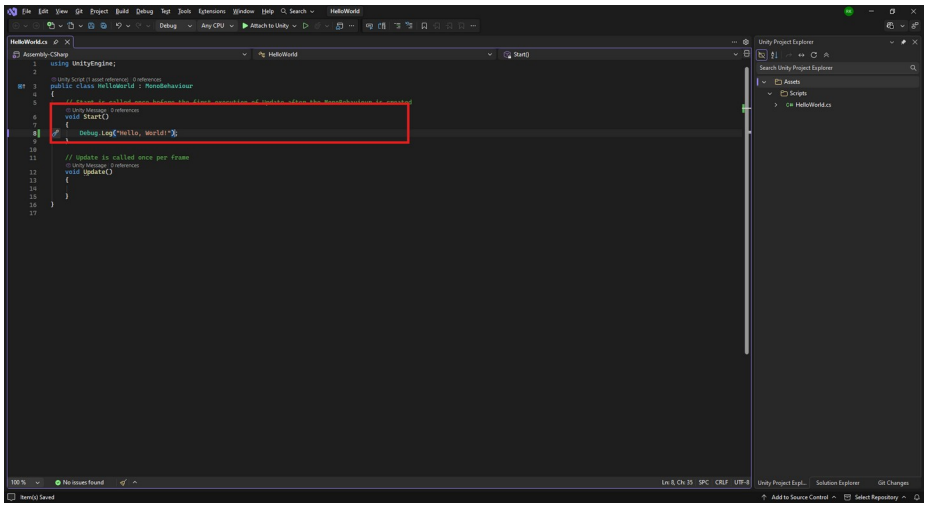
If prompted, "Skip and add accounts later" is fine for this class.



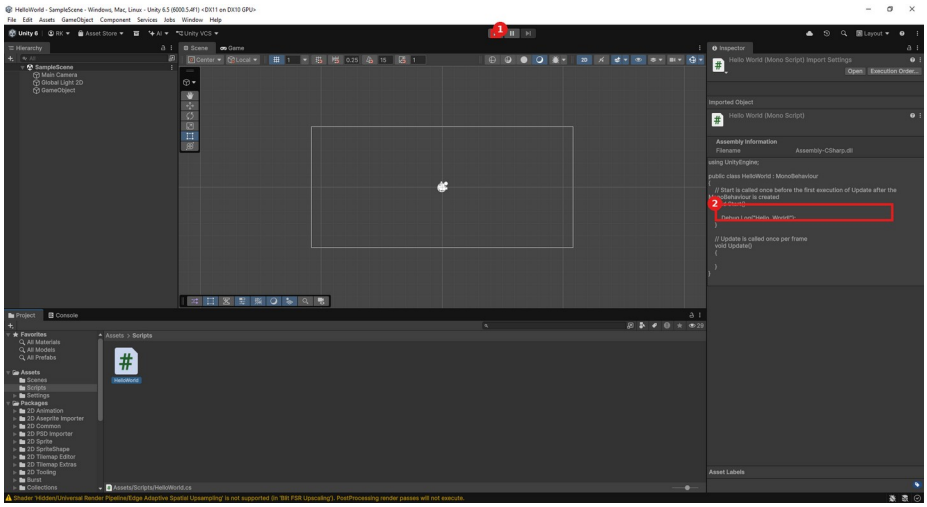
If prompted about Copilot, "Maybe later" - not needed for this class.



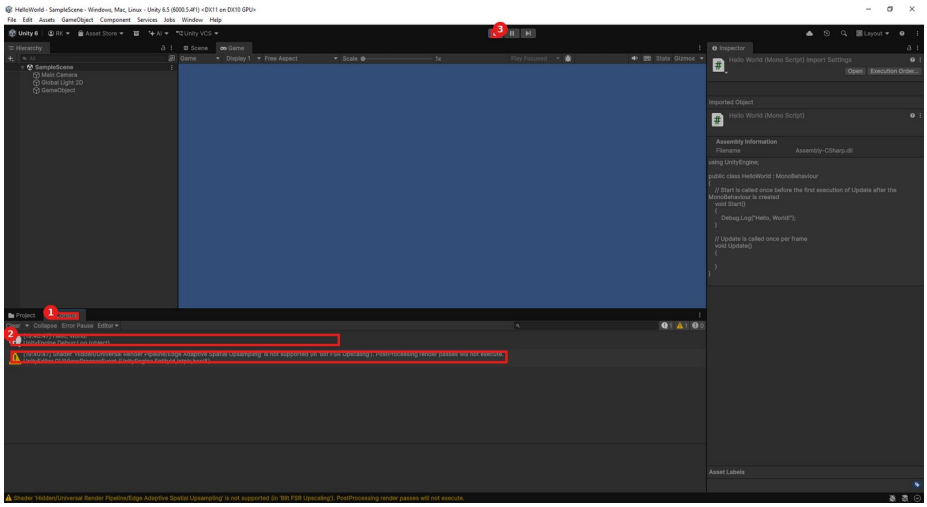
The HelloWorld script in the Visual Studio Project Explorer.



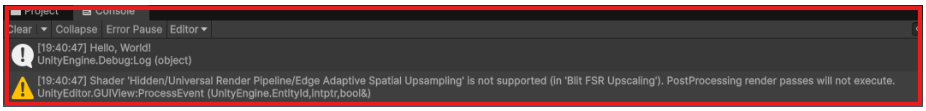
The edited Start() function, boxed.



1) The Play button, 2) the updated Start() method shown in the Inspector's script preview.



1) The Console tab, 2) "Hello, World!" in the output, 3) the Stop button. The boxed line below it is the shader warning covered in the GPU troubleshooting doc.



A closer look at the Console: the Hello World log line together with the FSR shader warning some older GPUs show - see Older_GPU_DirectX_Troubleshooting.docx.



On hardware with only partial DirectX 11 support, a GpuFence::Create() error may also appear - see Older_GPU_DirectX_Troubleshooting.docx for the fix.

4. GameMaker

Detailed Install

1. Go to gamemaker.io/en/download - accept cookies if prompted.
2. Click Download under Windows.
3. The browser downloads the GameMaker installer (a large file, ~184 MB).
4. Click Open file in the downloads flyout to launch it.
5. You're taken to a sign-in page - click Register / Sign In.
6. GameMaker accounts are handled by Opera (the company behind GameMaker) - sign in with an existing Google, Apple, or Facebook account, or any email address.
7. Once signed in, you land on your GameMaker account page - the free tier is all this class needs, no purchase required.
8. The installer unpacks itself.
9. The installer's own splash screen appears (this build is the LTS - Long Term Stable - release).
10. Choose a language and click OK.
11. Review the License Agreement and click I Agree.
12. On the Choose Components screen, Install Opera GX (a companion web browser) is optional and checked by default - leave it checked or uncheck it, either is fine for this class.
13. Choose an install location (the default is fine) and click Install.
14. Wait for GameMaker itself to install.
15. If Opera GX was selected, its own installer opens - review the terms and click Accept & Install.
16. Opera may show a usage-statistics screen - your choice, then continue.
17. Opera GX installs.
18. Opera GX opens with its own first-run welcome tour - Skip is fine, this class doesn't need it.
19. Meanwhile, GameMaker itself continues installing (you may see a Visual C++ Redistributable step run automatically).
20. The GameMaker installer confirms Installation Complete - click Next.
21. Click Finish to close the installer.

Hello, World! Verification

1. Launch GameMaker. On first start, watch the Installing indicator in the upper right while it finishes setting itself up.

2. If a Windows Defender Firewall prompt appears, click Allow access - GameMaker needs this to check for updates and license your account; you may need admin privileges to approve it.
3. On the Welcome To GameMaker screen, a Set Up Wizard walks you through creating your first project - click the Game project type icon to begin (or click New directly if the wizard isn't showing).
4. 1) Confirm Game is selected as the project type, 2) select the Blank Game template (its description confirms what it creates), 3) set Project Name to HelloWorld, 4) confirm the Location, 5) click Let's Go!
5. The new project opens directly into the Project Space, with a short built-in tour pointing out the toolbar, Asset Browser, Inspector, and Output Window.
6. In the Asset Browser panel (top right), click the + icon, then Create, then choose Object from the submenu.
7. GameMaker creates a new object and opens it for editing - 1) rename it from the default to obj_hello in the Asset Browser, 2) confirm the same name updates in the object editor's Name field, 3) click Add Event, 4) choose Draw from the event list, 5) choose the plain Draw event from the submenu (not Draw GUI or the others).
8. The first time you add a code-based event, GameMaker asks which scripting language to use for the project - 1) choose GML Code (not GML Visual), 2) click OK.
9. A code editor opens for the Draw event, already positioned to type - as you type, GameMaker's autocomplete suggests matching functions (for example, typing draw_t lists every draw_text-style function available).
10. Type the line below. Notice the Draw tab now shows an asterisk (*) - the code hasn't been saved yet; press Ctrl+S to save it.
11. Double-click Room1 in the Asset Browser to open the Room Editor.
12. Drag obj_hello from the Asset Browser into the room canvas to place an instance of it - 1) it now appears in the room's own instance list just below the search bar, 2) notice the asterisk (*) that has appeared in the window's title bar, meaning the whole project has unsaved changes - press Ctrl+S to save everything, 3) click the Run (Play) button in the toolbar to test.

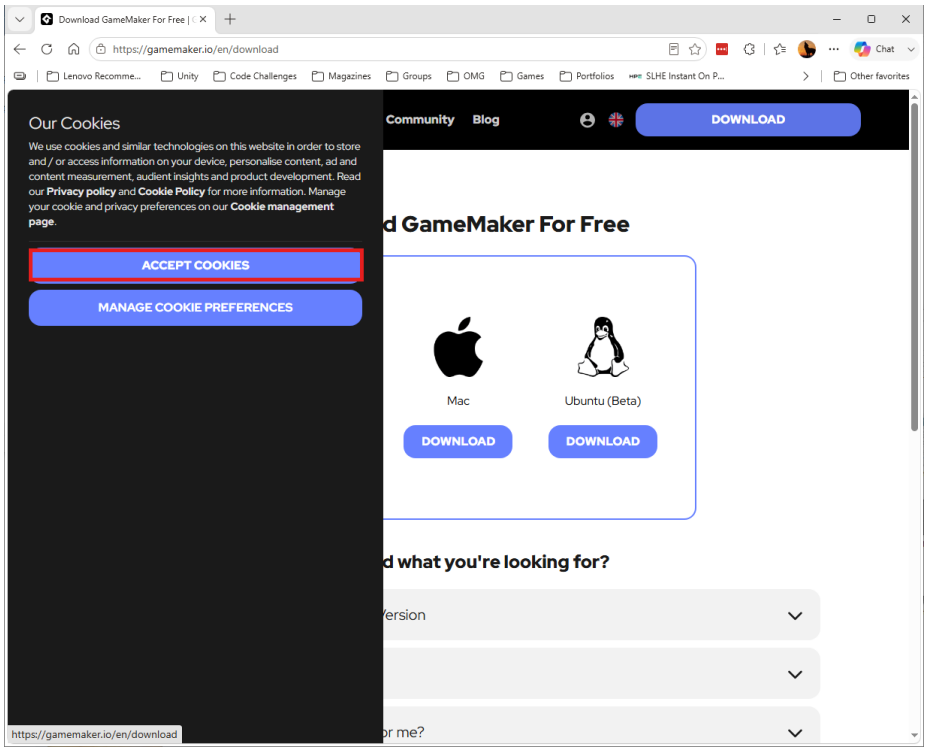
```
draw_text(100, 100, "Hello, World!");
```

Expected result: A game window titled HelloWorld opens showing the text "Hello, World!" near the top-left in the default font.

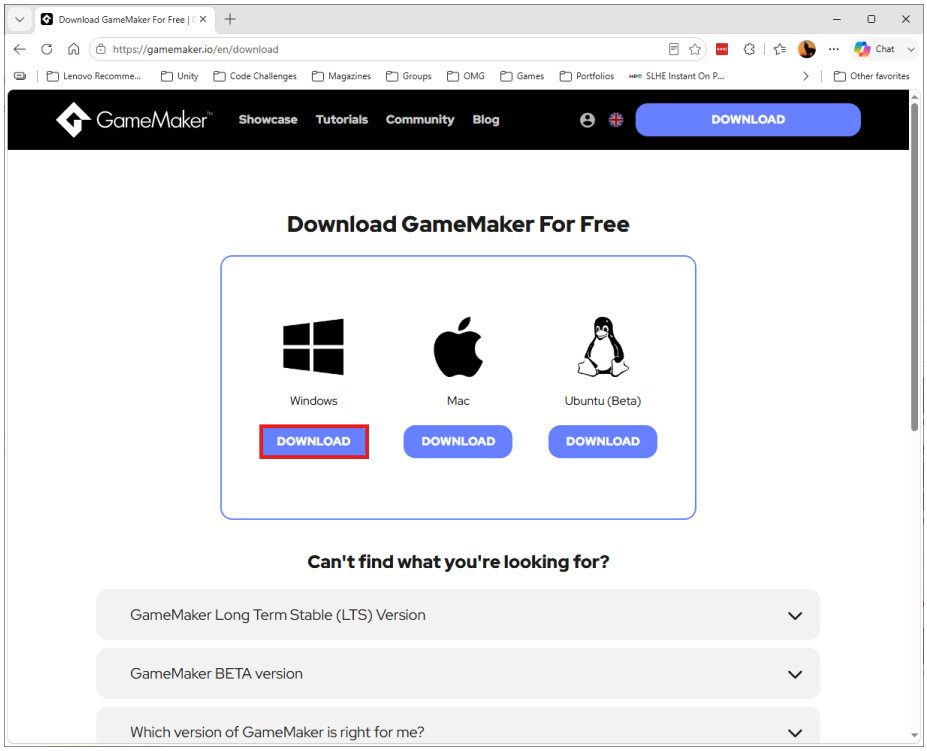
If something's wrong: *If nothing appears, confirm the object instance was actually dropped into the room (check the Room Editor's instance list, not just the Asset Browser) and that the Draw event code has no red underlines or errors in the Feather Messages panel at the bottom. If GameMaker prompts to save before running, that's expected the first time - accept it and continue.*

Visual Walkthrough (captured on GameMaker LTS2026)

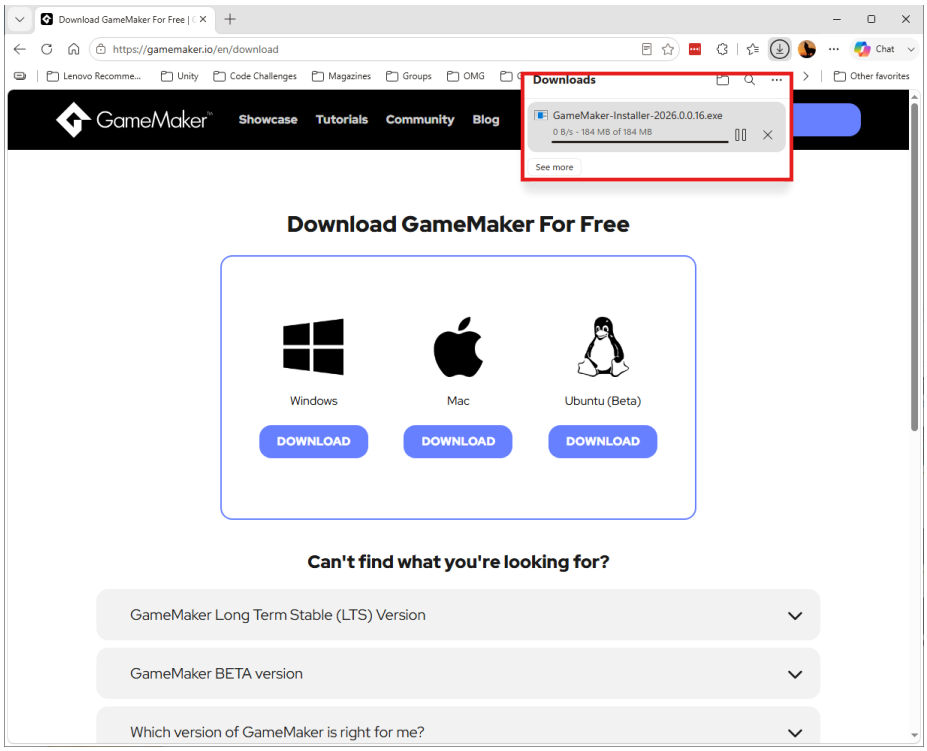
Screenshots from an actual run-through, in order. Circled/boxed elements mark exactly what to click or check at that step.



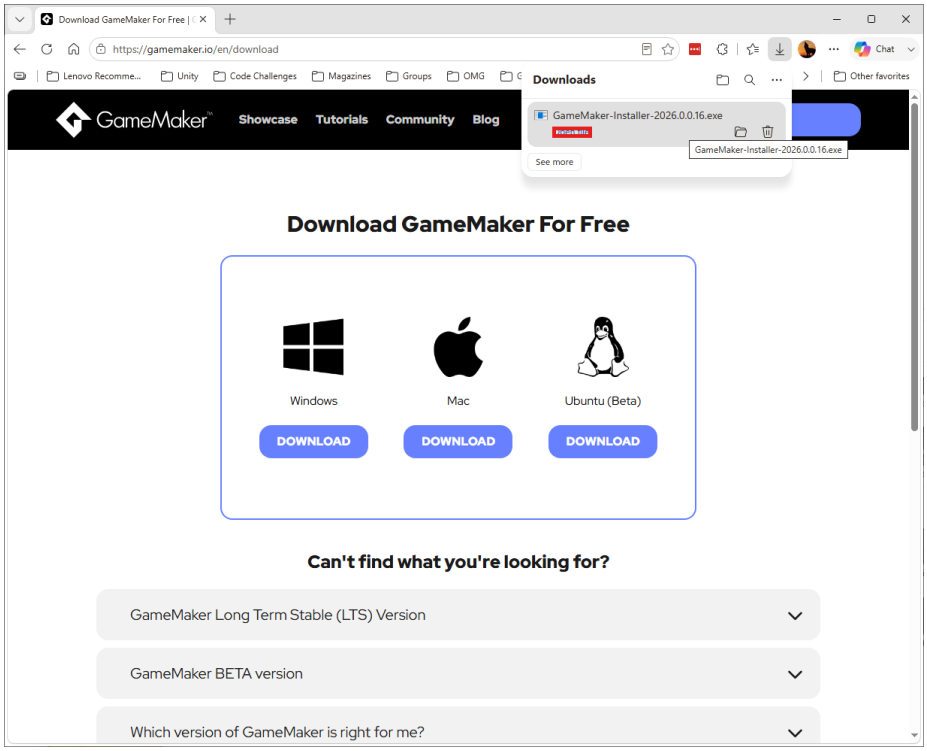
The download page - accept cookies to continue.



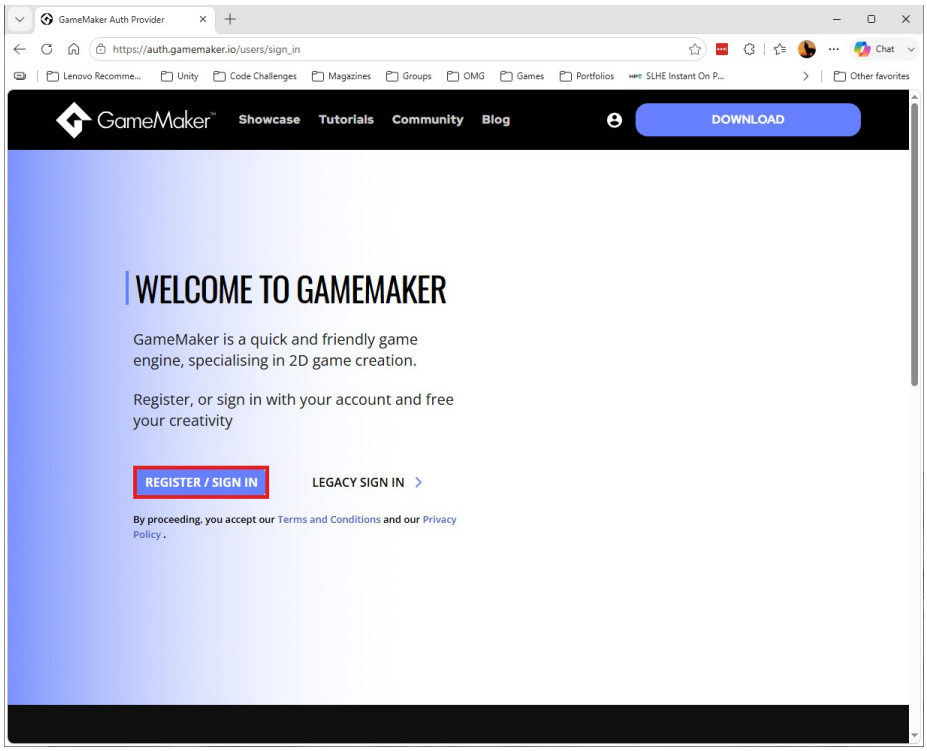
Click Download under Windows.



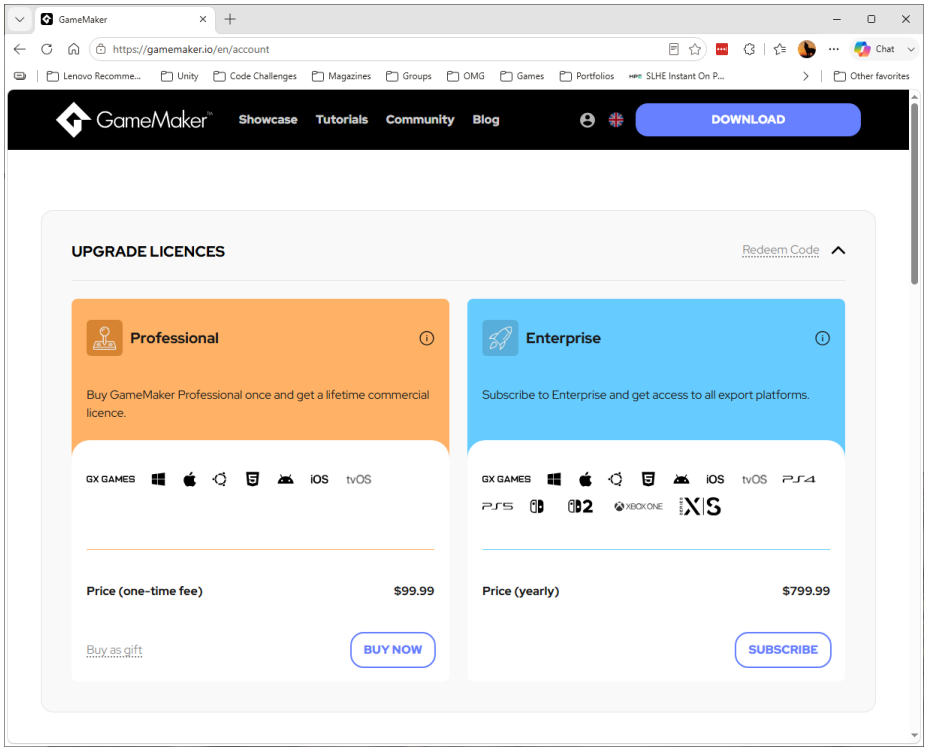
The installer downloading (~184 MB).



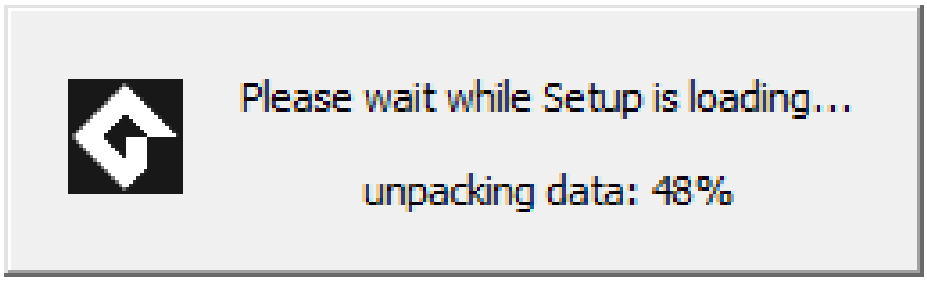
Click Open file to launch the installer.



Click Register / Sign In to create or sign in to a GameMaker account.



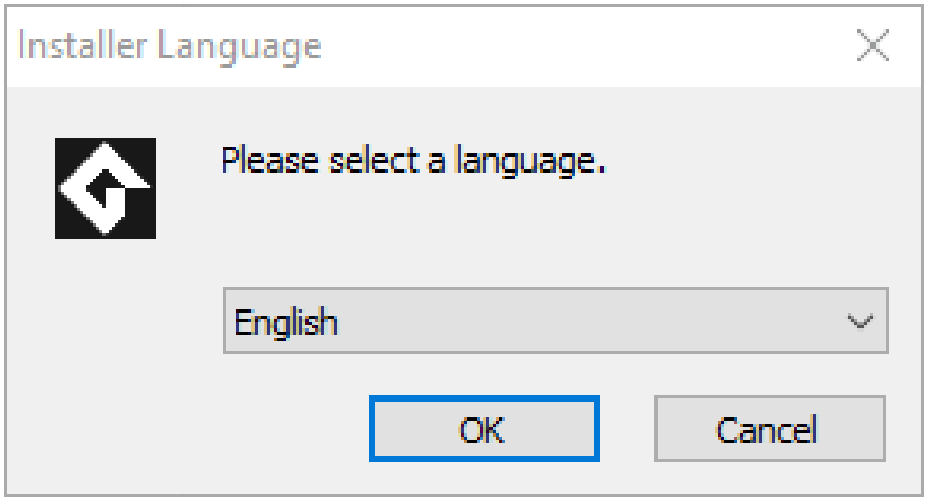
Your GameMaker account page - the free tier is all this class needs.



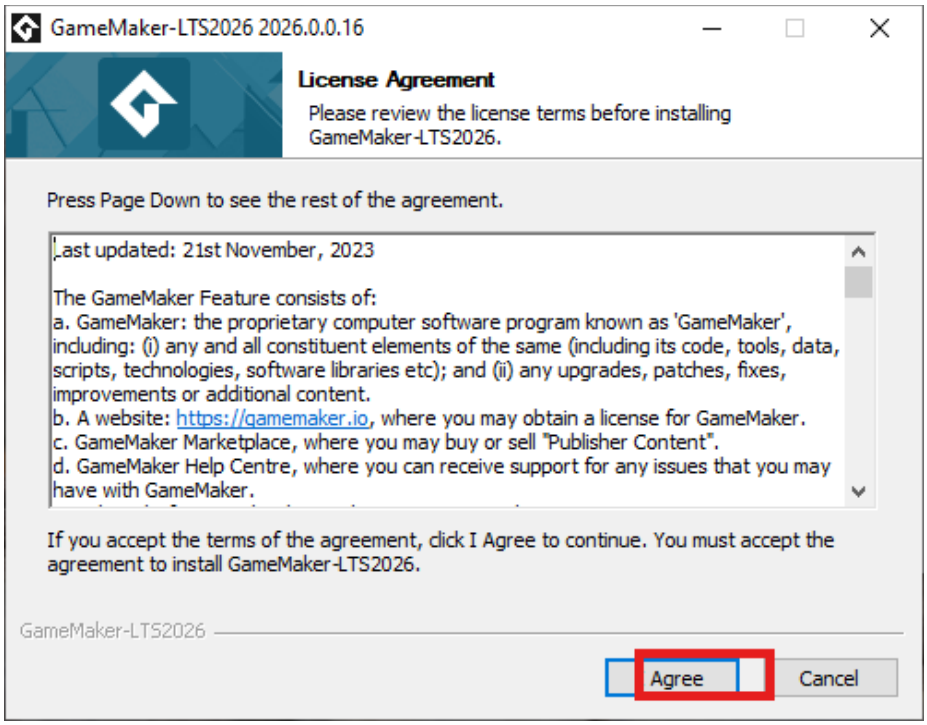
The installer unpacking itself.



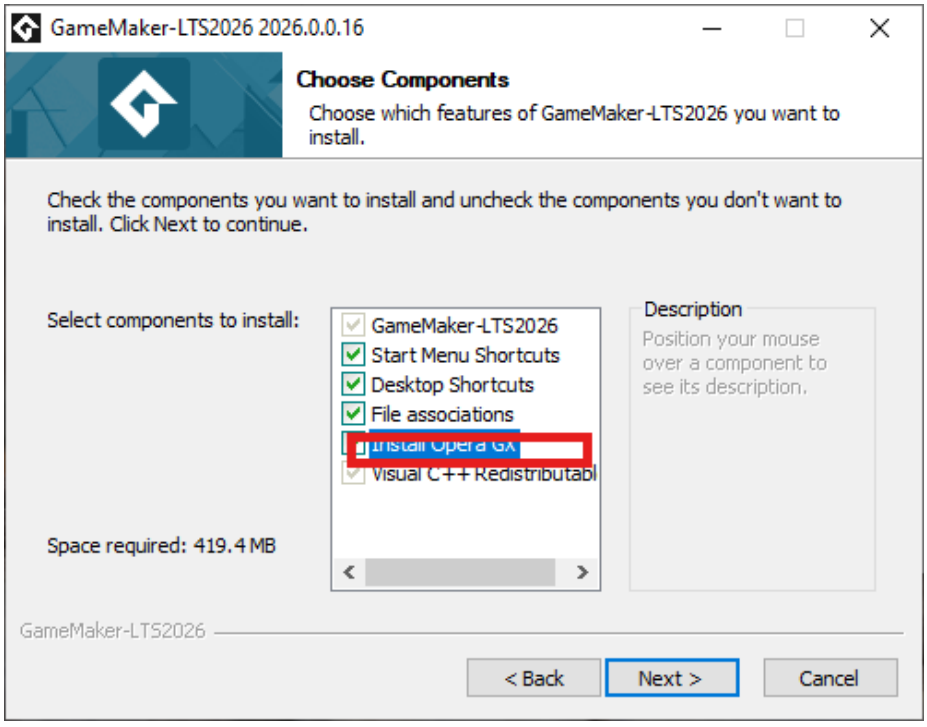
The installer's splash screen (LTS build).



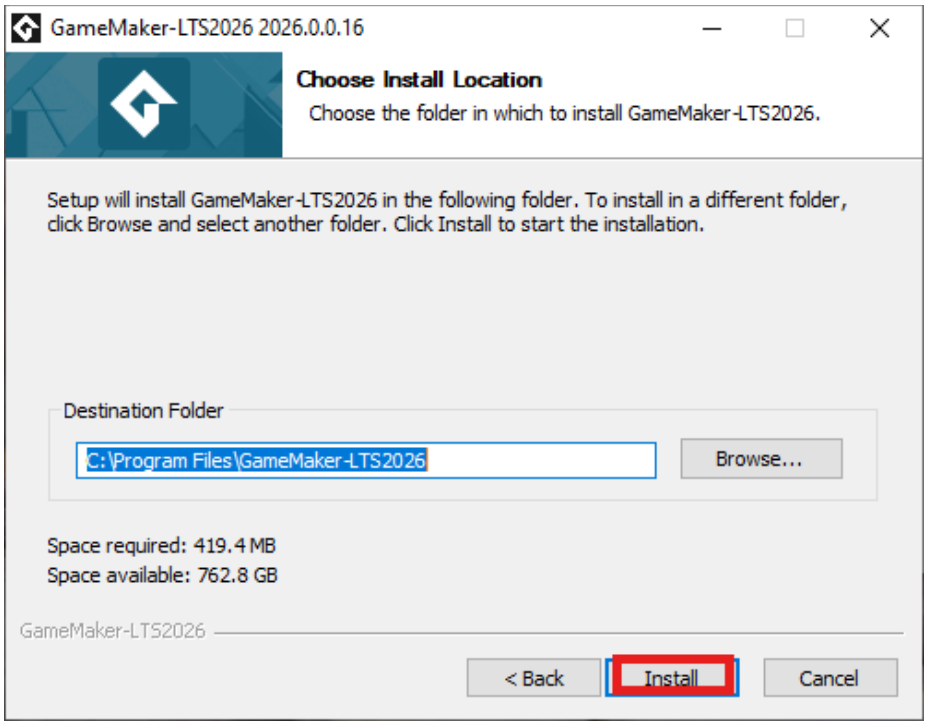
Choose a language and click OK.



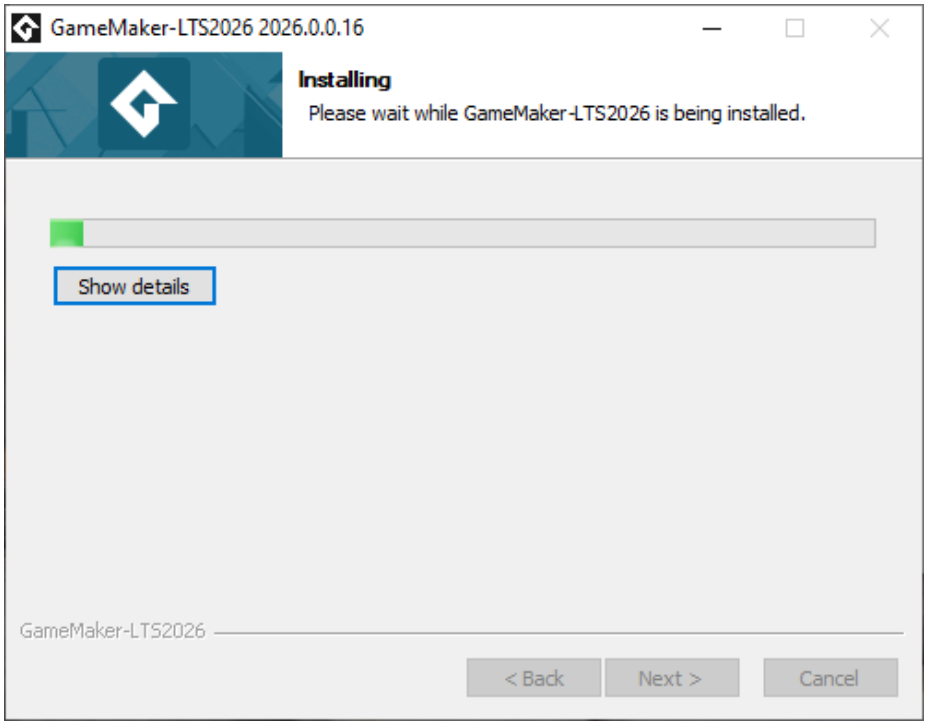
Click I Agree to accept the license.



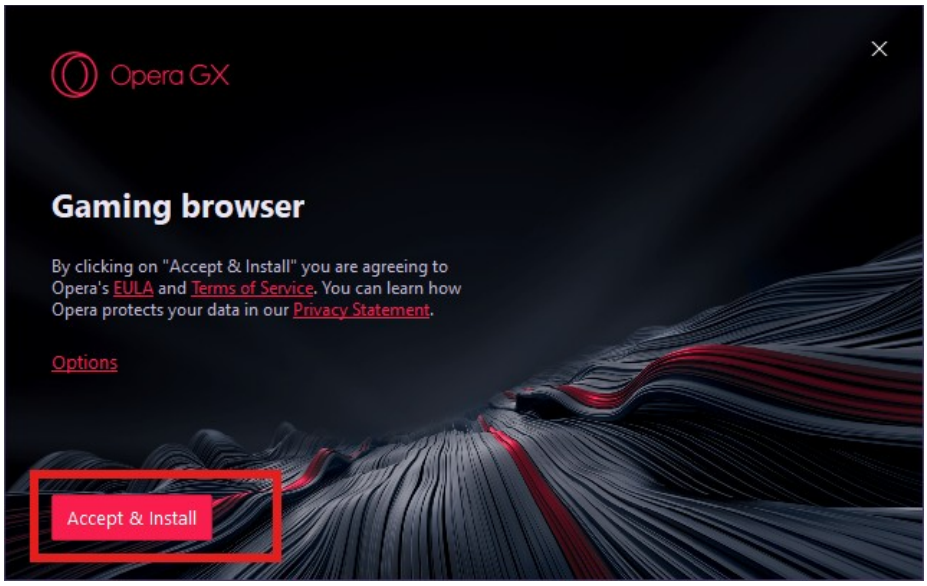
Install Opera GX is optional - leave it checked or uncheck it, either is fine.



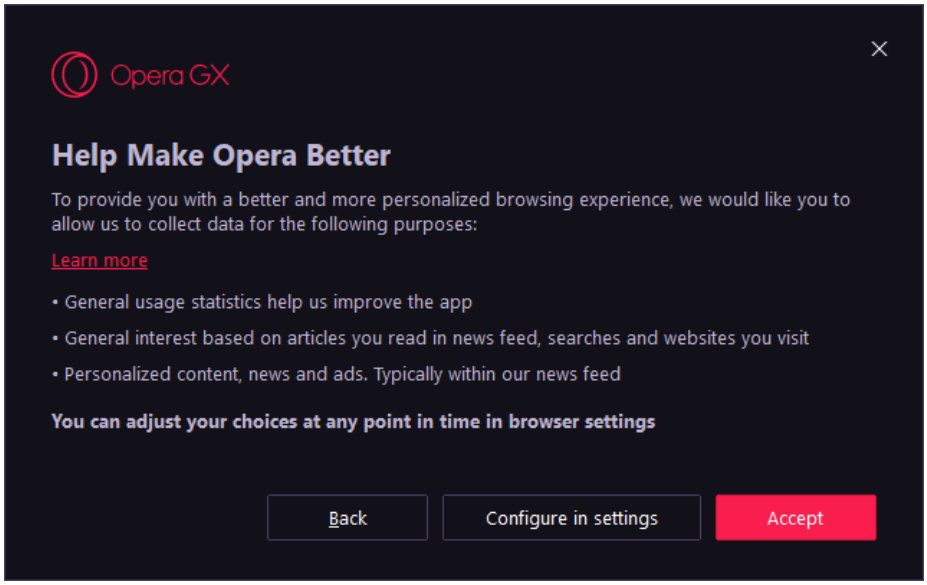
Confirm the install location and click Install.



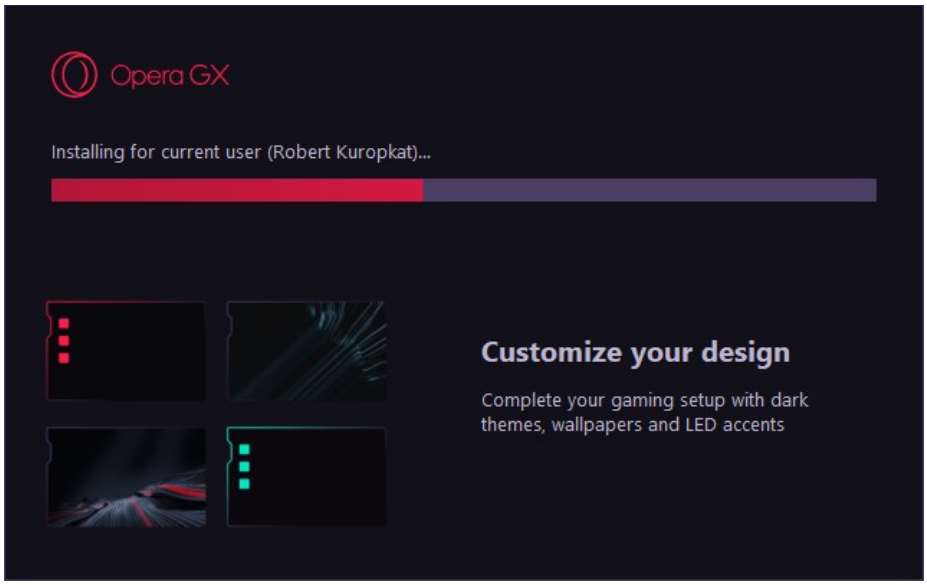
GameMaker installing.



If Opera GX was selected, its own installer screen appears - click Accept & Install to continue.



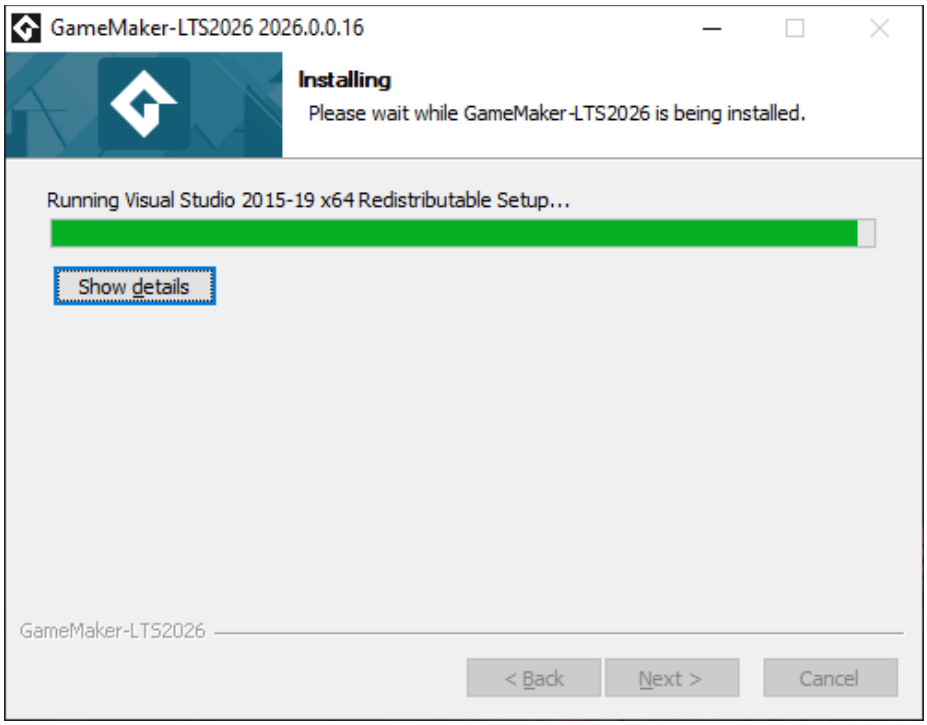
Opera's usage-statistics choice - your preference either way.



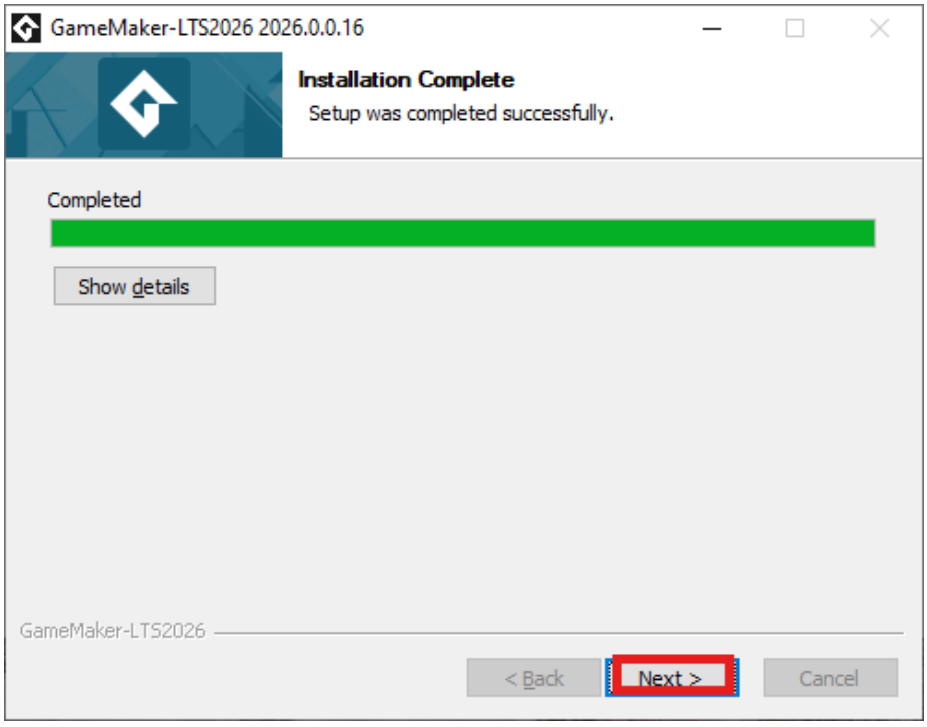
Opera GX installing.



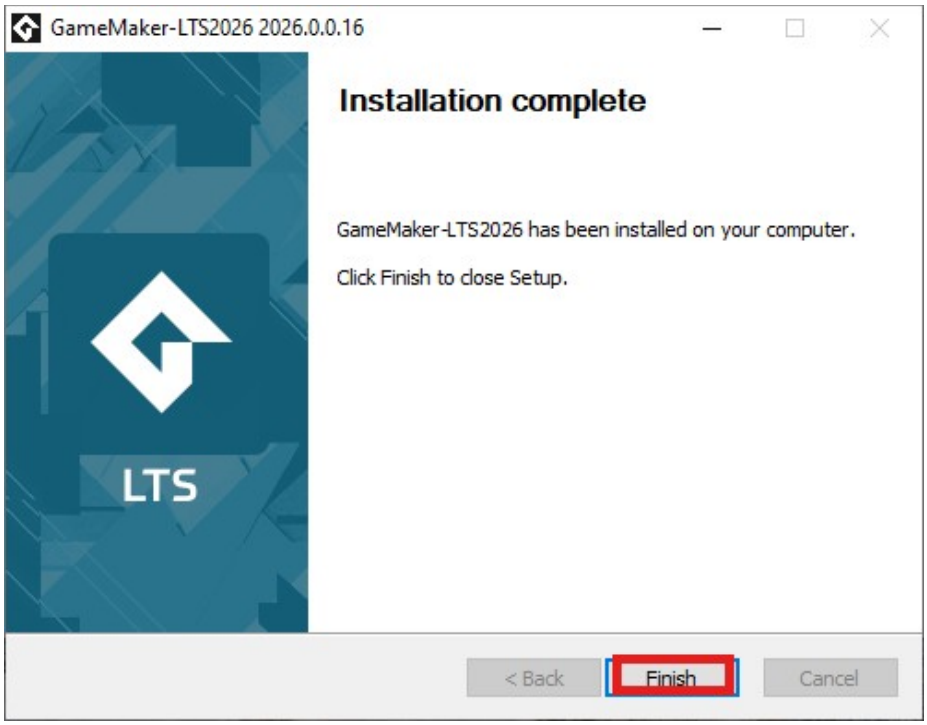
Opera GX's first-run tour - Skip is fine for this class.



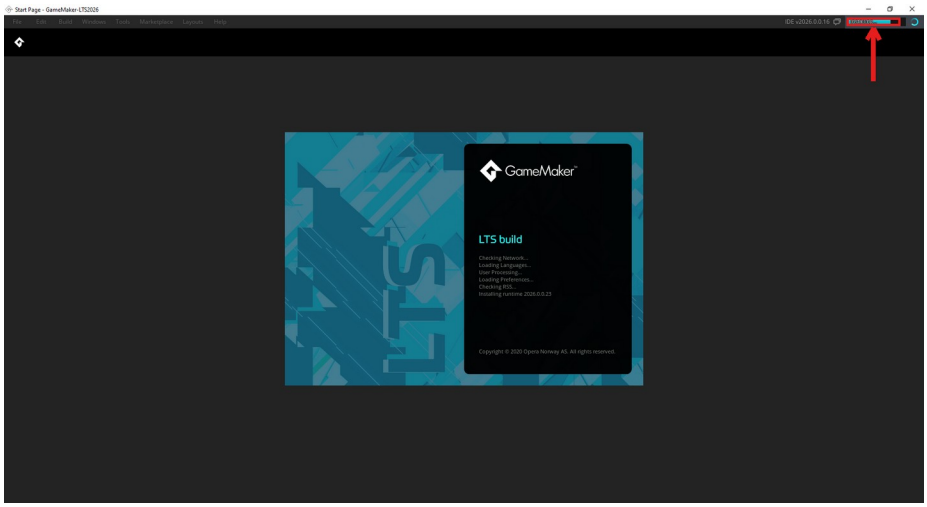
GameMaker continues installing its remaining components.



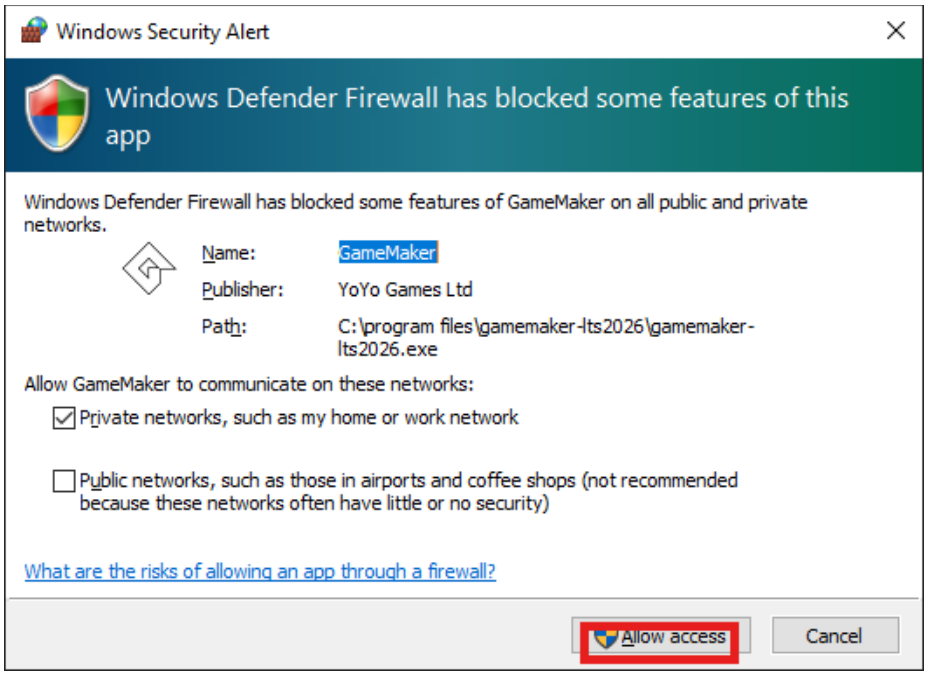
Installation Complete - click Next.



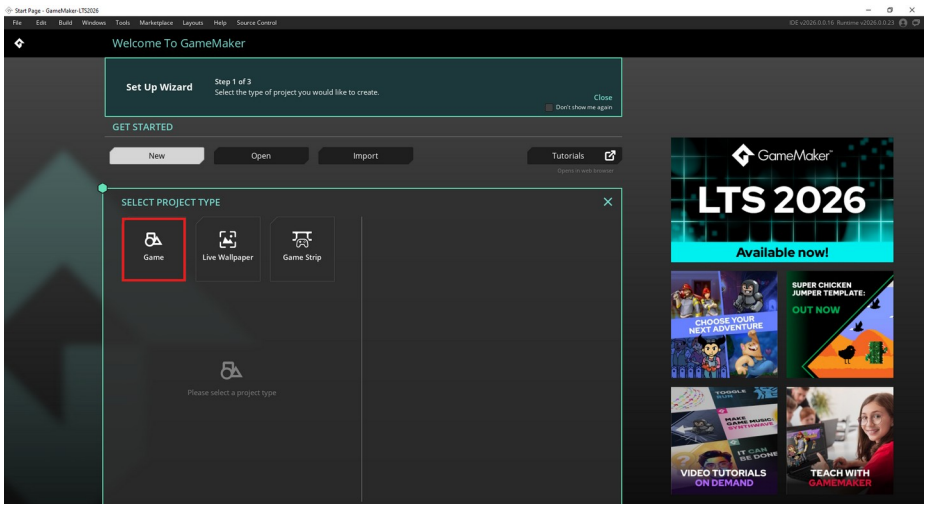
Click Finish to close the installer.



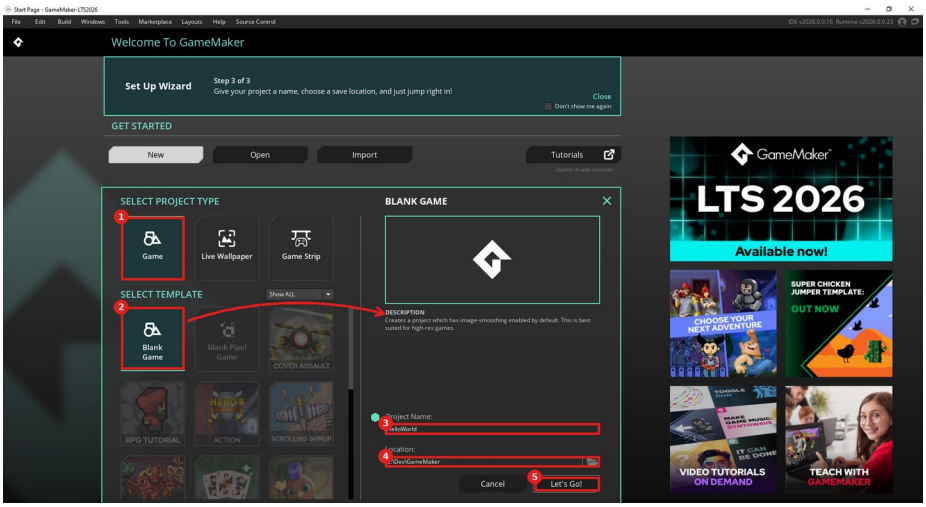
On first launch, watch the Installing indicator in the upper right while GameMaker finishes setting up.



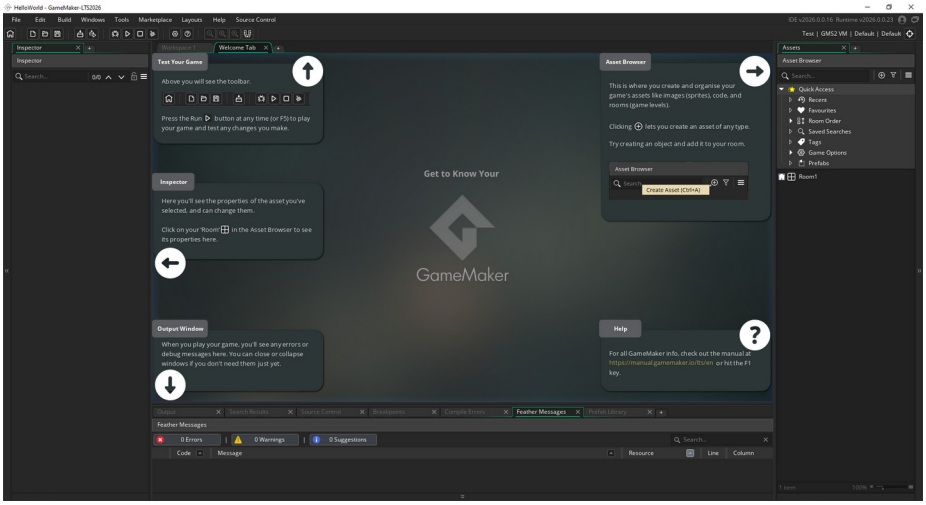
If Windows Defender Firewall prompts, click Allow access (admin privileges may be required).



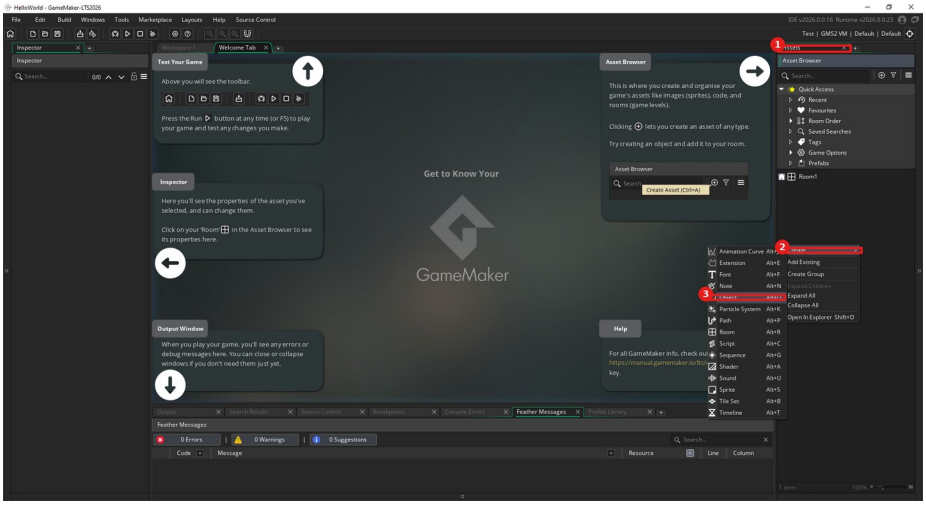
The Set Up Wizard - click the Game project type icon to begin.



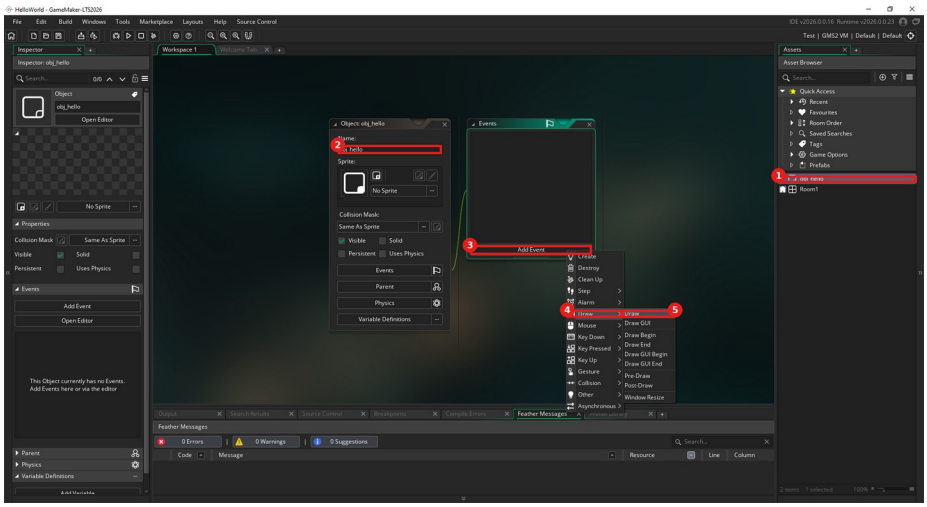
1) Game project type, 2) Blank Game template (arrow shows its description), 3) Project Name, 4) Location, 5) Let's Go!



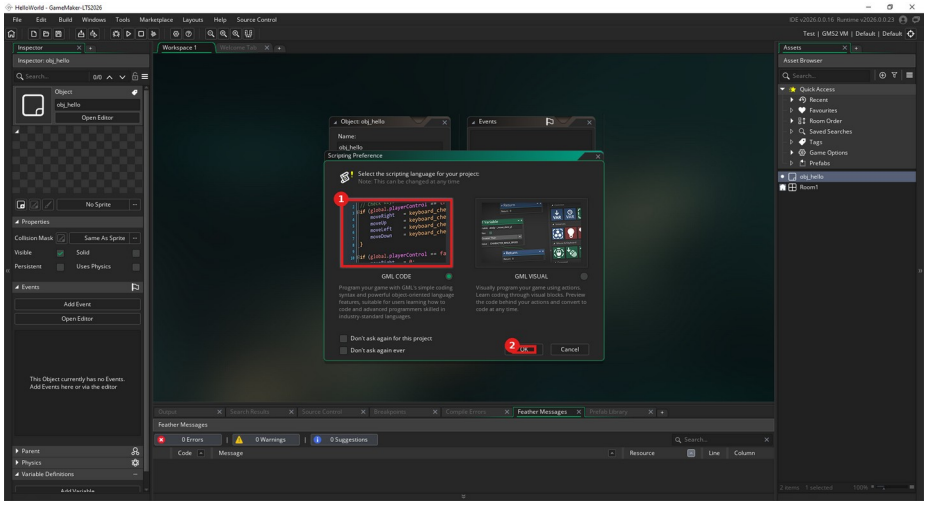
The new project opens with a built-in guided tour of the workspace.



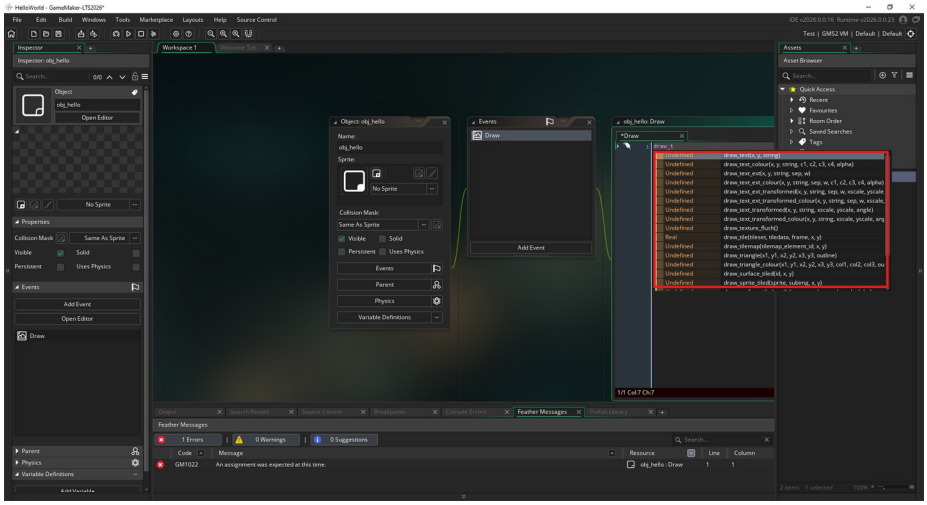
1) The Asset Browser tab, 2) Create in the first popup menu, 3) Object in the submenu.



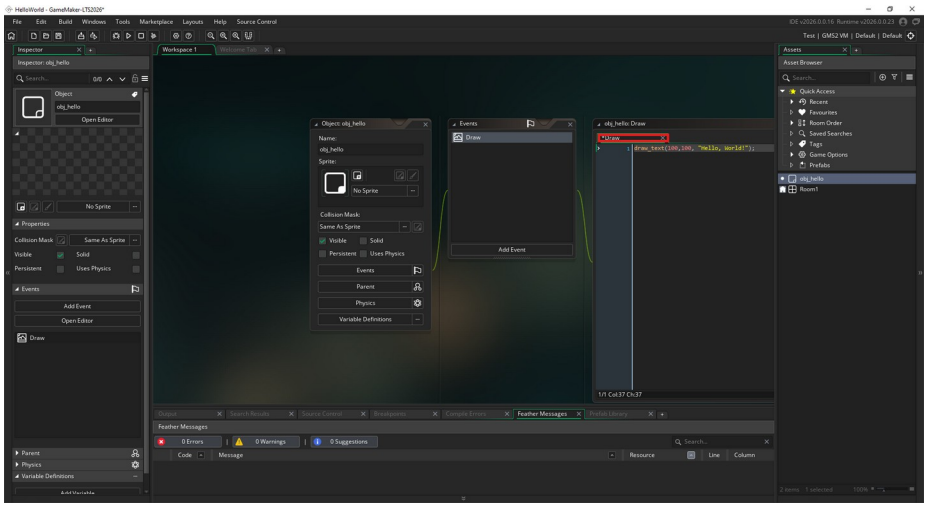
1) obj_hello renamed in the Asset Browser, 2) the same name in the object editor, 3) Add Event, 4) Draw in the first popup, 5) Draw in the submenu.



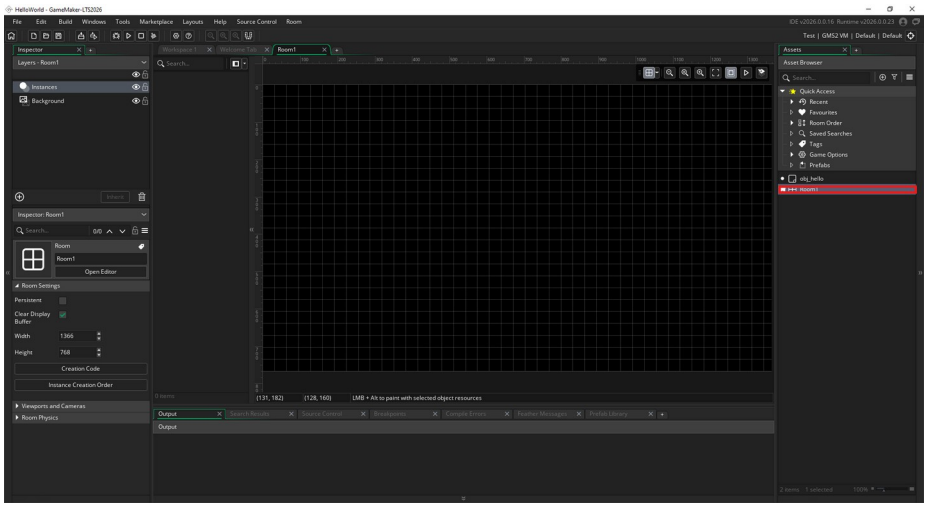
First code event: 1) choose GML Code, 2) click OK.



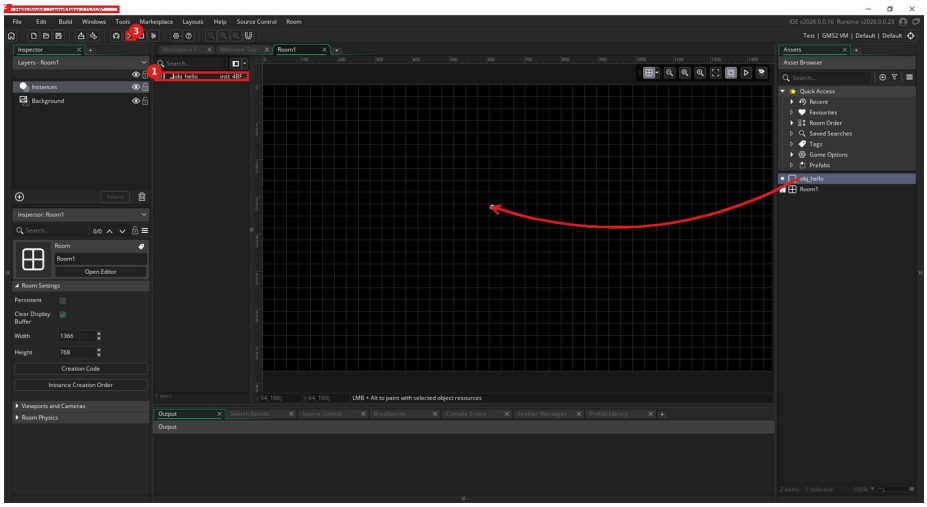
GameMaker's autocomplete suggests matching functions as you type.



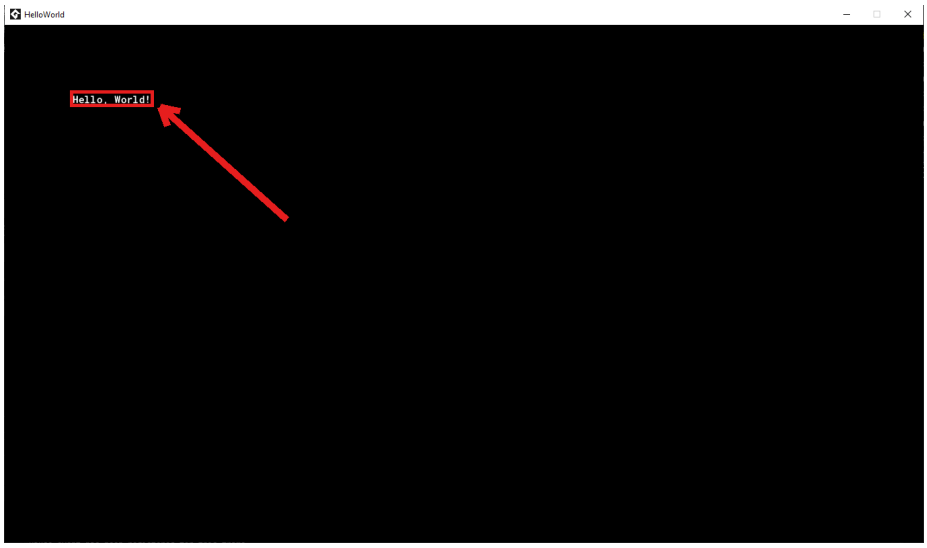
The asterisk on the Draw tab means the code isn't saved yet - press Ctrl+S to save it.



Double-click Room1 in the Asset Browser to open the Room Editor.



Drag obj_hello into the room (arrow) - 1) it appears in the room's instance list, 2) the unsaved asterisk in the title bar (Ctrl+S saves everything), 3) click Run to test.



Success - "Hello, World!" appears in the game window.

5. GDevelop

Detailed Install

1. Go to gdevelop.io/download and click the Windows icon under Download the desktop app (the free, no-account browser version at gdevelop.io also works as a no-install backup).
2. The browser downloads the GDevelop installer (~142 MB).
3. Click Open file in the downloads flyout to launch it.
4. Choose who the software installs for - Anyone who uses this computer needs admin credentials; Only for me doesn't. Either is fine for this class.
5. Confirm the install location (the default is fine) and click Install.
6. Wait for GDevelop to install.
7. On the Completing GDevelop 5 Setup screen, leave Run GDevelop 5 checked and click Finish.

Hello, World! Verification

1. On first launch, GDevelop's Start Page opens - click Log in if you already have an account, or Sign up to create one.
2. Sign in with Google, GitHub, Apple, or an email address, then click Login (or click Create an account if you're new).
3. Once signed in, your account name appears top right - click Create in the left-hand navigation.
4. Click + Create new game.
5. Choose Empty project from the template grid.
6. Pick a screen size (Desktop & Mobile landscape works fine for this class), set Project name to HelloWorld, confirm Where to store this project (GDevelop Cloud or Your computer both work), then click Create new game.
7. The new project opens directly into the Scene Editor - in the Objects panel (right side), click Add object.
8. Search "text" and choose Text (Displays a text on the screen).

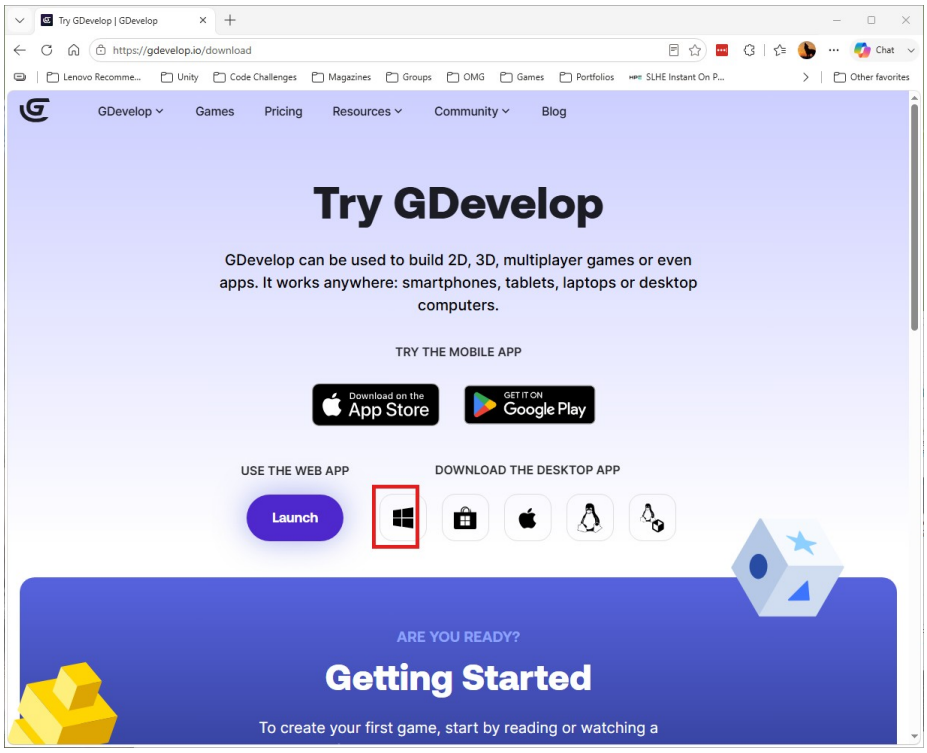
9. In the Edit dialog, set Object name to HelloWorld and Initial text to display to "Hello, World!", then click Apply.
10. Drag the new HelloWorld object from the Objects panel into the middle of the scene canvas - notice the save icon in the top-left toolbar now shows an unsaved dot; press Ctrl+S to save.
11. Click Preview in the toolbar to test.
12. If a Windows Defender Firewall prompt appears, click Allow access - GDevelop's preview needs this to run its local server.

Expected result: A separate "Preview of HelloWorld" window opens showing the text "Hello, World!" near the center of the screen.

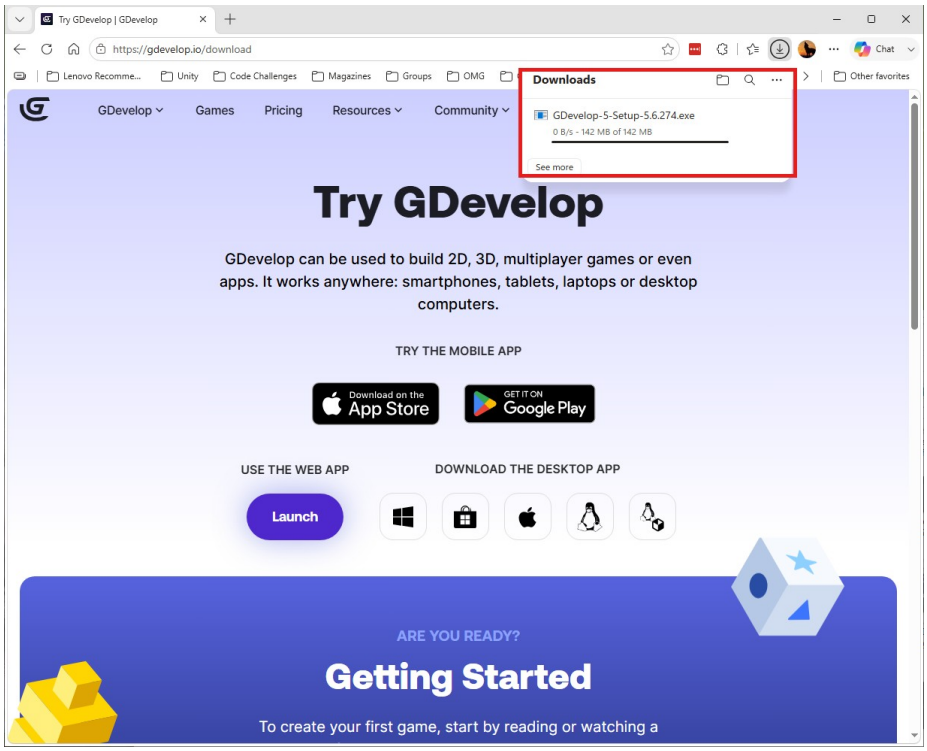
If something's wrong: *If nothing appears, confirm the object was actually dragged into the scene canvas (not just created in the Objects panel) and that Ctrl+S saved the change. If the Preview window doesn't seem to open at all, check for a Windows Defender Firewall prompt hiding behind other windows.*

Visual Walkthrough (captured on GDevelop 5.6.274)

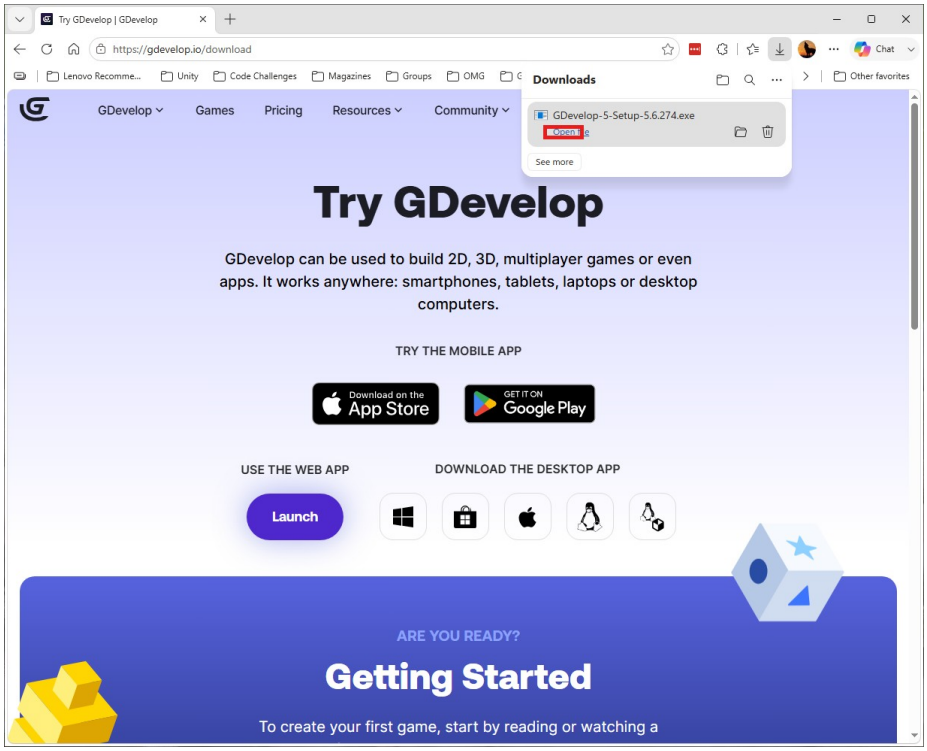
Screenshots from an actual run-through, in order. Circled/boxed elements mark exactly what to click or check at that step.



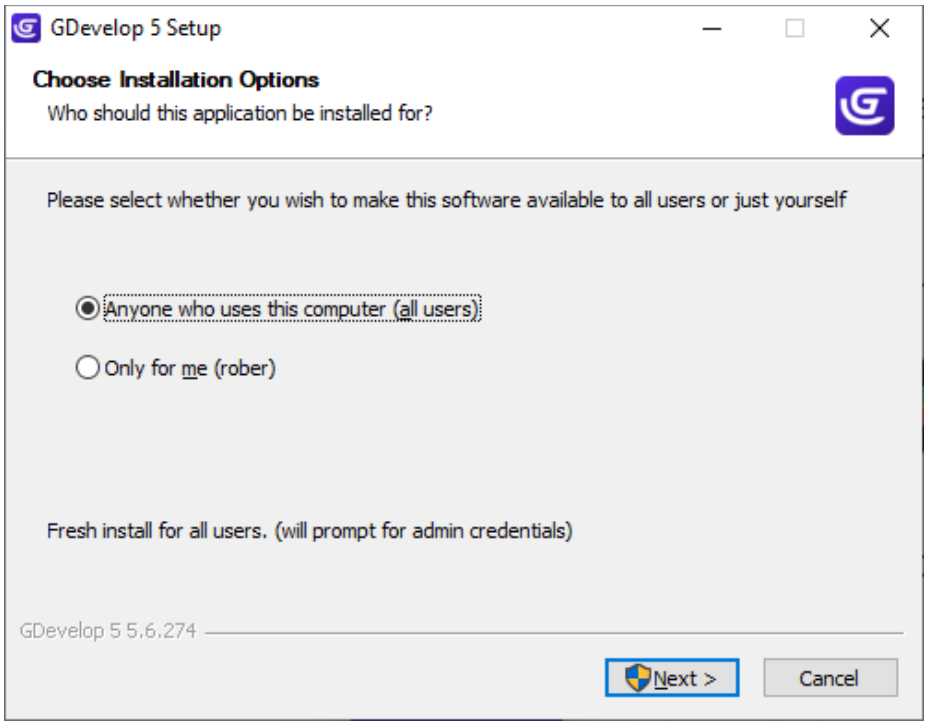
Click the Windows icon under Download the desktop app.



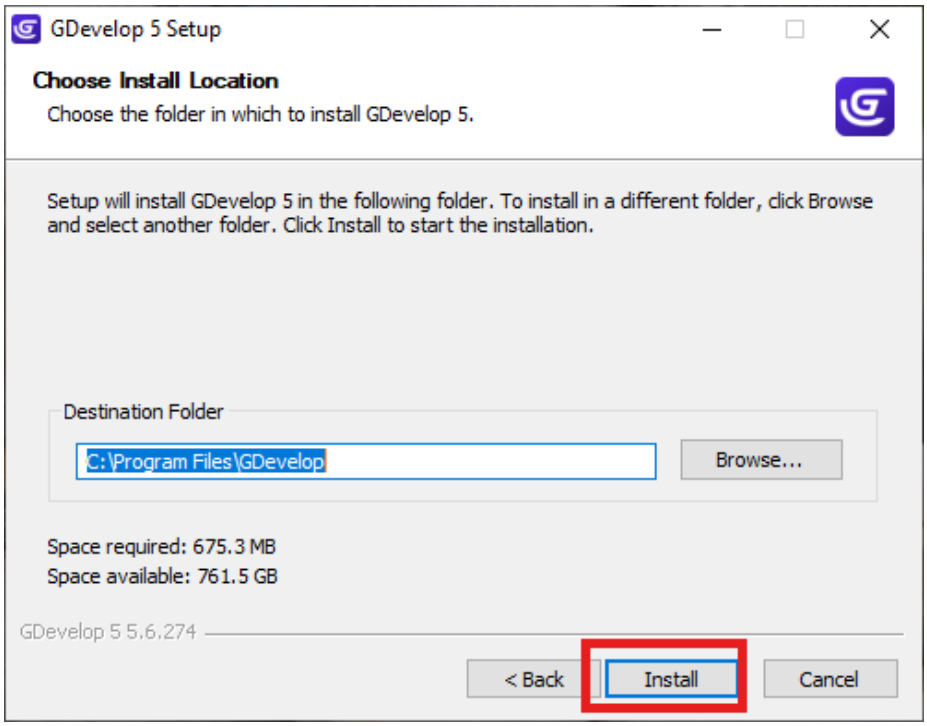
The installer downloading (~142 MB).



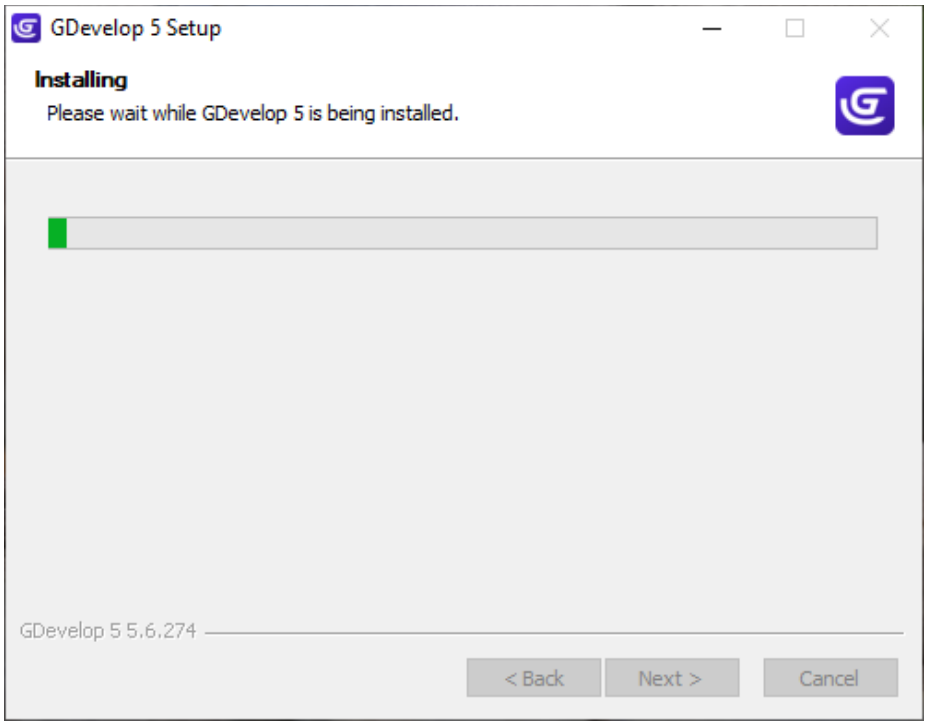
Click Open file to launch the installer.



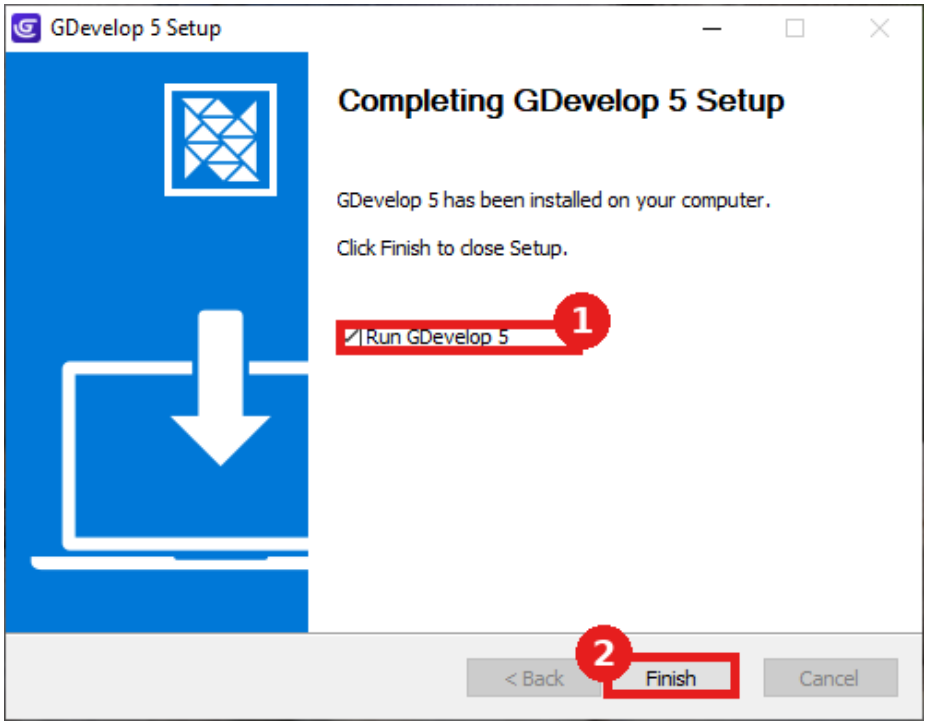
Choose who the software installs for - "Anyone" needs admin credentials, "Only for me" doesn't. Either is fine for this class.



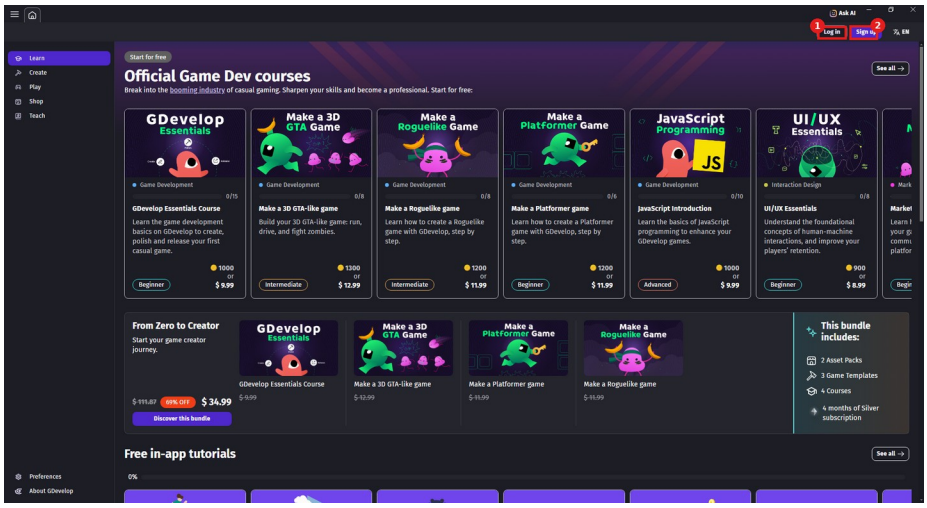
Confirm the install location and click Install.



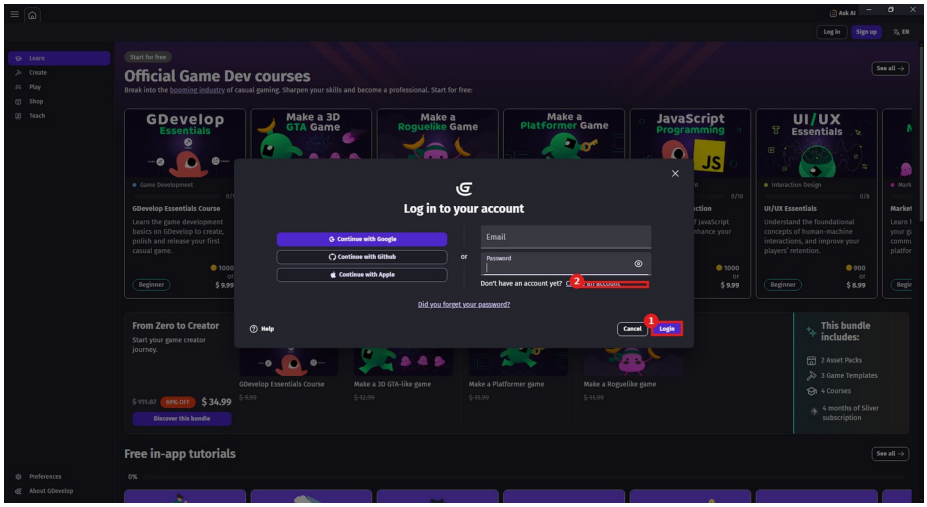
GDevelop installing.



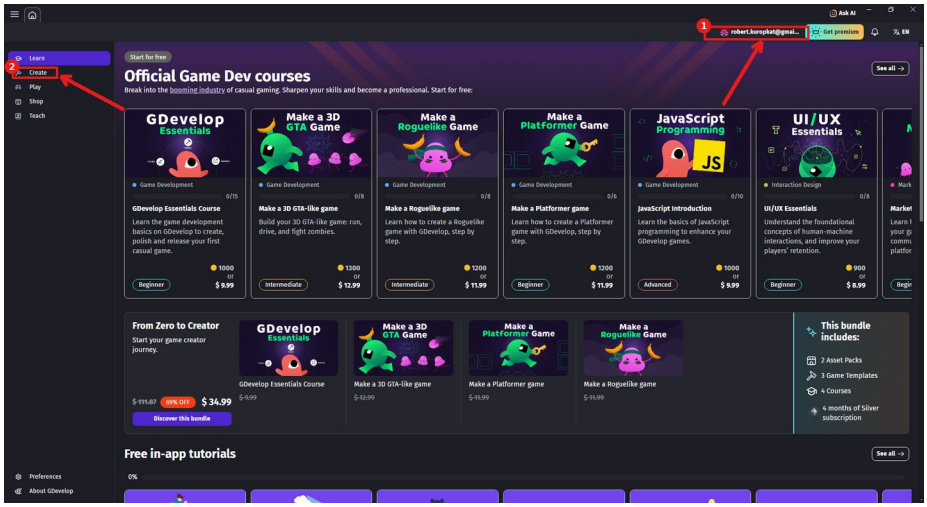
1) Leave Run GDevelop 5 checked, 2) click Finish.



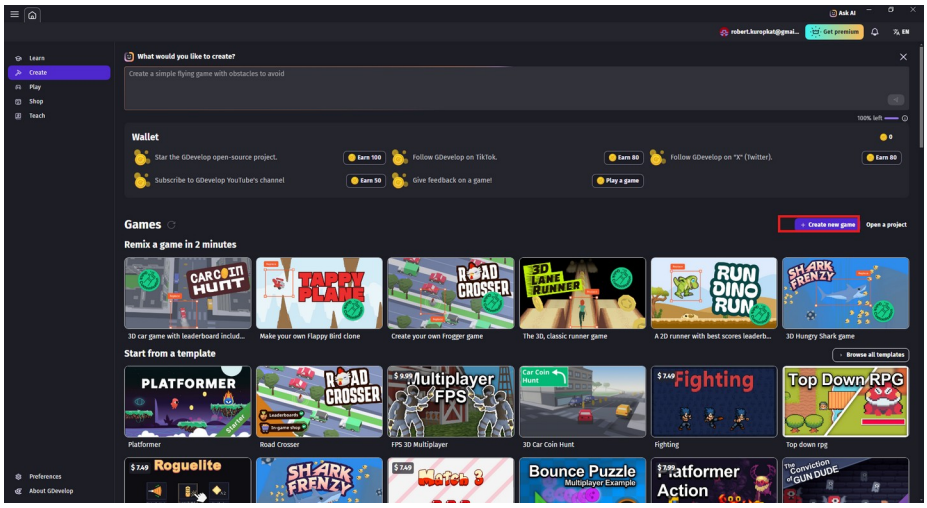
The Start Page - 1) Log in if you already have an account, 2) Sign up to create one.



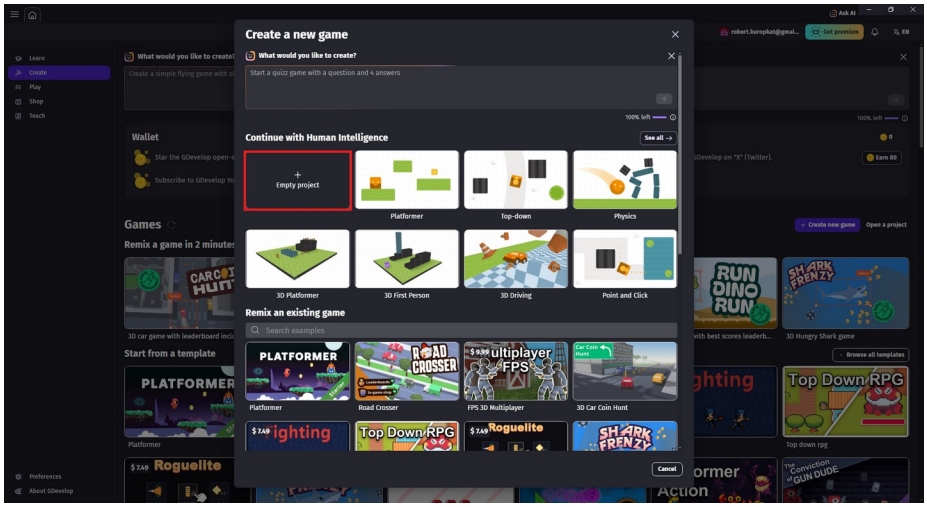
1) Sign in with Google, GitHub, Apple, or email and click Login, 2) or click Create an account if you're new.



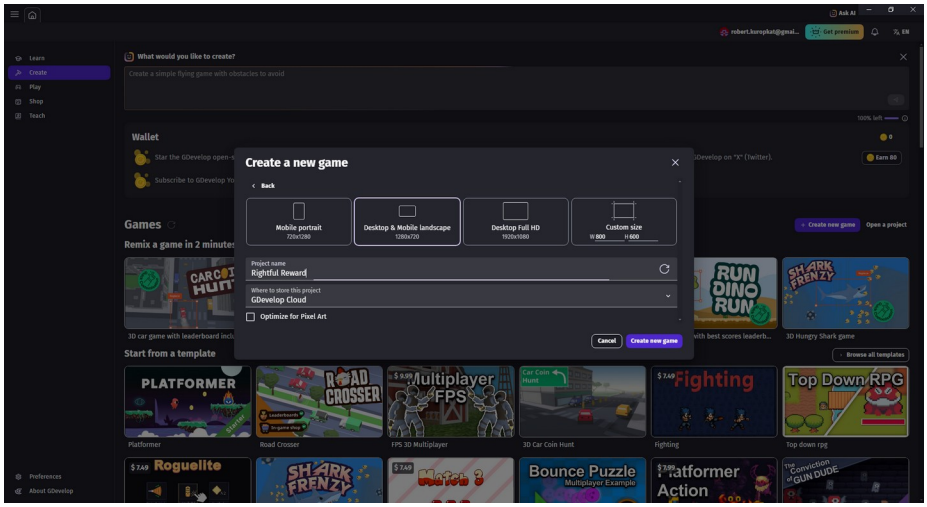
1) Your account name appears top right once signed in, 2) click Create in the left navigation.



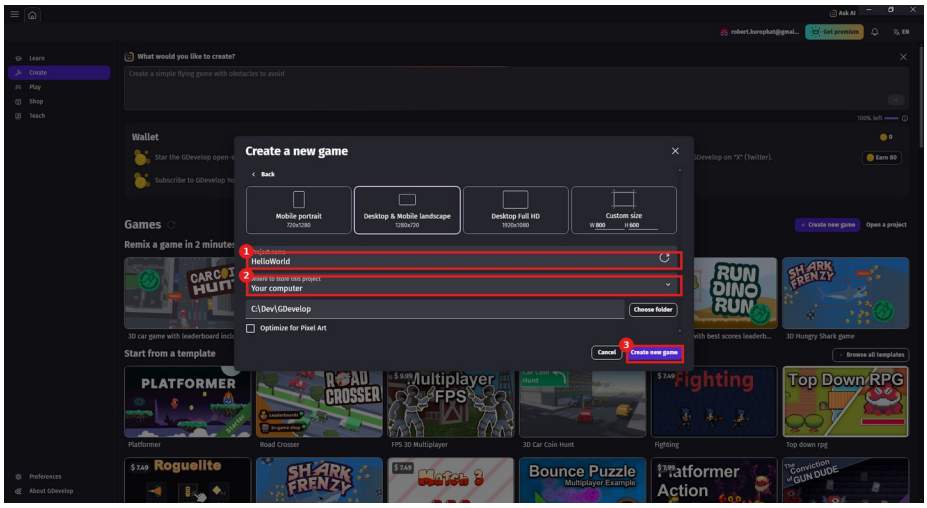
Click + Create new game.



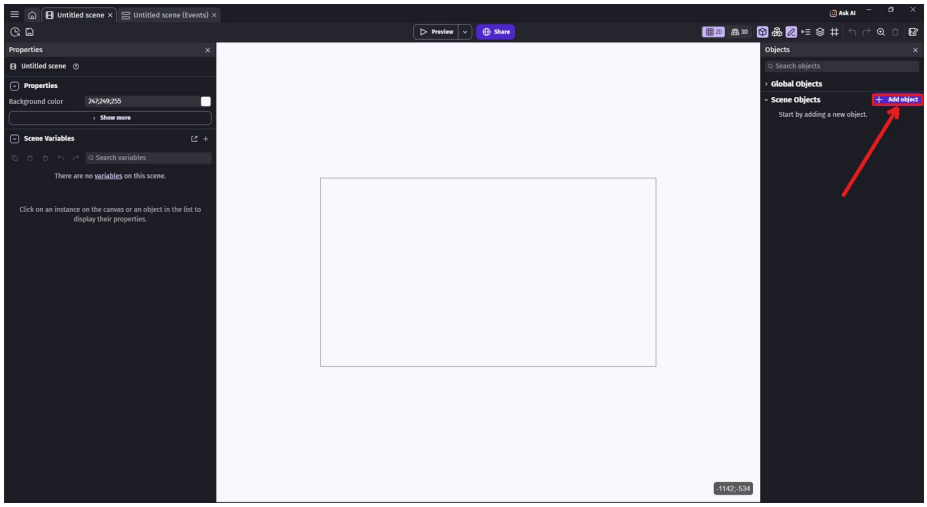
Choose Empty project.



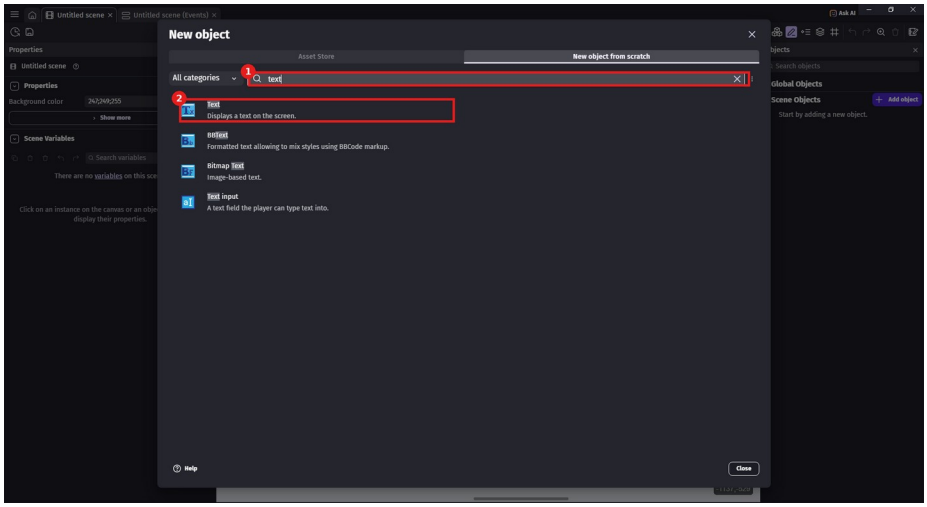
Pick a screen size - Desktop & Mobile landscape works fine for this class.



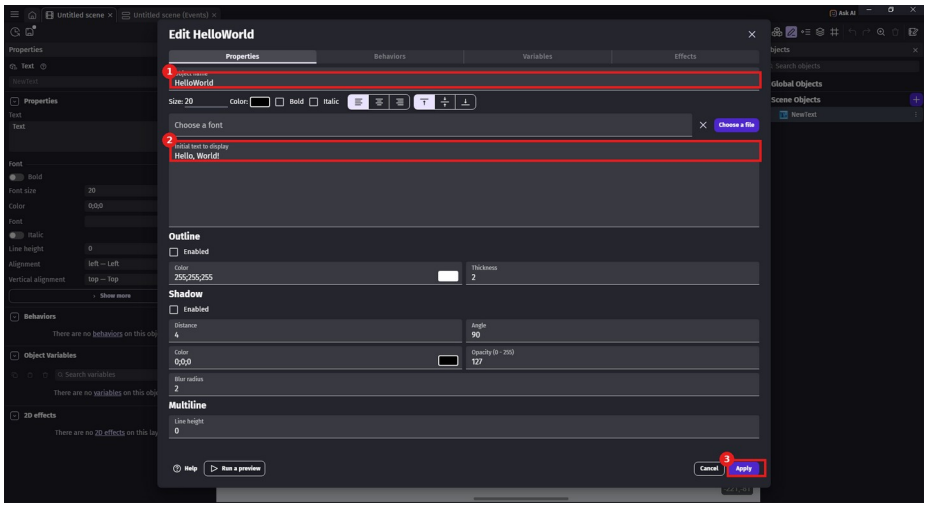
1) Set Project name to HelloWorld, 2) confirm where to store the project, 3) click Create new game.



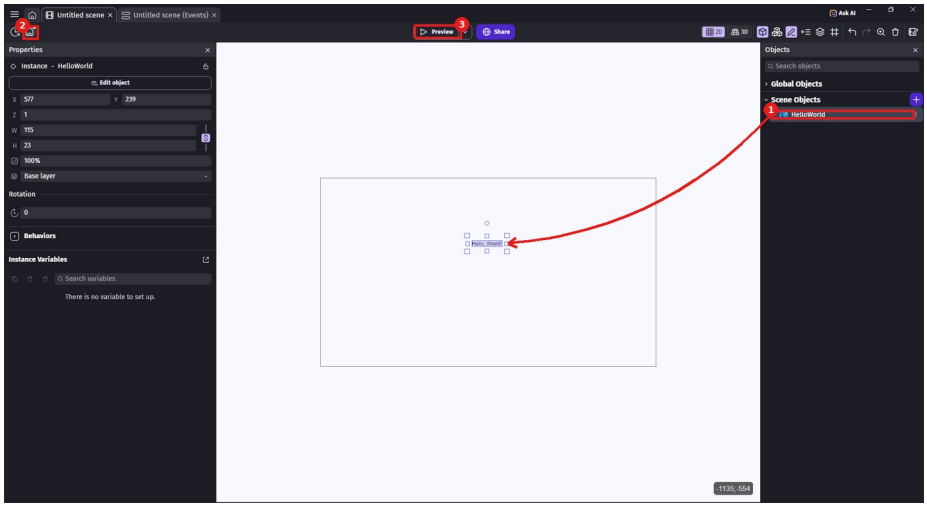
The Scene Editor opens directly - click Add object in the Objects panel.



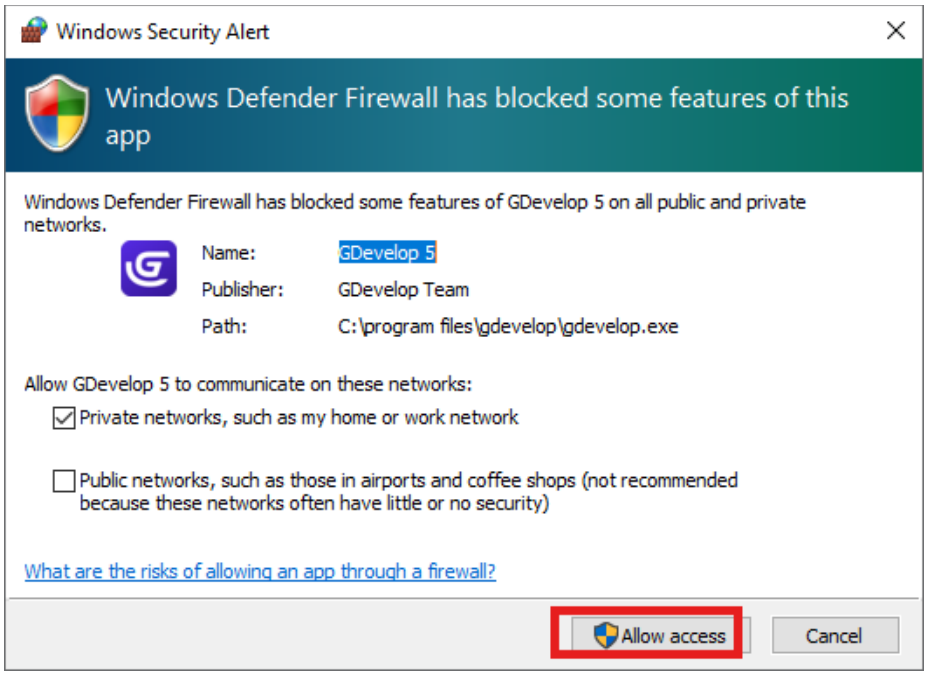
1) Search "text", 2) choose Text (Displays a text on the screen).



1) Set Object name to HelloWorld, 2) set Initial text to display to "Hello, World!", 3) click Apply.



Drag HelloWorld into the scene (arrow) - 1) it appears in Scene Objects, 2) the save icon shows an unsaved dot - press Ctrl+S, 3) click Preview to test.



If Windows Defender Firewall prompts, click Allow access - GDevelop's preview needs this for its local server.



Success - a separate Preview window shows "Hello, World!"

6. TIC-80

Detailed Install

1. Go to tic80.com/create and scroll down to Download - click tic80-v1.1-win.zip under Windows (the free build is all this class needs).
2. The browser downloads the zip - click Open file in the downloads flyout to reveal it in File Explorer (or just open your Downloads folder yourself).
3. Right-click the downloaded zip file and choose Extract All...
4. Confirm the destination folder (the default, right next to the zip, is fine) and click Extract.

5. Drag the extracted tic80 application to your Desktop for a shortcut you can double-click anytime.
6. Optional but recommended: right-click the tic80 icon, choose Properties, then on the Shortcut tab - 1) add a space followed by --fs "C:\Dev\Tic80" to the end of the Target field (two dashes before fs, the path in double quotes), 2) set Start in to that same folder, 3) click Apply. TIC-80's default working folder can be hard to find from the operating system - changing these properties makes your saved projects easy to locate afterward.
7. Double-click the tic80 icon to launch - there is no installer.

Hello, World! Verification

1. TIC-80 opens directly to its console prompt with a "hello! type help for help" message.
2. Type mkdir HelloWorld and press Enter to create a project folder.
3. Type dir and press Enter to confirm the new folder is there.
4. Type cd HelloWorld and press Enter to move into it.
5. Type new lua and press Enter - this creates a new Lua cart. Press Esc to open the Code Editor if it doesn't open automatically; it already contains TIC-80's default starter demo code.
6. In the Code Editor, delete the default t/x/y variable lines and the whole function TIC() body underneath them - we don't need this starter demo for Hello World.
7. Type the code below into the now-empty function TIC() body (feel free to update the title/author/desc/site comment lines at the top with your own info too).
8. Press Esc to return to the console, then type run and press Enter (or press Ctrl+R) to test.
9. Once it works, press Esc again to return to the console and type save HelloWorld, then press Enter to save your cart.

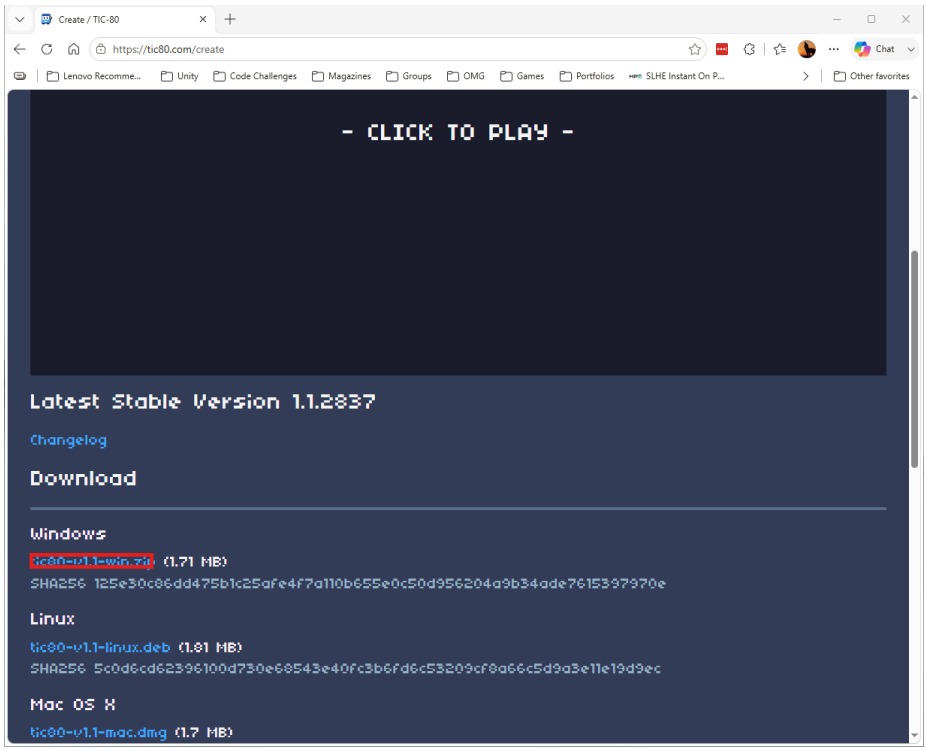
```
function TIC()
  cls(0)
  print("Hello, World!", 40, 60)
end
```

Expected result: The game screen shows "Hello, World!" printed near the middle-left of the screen in TIC-80's default font.

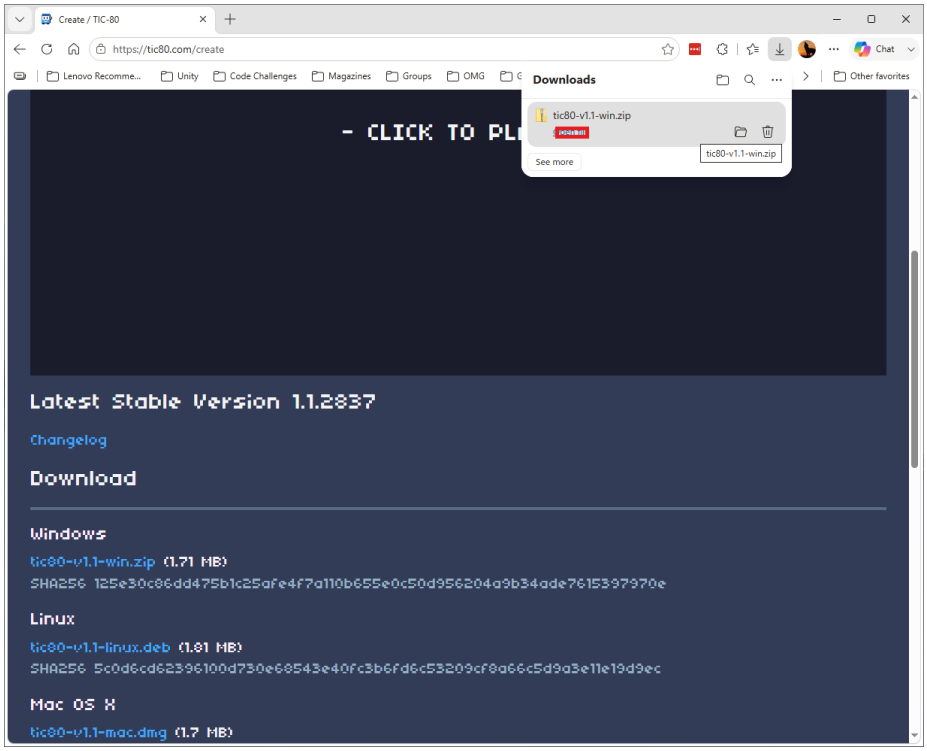
If something's wrong: *Make sure the TIC() function name is capitalized exactly as shown - TIC-80 calls this function every frame, and a typo means nothing will run. If commands like mkdir or cd don't seem to work, check for typos - the console is case-sensitive for folder names you create yourself.*

Visual Walkthrough (captured on TIC-80 1.1.2837)

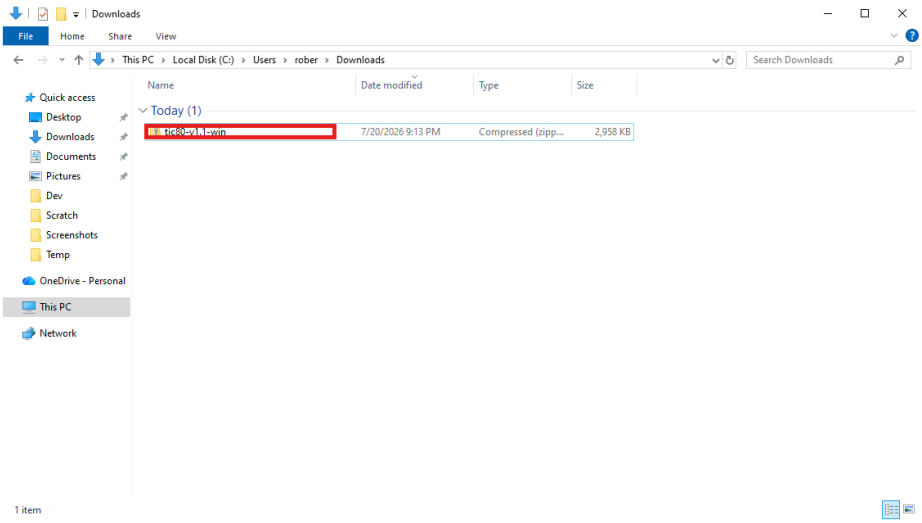
Screenshots from an actual run-through, in order. Circled/boxed elements mark exactly what to click or check at that step.



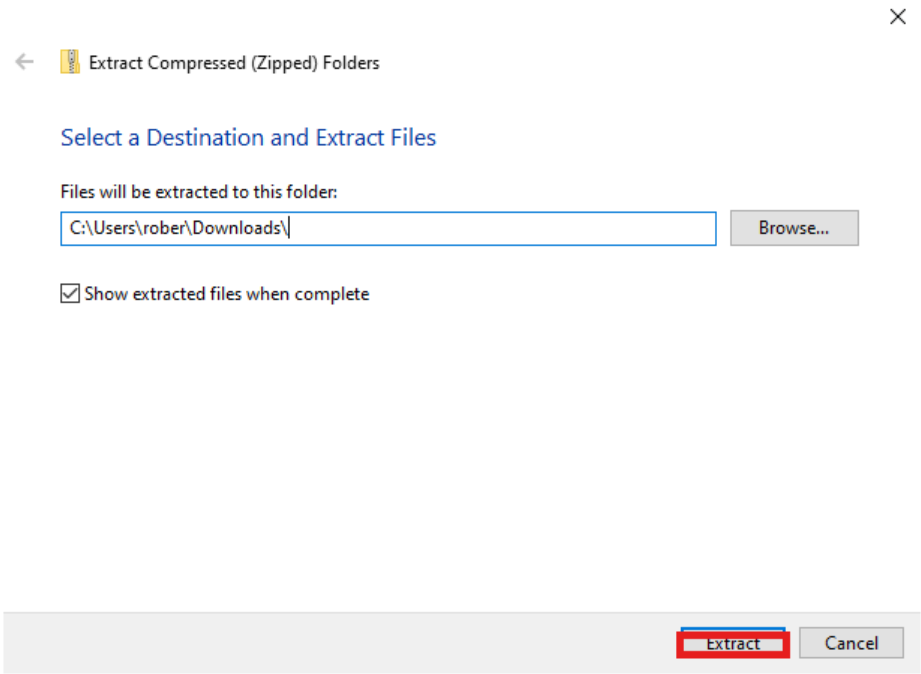
Scroll to Download and click tic80-v1.1-win.zip under Windows.



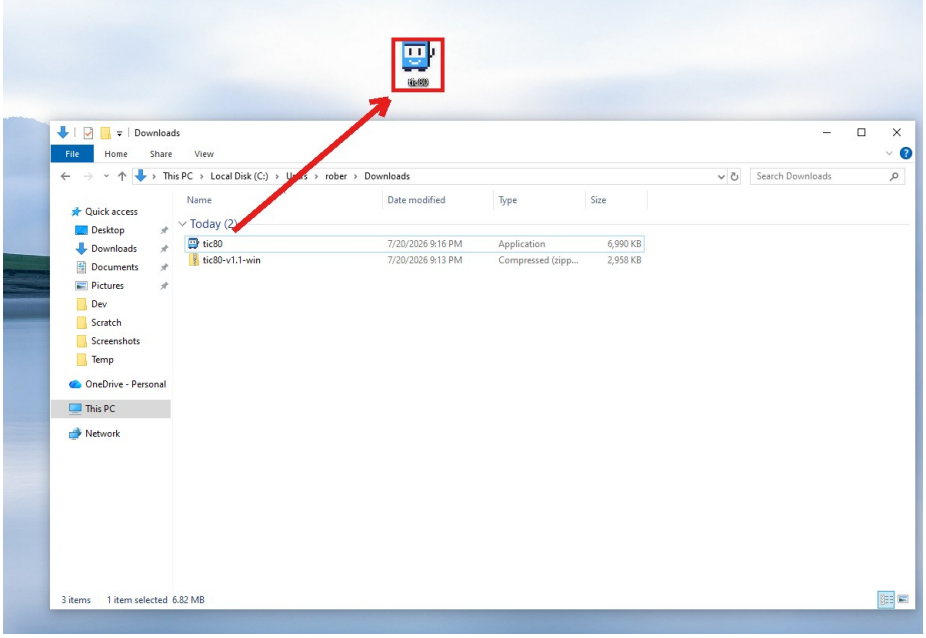
Click Open file to reveal the zip in File Explorer.



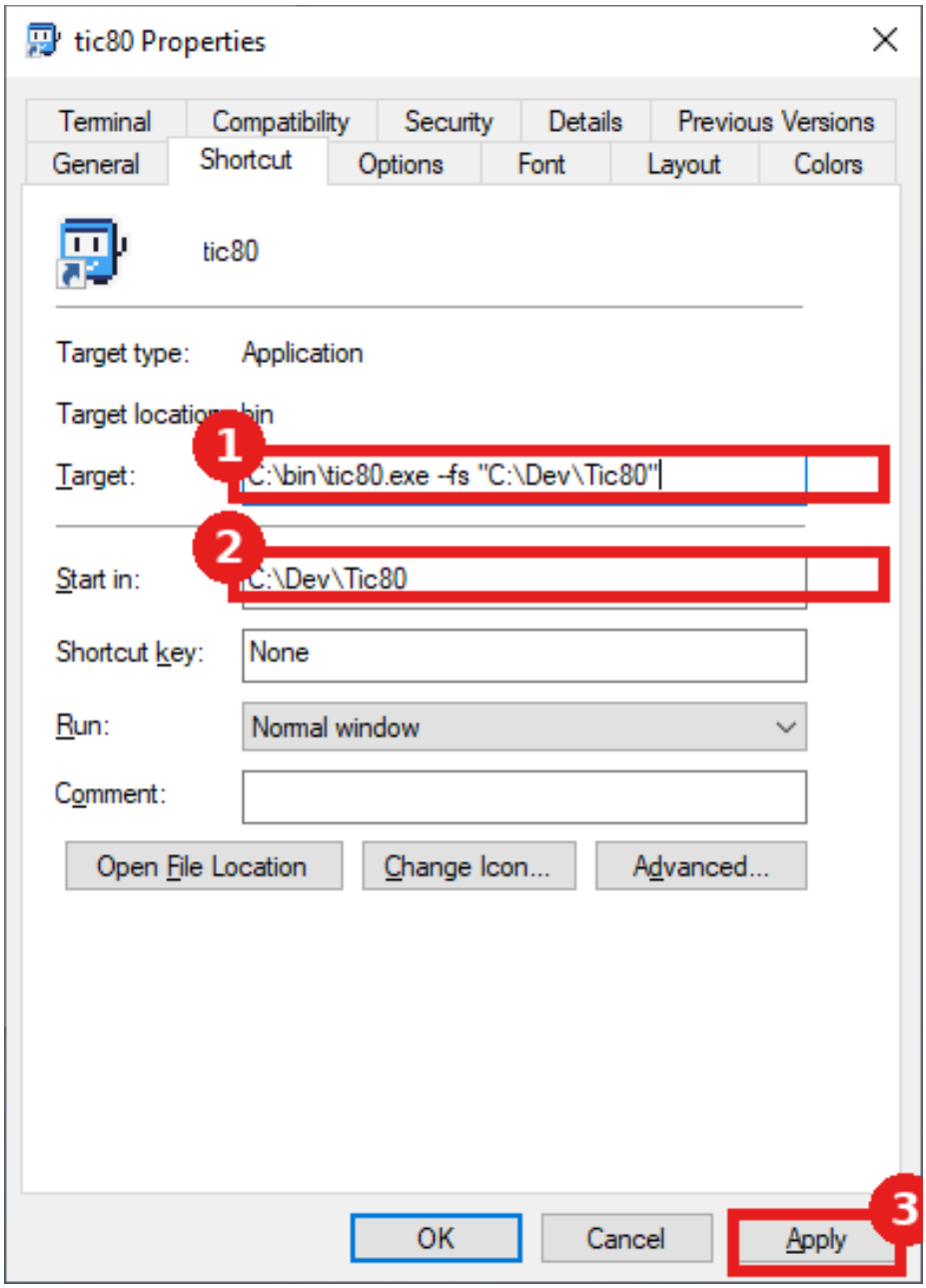
Right-click the downloaded zip file and choose Extract All...



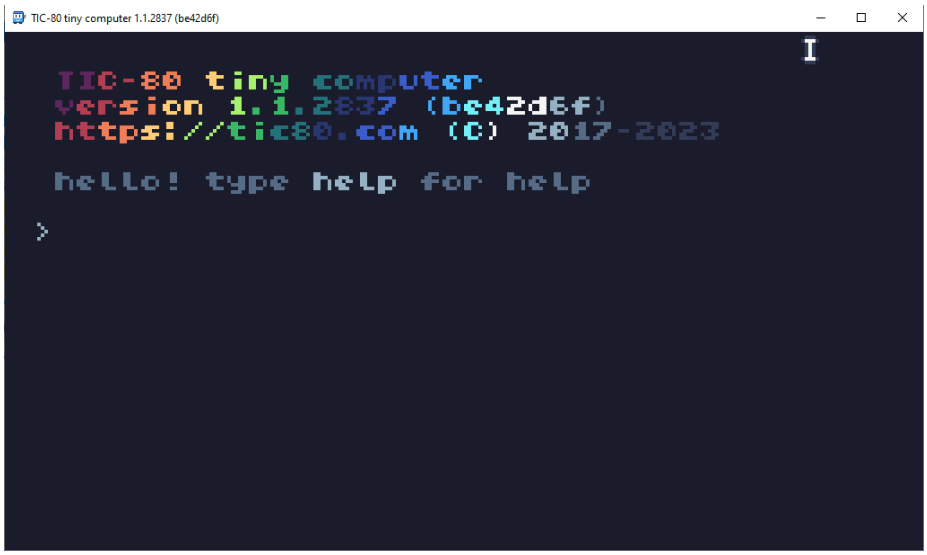
Confirm the destination folder and click Extract.



Drag the extracted tic80 application to your Desktop (arrow) for an easy shortcut.



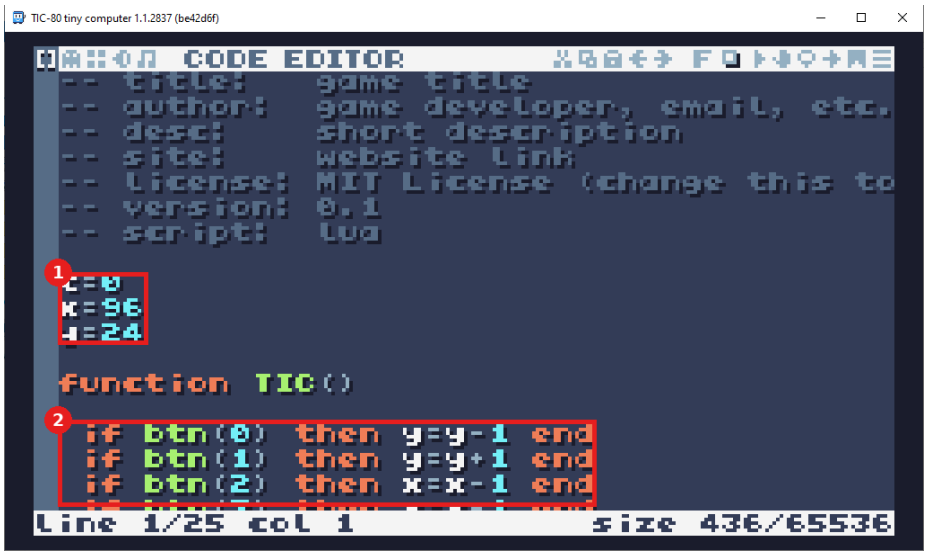
Optional: in the shortcut's Properties, 1) add the --fs "C:\Dev\Tic80" flag to Target, 2) set Start in to match, 3) click Apply - this makes your saved projects easy to find later.



TIC-80 opens directly to its console prompt.



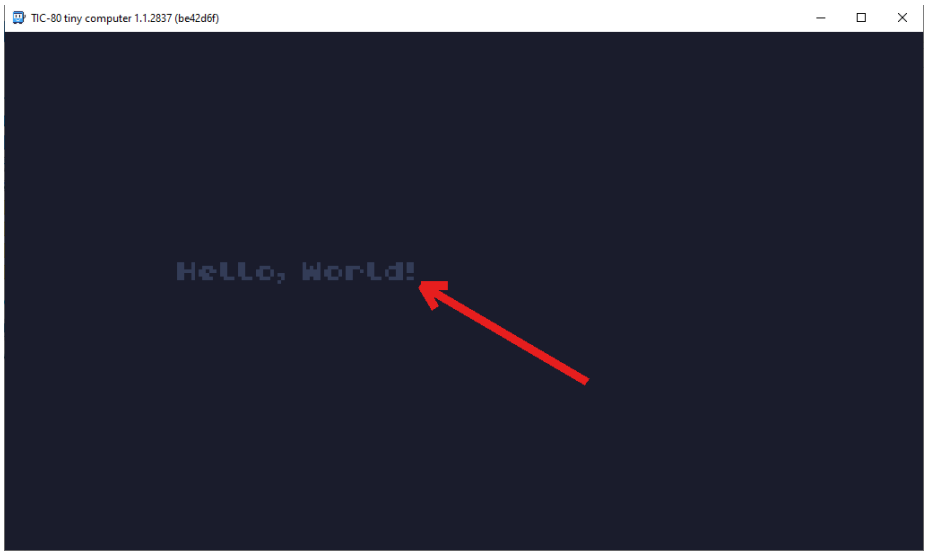
1) mkdir HelloWorld creates the project folder, 2) dir confirms it's there, 3) cd HelloWorld moves into it, 4) new lua creates a new cart and opens the Code Editor.



The default starter demo code - 1) delete these variables, 2) and this whole function TIC() body underneath them.



The new code in place - the print line is what draws "Hello, World!" to the screen.



Success - "Hello, World!" appears on screen.

7. 001 Game Creator (Windows only)

Detailed Install

1. In your Steam Library, select 001 Game Creator (Windows only, no Mac/Linux version exists) and click INSTALL.
2. Confirm the install options (Create desktop shortcut and Create start menu shortcut are both fine to leave checked) and click Install.
3. Review and click Accept on the End-User License Agreement.
4. Wait for Steam to download and install (~2.4 GB).
5. Once installed, click Launch from the Library page.

Hello, World! Verification

1. 001 Game Creator shows its own splash screen while it loads.
2. A New Game window may pop up automatically - click Cancel to close it for now; we want to set our save location first, and that option isn't available from this screen.
3. Go to File > Open Game... instead.
4. In the New / Open Game window, click into the path field at the bottom (it defaults to a Documents folder) and choose Browse... from the popup that appears, then select the folder where you'd like this class's projects saved. Once the path is set, click Create New Game.
5. This reopens the New Game window - 1) select Blank Template, 2) set Name of Project to HelloWorld, 3) click OK.
6. A Warning explains the Blank Template is intended for experienced users and doesn't include input triggers, menus, or actor templates - safe to ignore for this project; click Continue.
7. The main Workspace opens with your new HelloWorld project loaded in the left-hand project list.
8. Go to the Resources menu (not Game) and choose System Triggers.
9. Double-click Introduction - this system trigger runs automatically at the very start of the game.

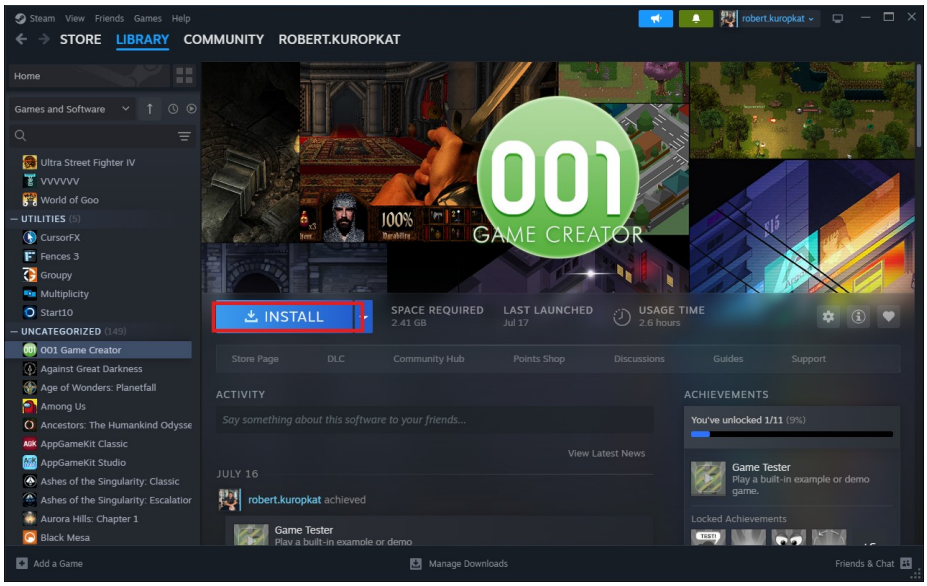
10. In the Script Editor, double-click Message Box in the events list on the right - a direct double-click drops it onto the script and connects it to Start automatically (dragging it out is not reliable and may not work).
11. In the Message Box properties, set Message to "Hello, World!", then click OK.
12. Click the Play (Test Game) button in the main toolbar to test.
13. A Testing Options window appears - the defaults are fine; confirm Emulation is set to None, then click OK to launch the test.

Expected result: A debugger panel and your game's window both open, with a message box on screen displaying "Hello, World!"

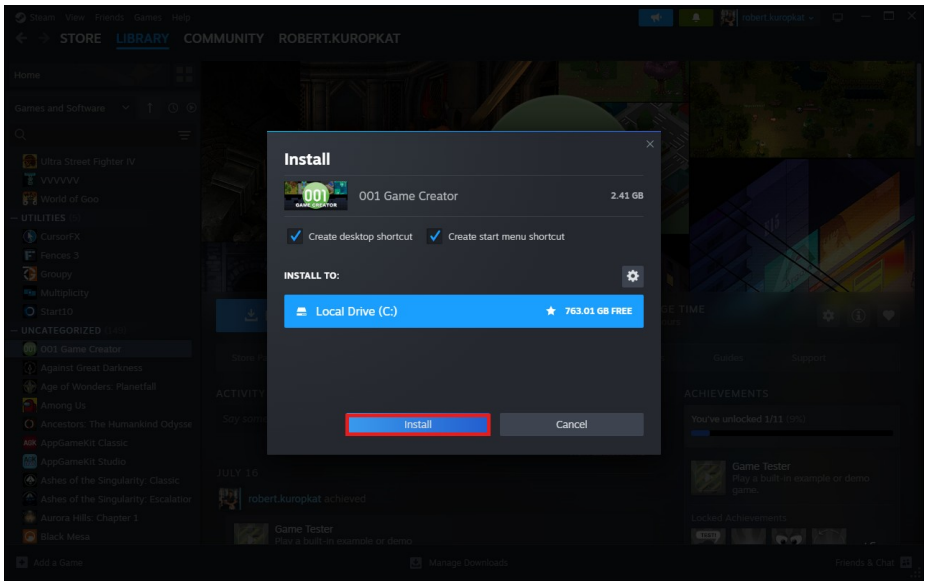
If something's wrong: *If nothing happens on Play, confirm the Message Box event is actually connected to the Start node with a visible line between them - double-clicking Message Box in the events list (rather than dragging it onto the canvas) connects it automatically, which is the reliable way to do this. If the New Game window keeps popping up before you can set your save folder, remember to close it (Cancel) and use File > Open Game instead - the save-location picker only appears on that screen, not on the New Game template screen.*

Visual Walkthrough (captured on 001 Game Creator 2025.011.000 via Steam)

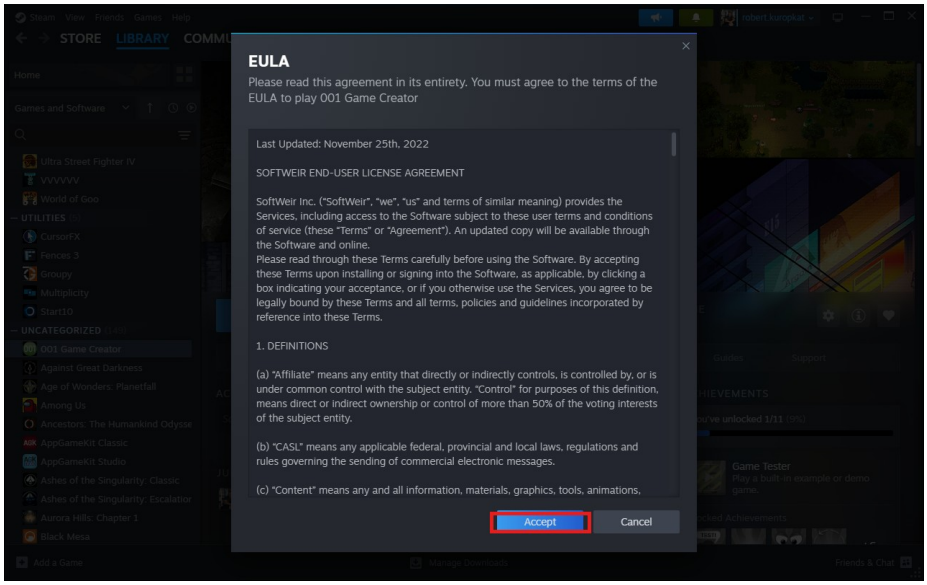
Screenshots from an actual run-through, in order. Circled/boxed elements mark exactly what to click or check at that step.



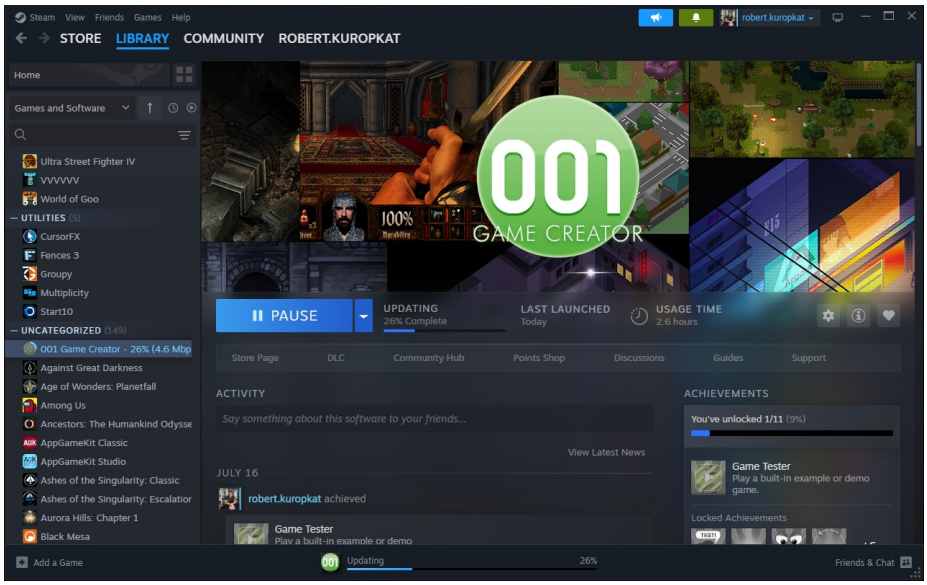
In your Steam Library, select 001 Game Creator and click INSTALL.



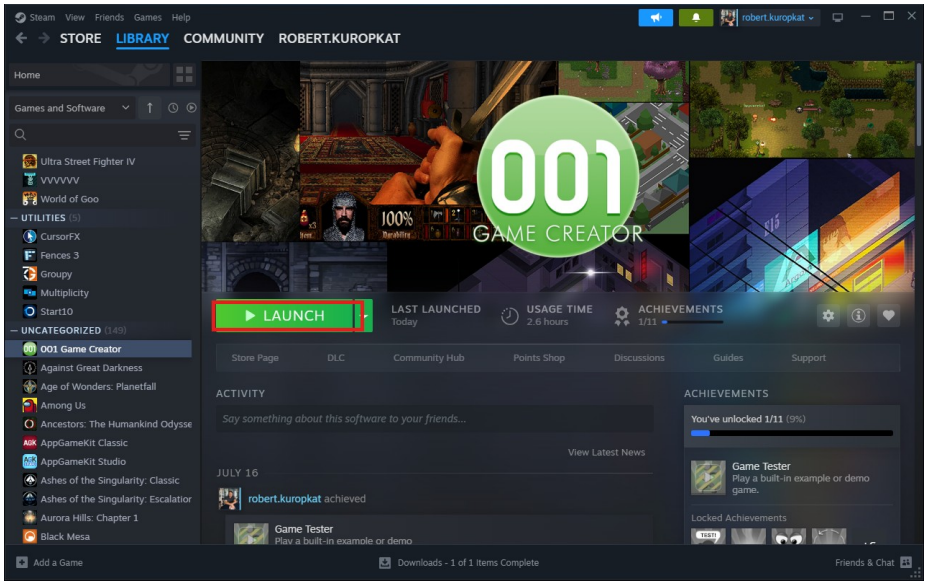
Confirm the install options and click Install.



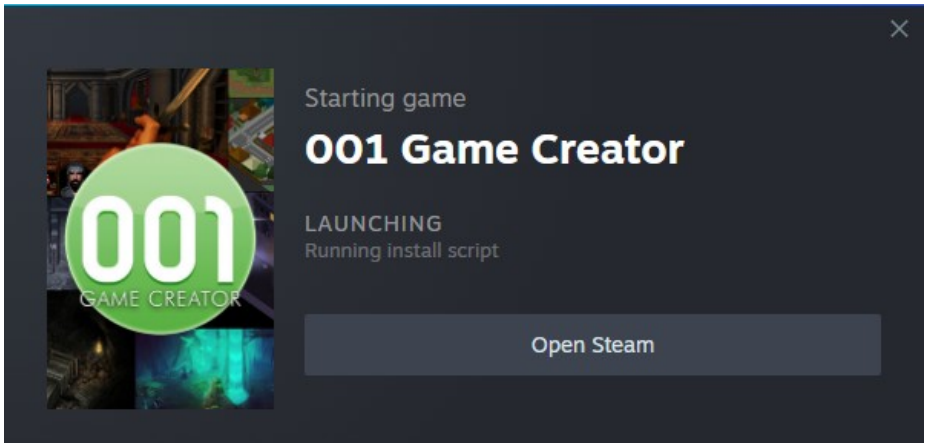
Review the EULA and click Accept.



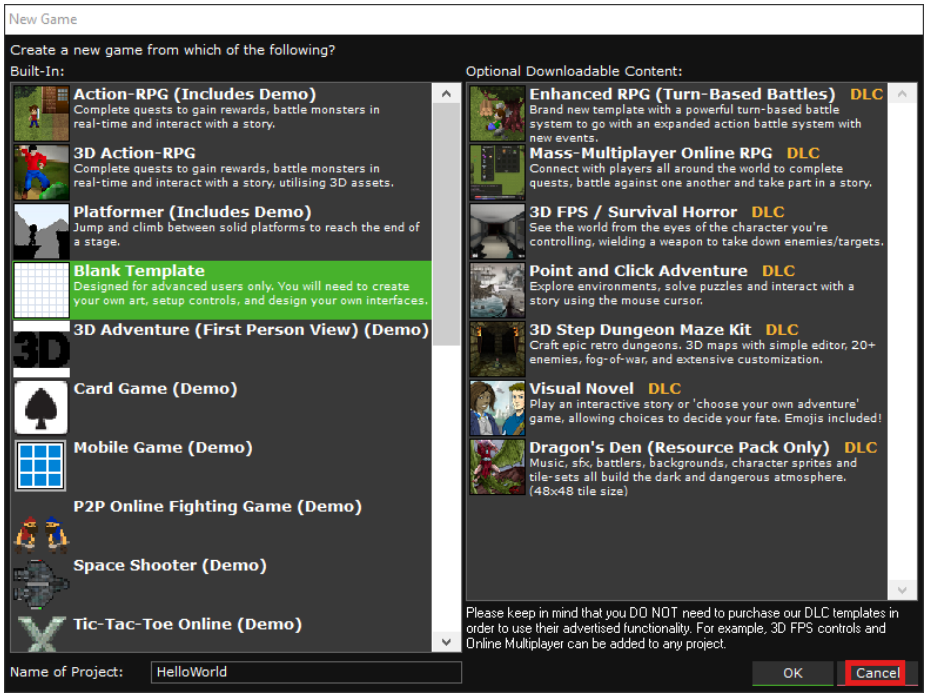
Steam downloads and installs (~2.4 GB).



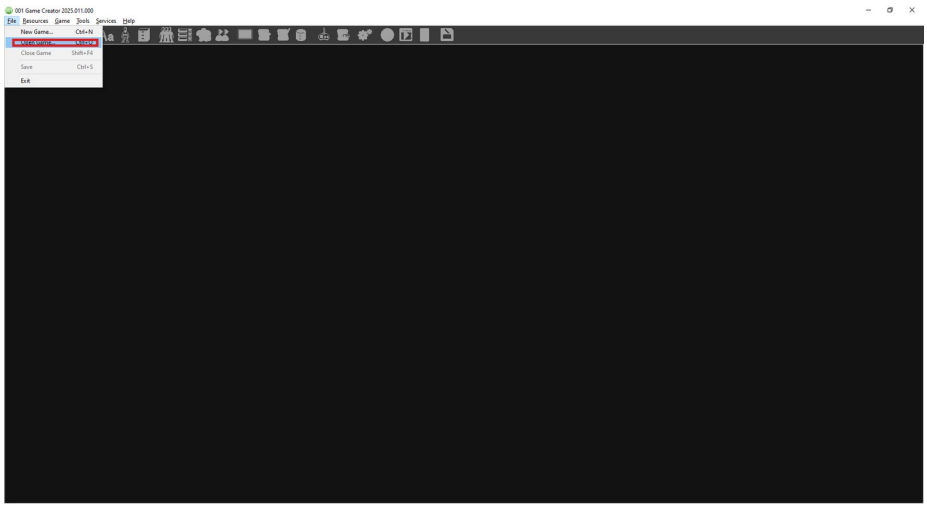
Once installed, click Launch to start 001 Game Creator.



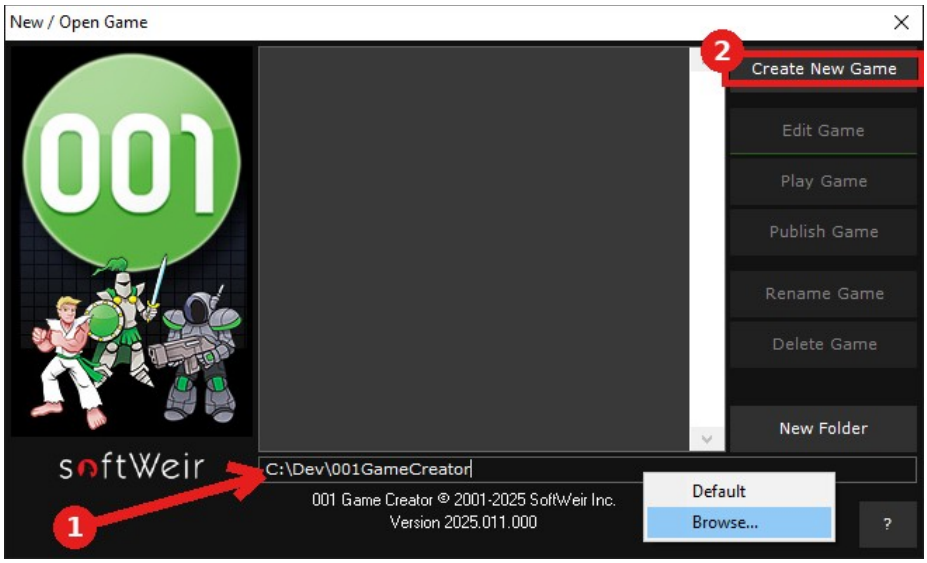
001 Game Creator's splash screen while it loads.



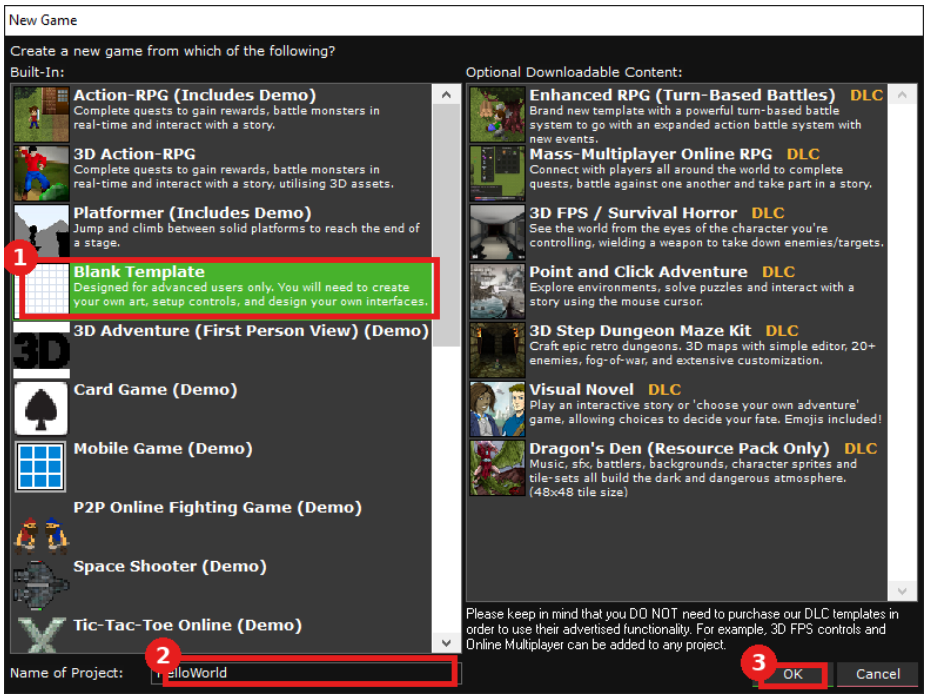
A New Game window may pop up automatically - click Cancel to close it for now.



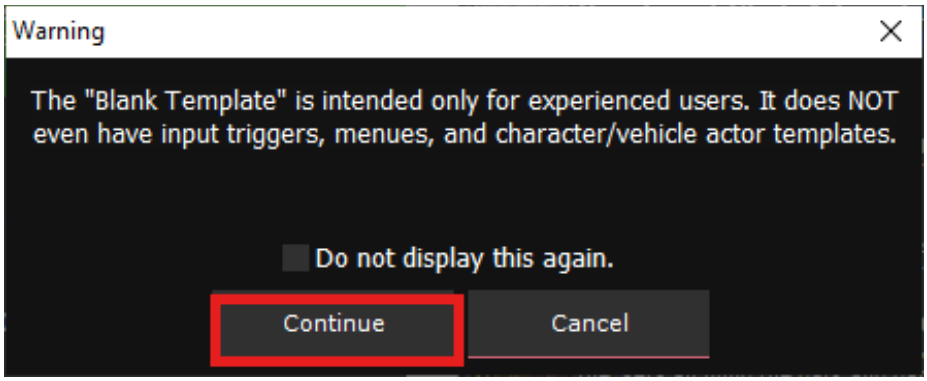
Go to File > Open Game... instead.



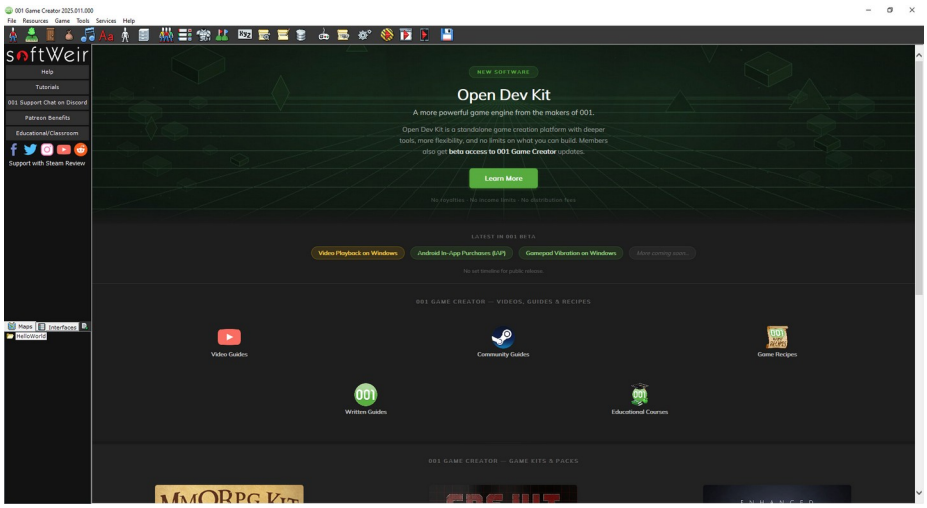
1) Click the path field and choose Browse... to set your save folder, 2) once it's set, click Create New Game.



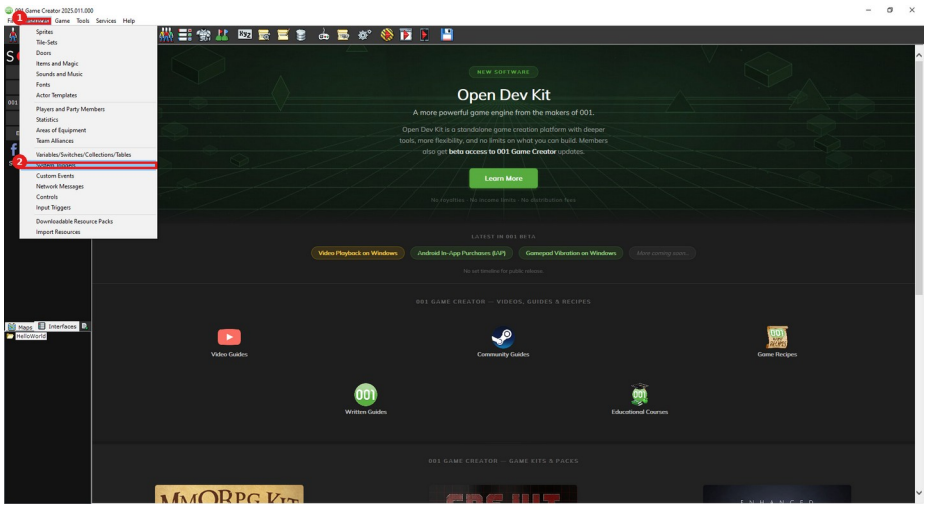
Back in the New Game window: 1) select Blank Template, 2) set Name of Project to HelloWorld, 3) click OK.



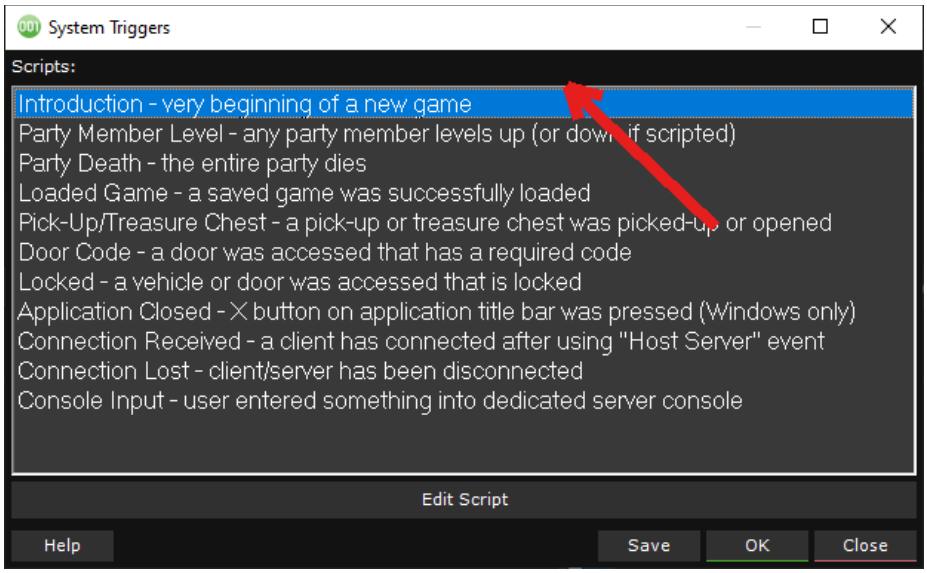
Safe to ignore for this project - click Continue.



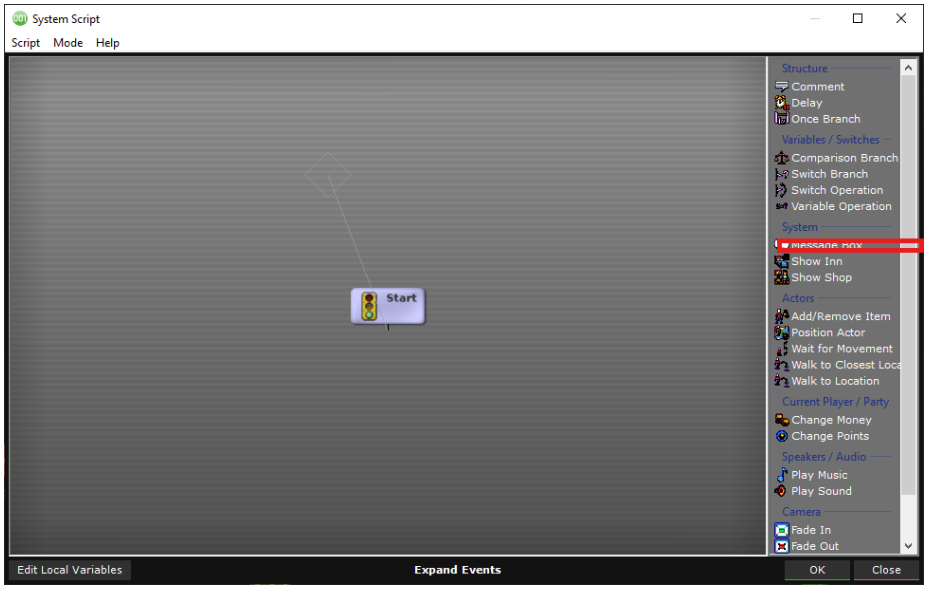
The main Workspace opens with your new HelloWorld project loaded on the left.



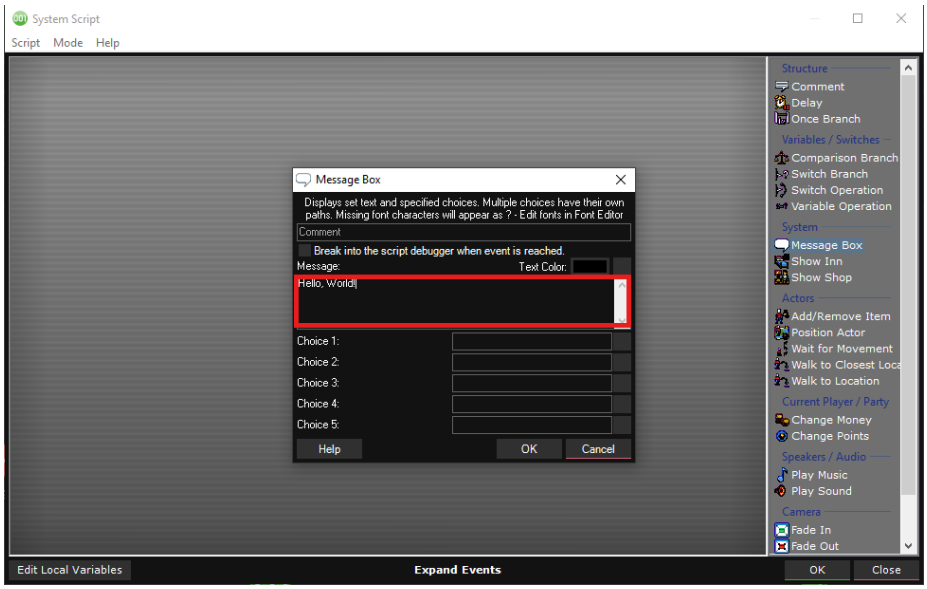
1) Open the Resources menu (not Game), 2) choose System Triggers.



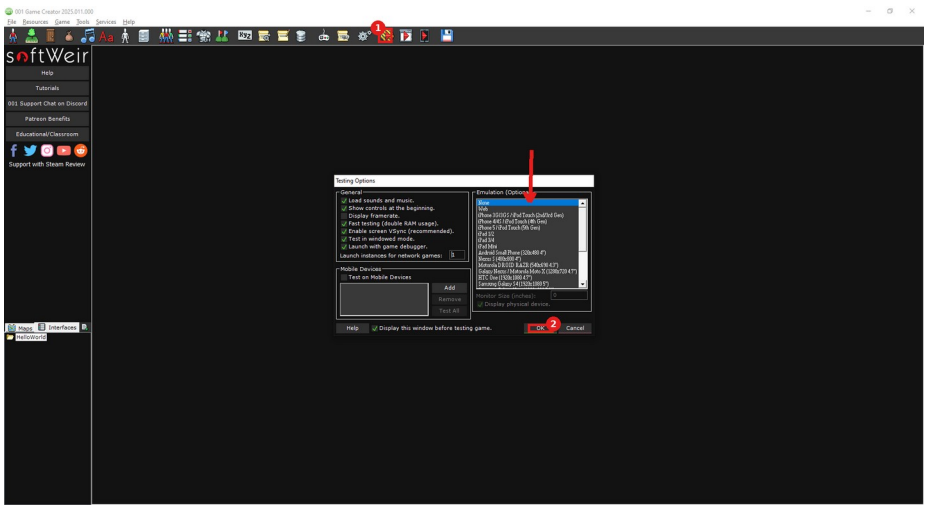
Double-click Introduction - this runs automatically at the very start of the game.



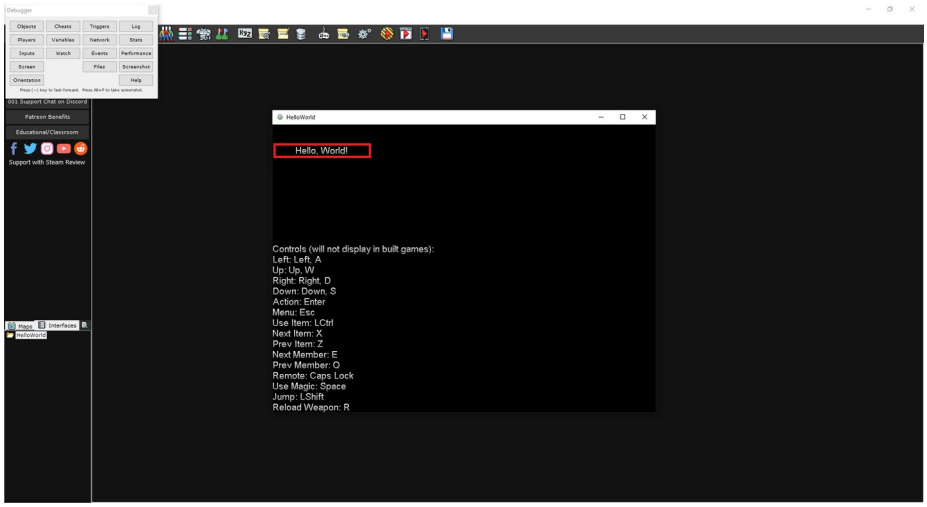
In the Script Editor, double-click Message Box in the events list on the right.



Set the Message field to "Hello, World!", then click OK.



1) Click the Play (Test Game) toolbar button, confirm Emulation is set to None, 2) click OK to launch the test.



Success - the message box displays "Hello, World!"

8. Unreal Engine (optional advanced track)

Detailed Install

1. Go to unrealengine.com and create a free Epic Games account if needed.
2. Download and install the Epic Games Launcher.
3. Sign in, go to the Unreal Engine tab, click Install Engine, choose the latest version.
4. Allow the very large (tens of GB) download/install to finish - do this well ahead of time on a strong connection, ideally days before class.

Hello, World! Verification

1. Launch Unreal Engine. In current versions this opens its own standalone app (separate from the Epic Games Launcher itself), with a Home screen - click New Project.
2. Choose the Games category > Blank template, confirm BLUEPRINT is selected (not C++), set the Project Name to HelloWorld, then click Create.
3. Once the editor opens, click the Blueprints icon in the main toolbar and choose Open Level Blueprint from the menu.
4. In the Event Graph, note that an Event BeginPlay node is already there by default - no need to add it.
5. Right-click in empty space, type "print" to search, and add Print String. Connect the white execution line from Event BeginPlay's output to Print String's input.
6. Click into the Print String node's In String field and set it to "Hello, World!"
7. Click Compile (its tooltip reads "Dirty; needs to be recompiled" until you do), close the Blueprint editor, then click Play in the main toolbar.
8. Save the level: File > Save Current (or Ctrl+S), name it HelloWorld. Confirm the saved name now appears in both the title bar and the Blueprint editor's tab.

Expected result: "Hello, World!" appears as on-screen text in the top-left corner of the Play window.

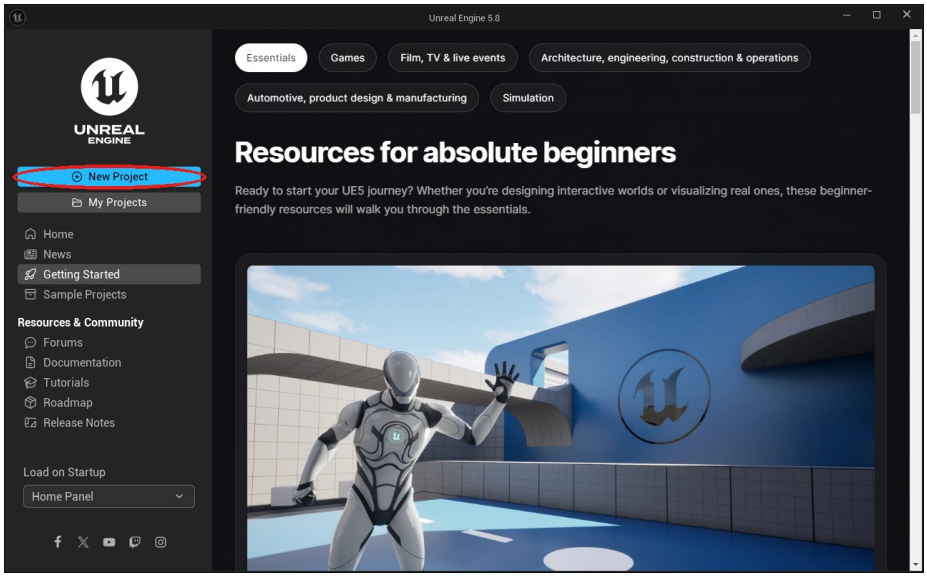
If something's wrong: *If nothing prints, confirm the white execution wire actually connects Event BeginPlay to Print String (not just placed nearby), and that you clicked Compile before pressing Play. If a "Missing support for advanced rendering features" dialog appears when the project first loads (about DirectX 12 / SM6 not being supported), this is expected on some hardware and safe to dismiss with OK - it only disables optional features like Nanite and Virtual Shadow Maps, which this class doesn't use.*

Visual Walkthrough (captured on Unreal Editor 5.8)

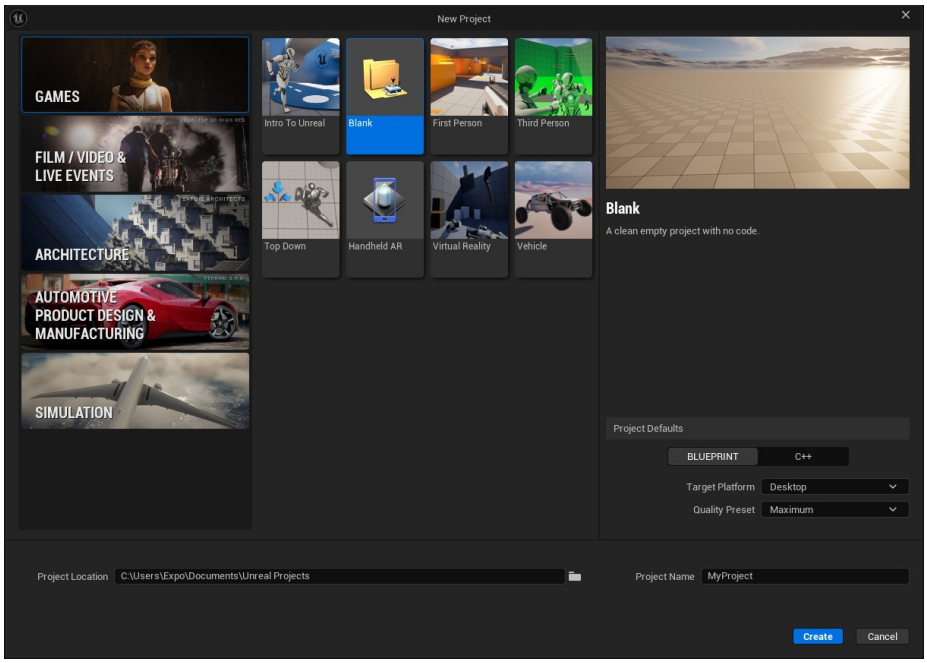
Screenshots from an actual run-through, in order. Circled/boxed elements mark exactly what to click or check at that step.



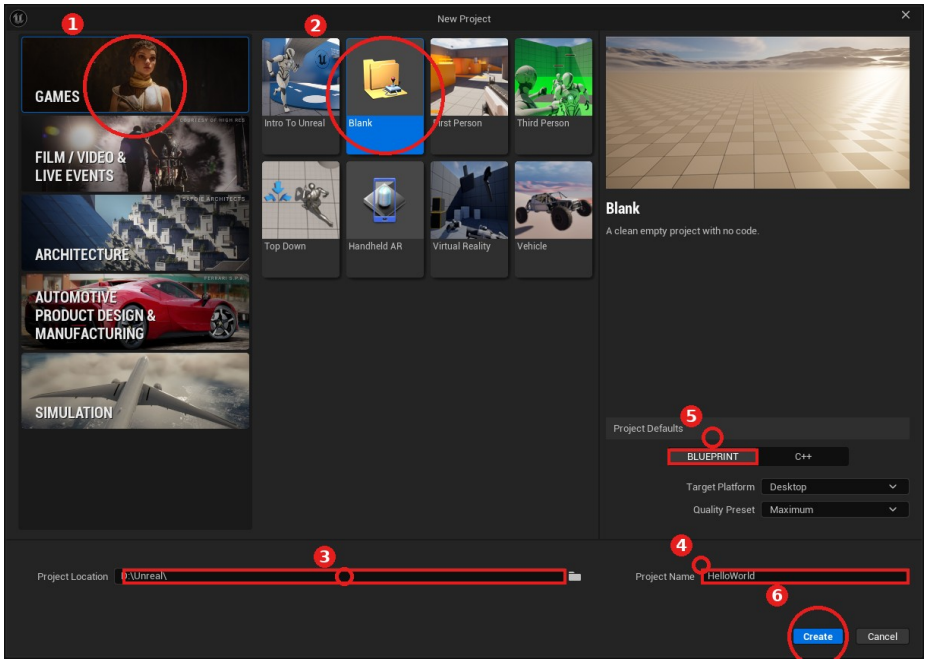
The Unreal Engine app's own loading splash.



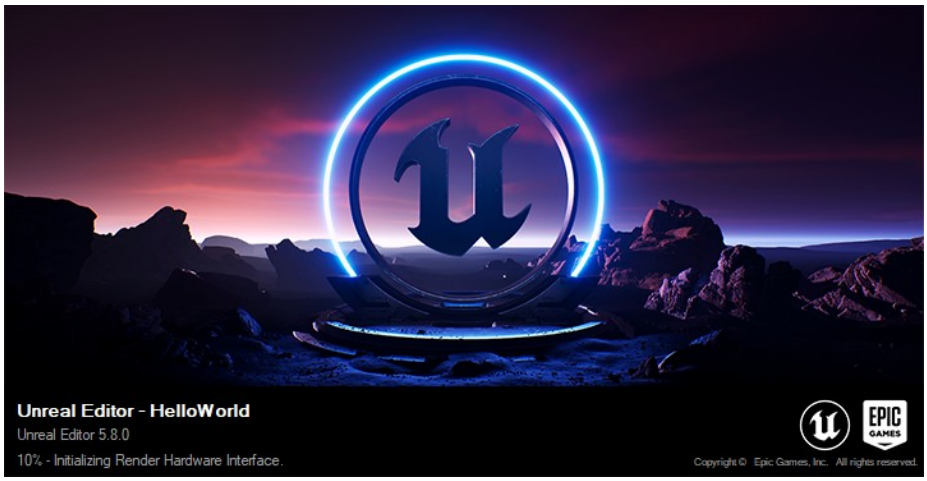
The app's Home screen - click New Project.



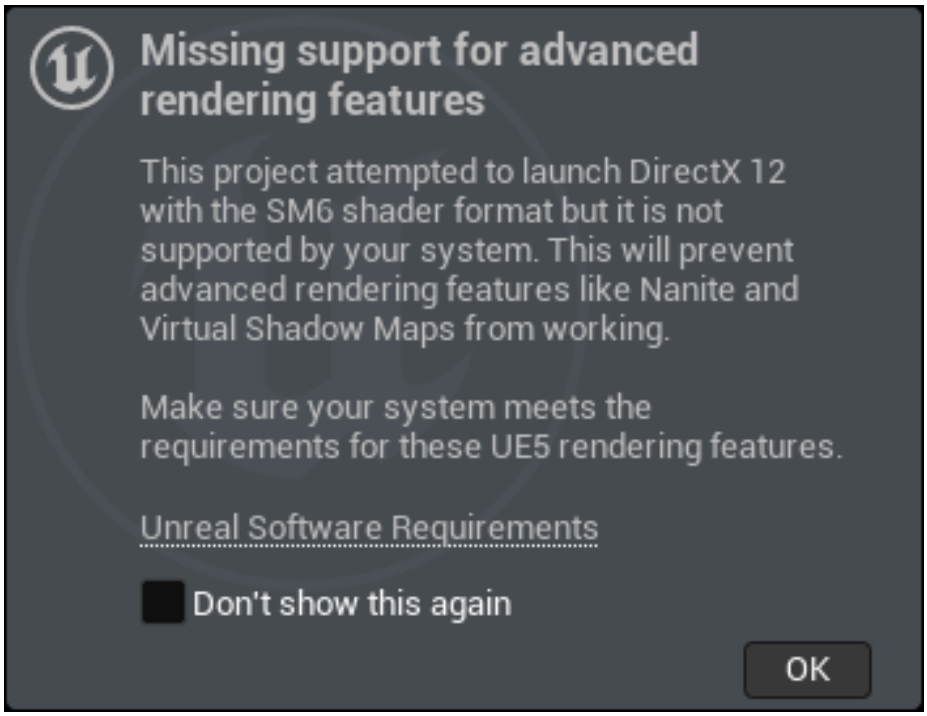
The New Project screen with its default values.



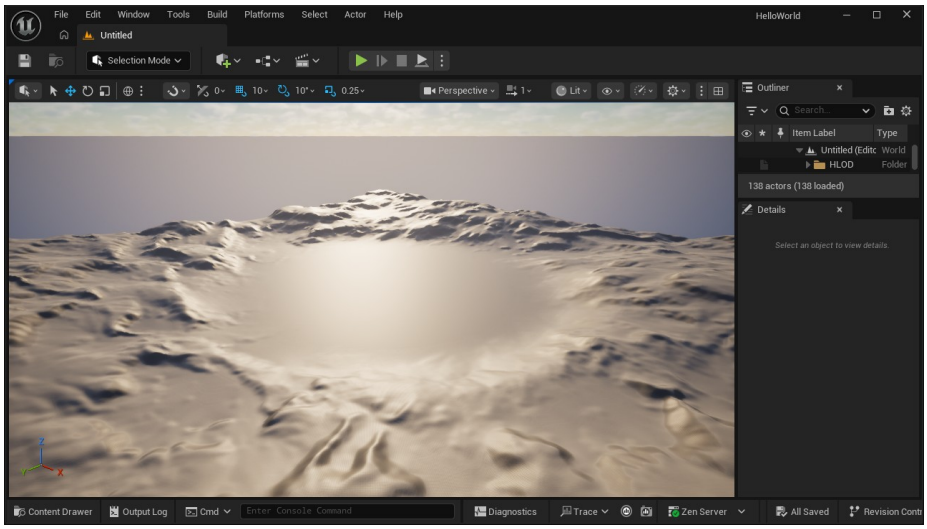
1) Games category, 2) Blank template, 3) Project Location, 4) Project Name set to HelloWorld, 5) BLUEPRINT selected (not C++), 6) Create.



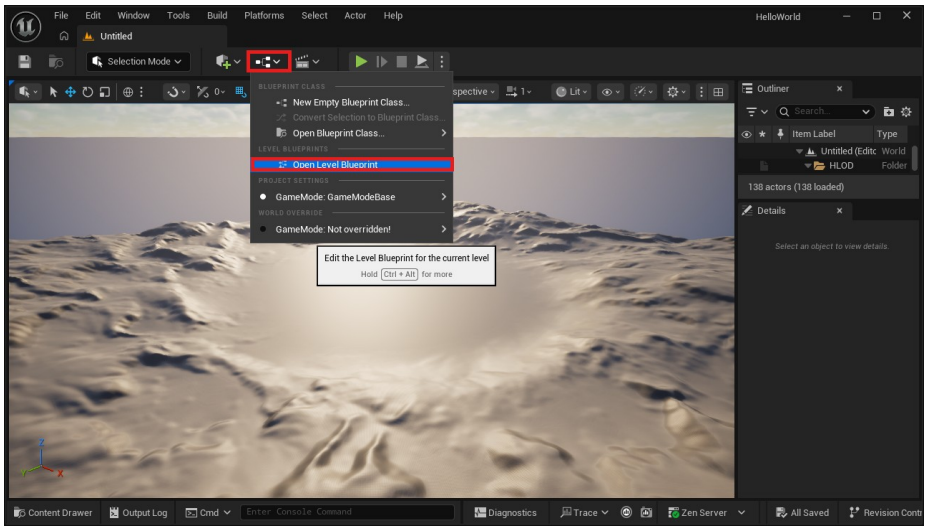
The full Editor loading the new HelloWorld project.



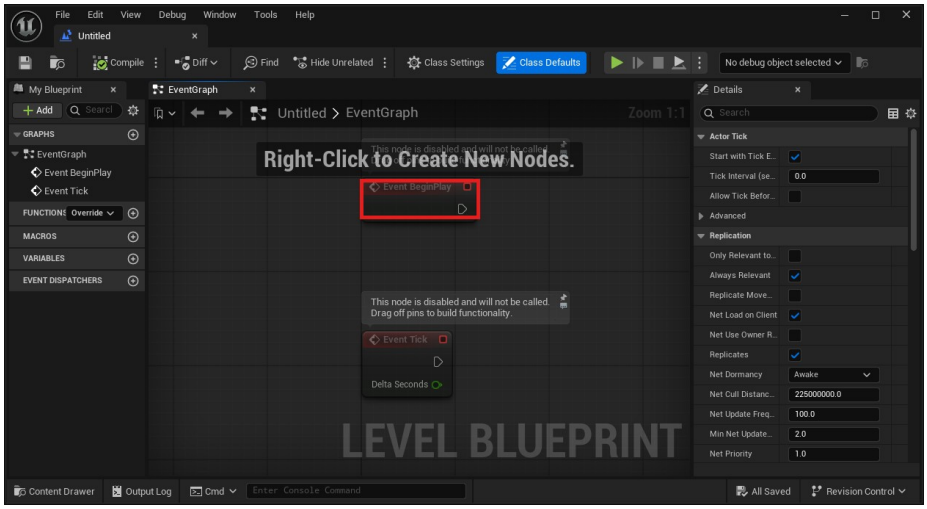
A DirectX 12/SM6 warning some hardware shows on first load - safe to click OK (see Troubleshooting).



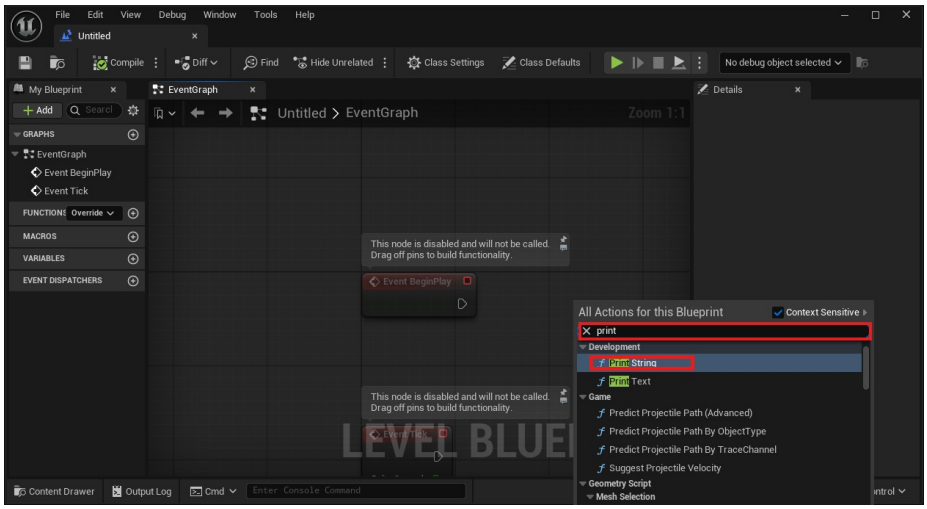
The Level Editor, open and ready.



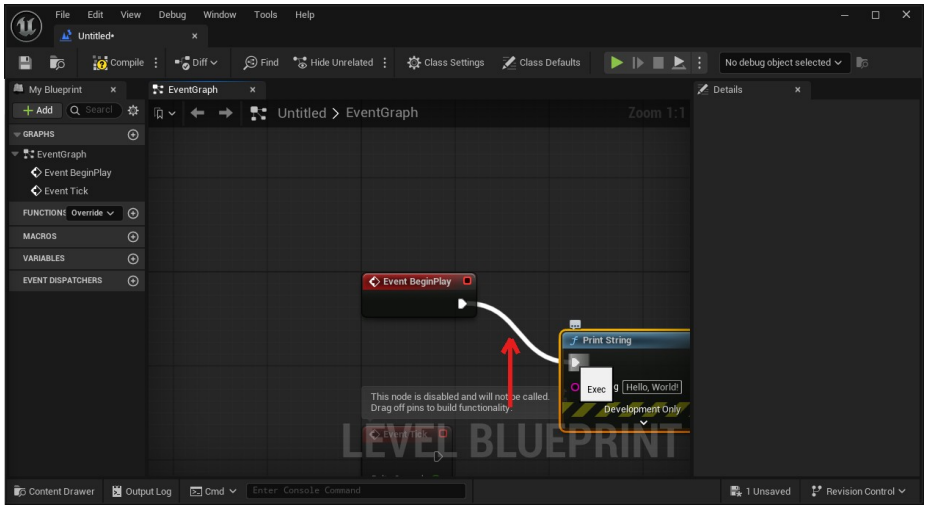
Open Level Blueprint, found under the Blueprints toolbar menu.



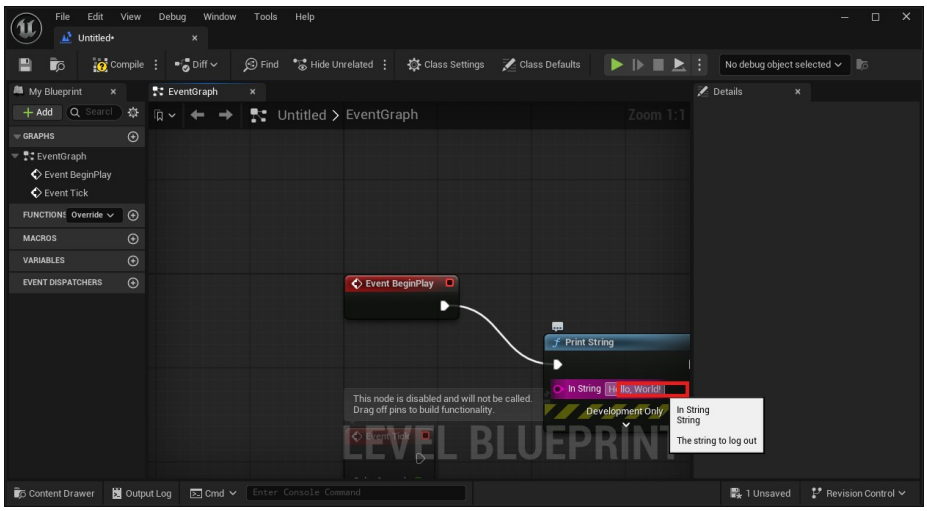
The Level Blueprint's Event Graph - Event BeginPlay is already there by default.



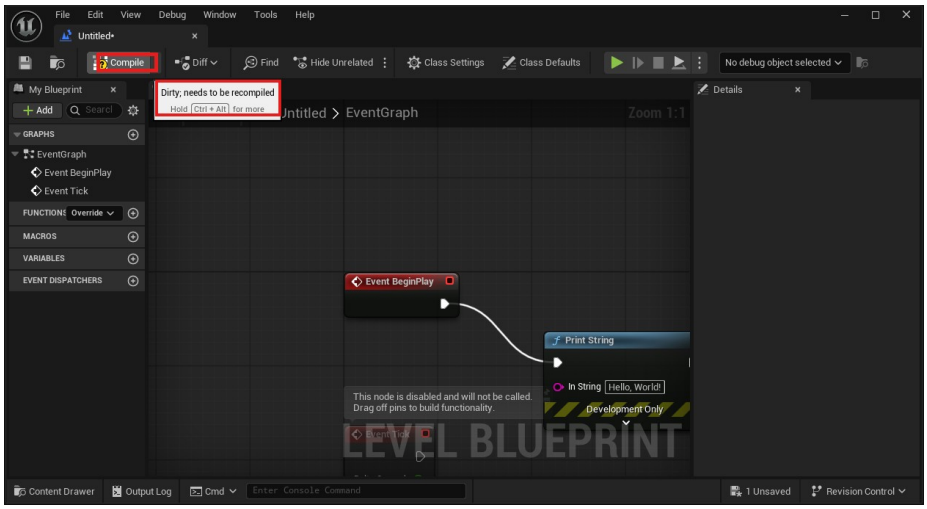
Right-click and search "print" to find and add Print String.



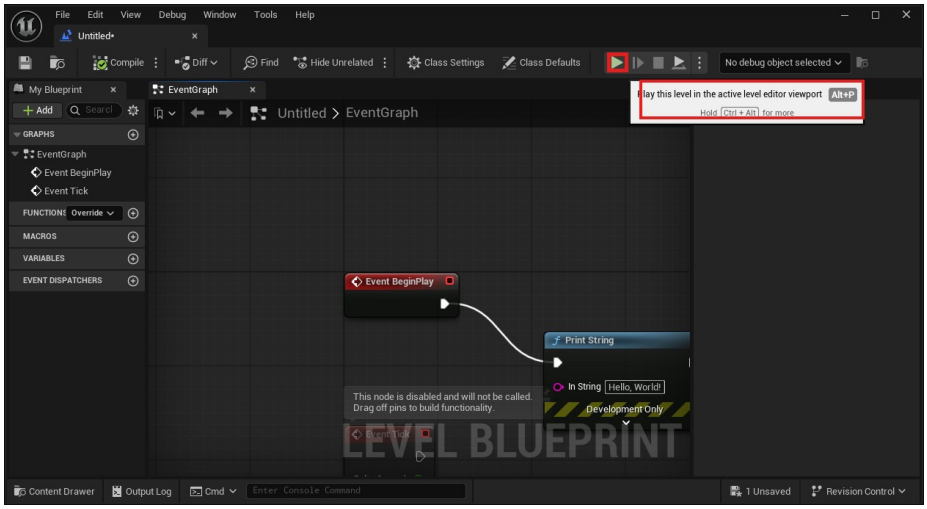
The white execution wire connecting Event BeginPlay to Print String.



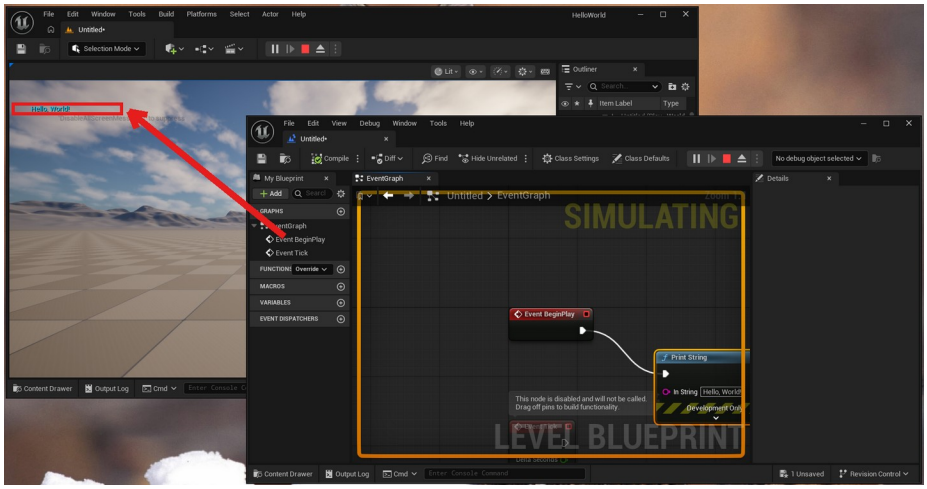
The In String field, set to "Hello, World!"



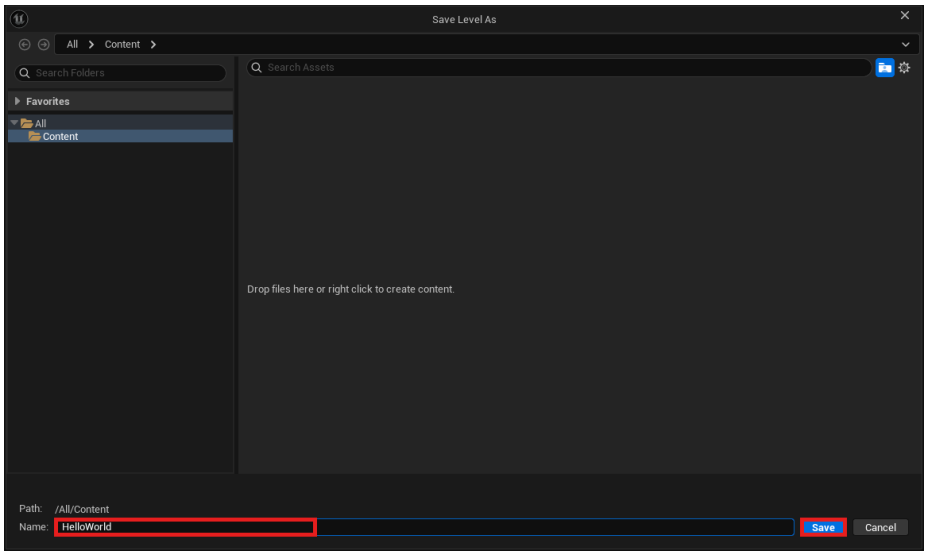
The Compile button - its tooltip warns "Dirty; needs to be recompiled" until you click it.



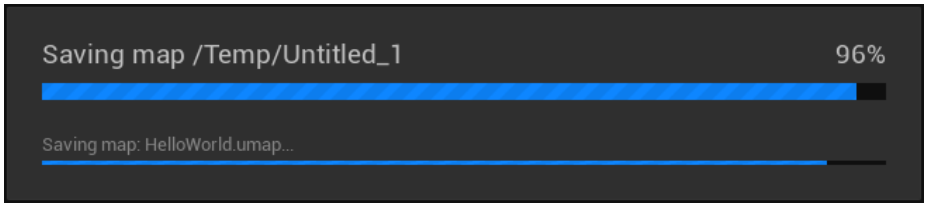
The Play button, ready to test.



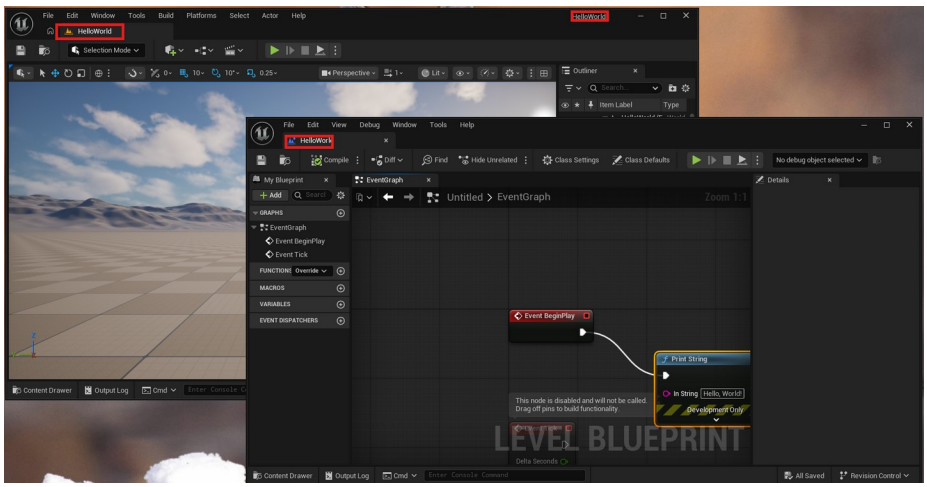
Success - "Hello, World!" printed in the top-left of the Play window.



Save Level As, with the name set to HelloWorld.



The save-in-progress indicator.



Confirmed: HelloWorld now appears in the title bar, the Blueprint editor's own tab, and the Game window's tab in the background.

Instructor Verification Checklist

Fill this in as you confirm each engine on your demo laptop. Re-check before Lesson 1 each year, and again mid-year if any engine has pushed a major update.

Engine	Version Installed	Hello World Confirmed?	Date Tested	Notes
Scratch				
Godot Engine				
Unity				
GameMaker				
GDevelop				
TIC-80				
001 Game Creator (Windows only)				
Unreal Engine (optional advanced track)				